

TECNOLOGIA DA INFORMAÇÃO

Linguagem CSS



Presidente: Gabriel Granjeiro

Vice-Presidente: Rodrigo Calado

Diretor Pedagógico: Erico Teixeira

Diretora de Produção Educacional: Vivian Higashi

Gerente de Produção Digital: Bárbara Guerra

Todo o material desta apostila (incluindo textos e imagens) está protegido por direitos autorais do Gran. Será proibida toda forma de plágio, cópia, reprodução ou qualquer outra forma de uso, não autorizada expressamente, seja ela onerosa ou não, sujeitando-se o transgressor às penalidades previstas civil e criminalmente.

CÓDIGO:

250828508280



PATRÍCIA QUINTÃO

Mestre em Engenharia de Sistemas e computação pela COPPE/UFRJ, Especialista em Gerência de Informática e Bacharel em Informática pela UFV. Atualmente é professora no Gran Cursos Online; Analista Legislativo (Área de Governança de TI), na Assembleia Legislativa de MG; Escritora e Personal & Professional Coach. Atua como professora de Cursinhos e Faculdades, na área de Tecnologia da Informação, desde 2008. É membro: da Sociedade Brasileira de Coaching, do PMI, da ISACA, da Comissão de Estudo de Técnicas de Segurança (CE-21:027.00) da ABNT, responsável pela elaboração das normas brasileiras sobre gestão da Segurança da Informação. Autora dos livros: Informática FCC – Questões comentadas e organizadas por assunto, 3ª. edição e 1001 questões comentadas de informática (Cespe/UnB), 2ª. edição, pela Editora Gen/Método. Foi aprovada nos seguintes concursos: Analista Legislativo, na especialidade de Administração de Rede, na Assembleia Legislativa do Estado de MG; Professora titular do Departamento de Ciência da Computação do Instituto Federal de Educação, Ciência e Tecnologia; Professora substituta do DCC da UFJF; Analista de TI/Suporte, PRODABEL; Analista do Ministério Público MG; Analista de Sistemas, DATAPREV, Segurança da Informação; Analista de Sistemas, INFRAERO; Analista – TIC, PRODEMGE; Analista de Sistemas, Prefeitura de Juiz de Fora; Analista de Sistemas, SERPRO; Analista Judiciário (Informática), TRF 2ª Região RJ/ES, etc.

GRAN
CONCURSOS

SUMÁRIO

Apresentação	6
Linguagem CSS	7
1. Termos, Definições e Abreviaturas.....	7
2. Fundamentos do CSS.....	8
2.1. Introdução ao CSS.....	8
2.2. Métodos de Aplicação.....	11
2.3. Cascata e Prioridade de Estilos.....	13
2.4. Sintaxe Básica	15
2.5. Esquemas Visuais Consolidados	17
3. Seletores Essenciais.....	18
3.1. Introdução aos Seletores.....	18
3.2. Seletores Básicos	19
3.3. Especificidade Básica	23
3.4. Esquemas Visuais Consolidados	25
4. Box Model	26
4.1. Conceitos do Box Model	26
4.2. Propriedades de Dimensão	27
4.3. Margens (Margins).....	29
4.4. Bordas (Border)	31
4.5. Preenchimento (Padding)	33
4.6. Cálculos Práticos de Dimensões	35
4.7. Esquemas Visuais Consolidados	36
5. Cores e Backgrounds	37
5.1. Sistema de Cores CSS.....	37
5.2. Propriedades de Background.....	40
5.3. Esquemas Visuais Consolidados	44

6. Tipografia e Texto	45
6.1. Propriedades de Texto	45
6.2. Propriedades de Fonte	50
6.3. Recursos CSS3 para Tipografia	55
6.4. Esquemas Visuais Consolidados	57
7. Display e Posicionamento	58
7.1. Propriedade Display	58
7.2. Propriedade Posicionamento (Position).	62
7.3. Z-Index e Empilhamento	64
7.4. Float e Clear.	66
7.5. Esquemas Visuais Consolidados	67
8. Flexbox e Grid	69
8.1. Flexbox – Layout Unidimensional	69
8.2. Grid – Layout Bidimensional	71
8.3. Flexbox vs Grid	74
8.4. Esquemas Visuais Consolidados	75
9. Responsividade	76
9.1. Conceitos Fundamentais	76
9.2. Media Queries	78
9.3. Breakpoints Padrão.	80
9.4. Mobile-First Approach	81
9.5. Unidades Responsivas	82
9.6. Técnicas Responsivas Avançadas	84
9.7. Esquemas Visuais Consolidados	85
10. Seletores Avançados	86
10.1. Combinadores	86
10.2. Pseudo-Classes	88
10.3. Pseudo-Elementos	91

10.4. Seletores de Atributo	94
10.5. Especificidade Avançada	95
10.6. Esquemas Visuais Consolidados	97
Resumo	99
Questões de Concurso.	106
Gabarito	116
Gabarito Comentado.	117
Referências	141

APRESENTAÇÃO

Olá, querido (a) amigo(a), tudo bem?

“O Poder do Entusiasmo”: Descubra o entusiasmo na Vida!!

Nós somos capazes de transformar as coisas e fazê-las acontecer. Faça o entusiasmo ocorrer pela certeza de que você é capaz de realizações eficazes e de... **VENCER OBSTÁCULOS E SER FELIZ!!**

Trago aqui a aula sobre **CSS (CASCADING STYLE SHEETS)** e tópicos relacionados. Espero que aproveitem!

Continue firme nessa caminhada e ótimos estudos!

Um abraço,

Profª Patrícia Lima Quintão



LINGUAGEM CSS

1. TERMOS, DEFINIÇÕES E ABREVIATURAS

A seguir destacamos alguns **termos/definições e abreviaturas**:

- **Animation:** Conjunto de regras para criar animações complexas no CSS, definindo quadros-chave e transições.
- **Box Model:** Modelo de caixa do CSS que define como elementos são renderizados em camadas: conteúdo, padding, border e margin.
- **Box-sizing:** Propriedade que define como largura e altura são calculadas, considerando ou não padding e border.
- **Cascade:** Mecanismo que determina a ordem de aplicação das regras CSS, considerando origem, especificidade e ordem no código.
- **Clamp:** Função CSS que define valores mínimos, ideais e máximos para propriedades, facilitando layouts responsivos.
- **Combinator:** Símbolos que estabelecem relações hierárquicas ou posicionais entre seletores (>, +, ~, espaço).
- **CSS (Cascading Style Sheets):** Linguagem usada para definir a apresentação visual de páginas web, permitindo personalizar cores, fontes, tamanhos, espaçamentos e layouts.
- **DOM (Document Object Model):** Representação em árvore do documento HTML que pode ser estilizada por CSS e manipulada por JavaScript.
- **External Style Sheet:** Arquivo separado com extensão.css, vinculado ao HTML para aplicar estilos de forma centralizada.
- **Flexbox:** Módulo CSS para criar layouts unidimensionais, organizando elementos em linha ou coluna com flexibilidade.
- **Grid:** Módulo CSS para criar layouts bidimensionais, permitindo organizar elementos em linhas e colunas.
- **HTML (HyperText Markup Language):** Linguagem de marcação usada para estruturar conteúdo na web.
- **Inheritance (Herança):** Característica do CSS que permite que algumas propriedades sejam herdadas pelos elementos filhos.
- **Inline Style:** Estilo CSS aplicado diretamente dentro de uma tag HTML por meio do atributo *style*.
- **Inherit:** Valor de propriedade CSS que força um elemento a herdar o valor da propriedade de seu elemento pai.
- **Media Query:** Regras CSS que aplicam estilos diferentes com base em características do dispositivo (tamanho de tela, resolução, etc.).

- **Media Types:** Tipos de mídia que indicam em quais dispositivos ou contextos certas regras CSS devem ser aplicadas (screen, print, all).
- **Pseudo-class:** Palavra-chave adicionada a seletores para definir estilos em estados especiais de um elemento (ex.: hover).
- **Pseudo-element:** Permite estilizar partes específicas de um elemento ou inserir conteúdo virtual (ex.: before, after).
- **Rem/Em:** Unidades relativas. *Em* é relativo ao elemento pai, enquanto *Rem* é relativo à fonte raiz do documento.
- **Reset CSS:** Técnica usada para remover ou uniformizar estilos padrão aplicados pelos navegadores.
- **Responsive Design (Design Responsivo):** Abordagem de desenvolvimento que adapta o layout a diferentes dispositivos e tamanhos de tela.
- **Shorthand Property:** Propriedade abreviada que resume várias declarações CSS em uma única linha (ex.: margin, padding, border).
- **Specificity (Especificidade):** Algoritmo que calcula a prioridade entre seletores para determinar qual regra será aplicada.
- **Tag:** Elemento básico do HTML que define a estrutura e o significado do conteúdo (ex.: <p>, <h1>).
- **Transform:** Propriedade que permite aplicar rotação, escala, inclinação e translação a elementos.
- **Transition:** Propriedade usada para criar efeitos de transição suave entre dois estados de um elemento.
- **VH/VW:** Unidades relativas à altura (vh) e largura (vw) da *viewport*, a área visível do navegador.
- **Viewport:** Área visível da página em um dispositivo, que varia conforme tamanho e resolução da tela.
- **Z-index:** Propriedade CSS que controla a ordem de empilhamento de elementos no eixo Z (profundidade).

2. FUNDAMENTOS DO CSS

2.1. INTRODUÇÃO AO CSS

2.1.1. O QUE É CSS?

O **CSS** é reconhecido como uma **linguagem de estilo** utilizada por desenvolvedores para dar dinâmica, personalização e efeitos a páginas da web e aplicativos construídos sobre **linguagens de marcação**, como o **HTML**.



Uma linguagem de estilo tem como objetivo realçar a interface e a aparência de um documento baseado em marcação. Nesse sentido, o CSS atua como um complemento ao HTML, tornando-se essencial nos processos mais comuns de desenvolvimento web.

A sigla **CSS** vem do inglês **Cascading Style Sheets**, que pode ser traduzido como **folhas de estilo em cascata**. Na prática, funciona como uma camada de personalização aplicada ao conteúdo visível.

O CSS permite separar o conteúdo do HTML das regras de apresentação visual, facilitando a manutenção e atualização da página. Assim, **o desenvolvedor pode alterar o design sem modificar a estrutura ou os componentes de marcação**.

Além disso, ele possibilita mudanças rápidas no layout e no formato dos conteúdos, definindo como serão exibidos em cada página. **Com o CSS, é possível configurar:**

- **fontes tipográficas do texto;**
- **tamanhos e proporções dos elementos;**
- **posicionamento dos itens na página;**
- **efeitos visuais**, como transições e animações (por exemplo, quando um ícone ganha destaque ao passar o mouse sobre ele);
- **dentre outras** inúmeras possibilidades.

Em uma analogia prática, imagine construir uma casa, em que:

- **HTML** = Estrutura (Paredes, portas, janelas, fundação) ⇒ **Linguagem de marcação**.

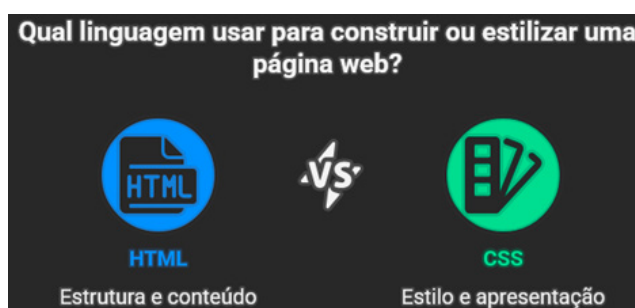
EXEMPLO

```
<h1>Título</h1>
```

- **CSS** = Acabamentos (Pintura, decoração, iluminação) ⇒ **Linguagem de Estilo**

EXEMPLO

```
h1{color: blue;}
```





Vantagens da separação HTML/CSS:

- **manutenção facilitada:** alterar visual sem mexer na estrutura;
- **reutilização:** um CSS pode ser usado em várias páginas;
- **consistência visual:** padrão uniforme em todo o site;
- **performance:** arquivo CSS é carregado uma vez e reutilizado;
- **acessibilidade:** diferentes estilos para diferentes necessidades;
- **trabalho em equipe:** desenvolvedores podem trabalhar separadamente.

2.1.2. EVOLUÇÃO HISTÓRICA

A evolução do **CSS** pode ser entendida em uma **linha do tempo**:

- **CSS1 (1996):** primeira versão, limitada a estilos básicos como fontes, cores e alinhamento.
- **CSS2 (1998):** introduziu posicionamento absoluto, relativo e fixo, além de suporte a diferentes mídias e maior flexibilidade no layout.
- **CSS2.1 (2004–2011):** revisão que corrigiu inconsistências entre navegadores e consolidou a base para o CSS moderno.
- **CSS3 (2005 em diante):** grande marco da linguagem, passou a ser modular e trouxe recursos como bordas arredondadas, sombras, gradientes, animações, transformações 2D/3D, flexbox, media queries e seletores avançados.
- **CSS4 (2017 em diante):** **não existe como versão oficial**, mas sim como a **evolução contínua dos módulos do CSS3**. Entre as novidades modernas estão o Grid Layout, variáveis CSS, novos seletores (:is(),:where(),:has()), consultas a contêineres (container queries) e funções matemáticas como clamp(), min() e max().

Versão	Ano	Principais Recursos
CSS1	1996	Propriedades básicas (cores, fontes, margens)
CSS2	1998	Posicionamento, media types, z-index
CSS2.1	2004	Correções e melhorias do CSS2
CSS3	2005+	Módulos (flexbox, grid, animações, gradientes)

Quando se usa o termo **CSS** se refere à linguagem em geral, enquanto **CSS3** indica o conjunto de módulos mais modernos que trouxe grandes avanços, como **bordas arredondadas, sombras, gradientes, transições, animações, flexbox e media queries para responsividade**. Hoje, ao falar em CSS, normalmente já se considera todas as funcionalidades do CSS3, pois são suportadas pelos navegadores modernos.

DICA

CSS3 não é uma versão única, mas um conjunto de módulos independentes que evoluem separadamente.

2.2. MÉTODOS DE APLICAÇÃO

Existem três formas de aplicar CSS aos elementos HTML:

- **CSS Inline (No Elemento)**
- **CSS Interno (Na Página)**
- **CSS Externo (Arquivo Separado)**

2.2.1. CSS INLINE (NO ELEMENTO)

CSS aplicado diretamente no **elemento HTML** usando o **atributo style**.

Sintaxe:

```
<elemento style="propriedade: valor;">
```

Exemplo:

```
<p style="color: red; font-size: 18px;">Texto vermelho</p>
<h1 style="background-color: blue; text-align: center;">Título centralizado</h1>
```

Vantagem: Prioridade máxima, efeito imediato.

Desvantagem: Dificulta manutenção, não é reutilizável

Uso: Estilos únicos e específicos

2.2.2. CSS INTERNO (NA PÁGINA)

CSS definido **dentro da tag <style> no <head>** do documento HTML.

Sintaxe:

```
<head>
  <style>
    seletor {
      propriedade: valor;
    }
  </style>
</head>
```

Exemplo:

```
<head>
  <style>
    h1 {
      color: blue;
      text-align: center;
    }
    p {
      font-family: Arial;
      line-height: 1.5;
    }
  </style>
</head>
```

Vantagem: Aplica-se a toda a página, mais organizado que inline.

Desvantagem: Não é reutilizável em outras páginas.

Uso: Estilos específicos de uma única página.

2.2.3. CSS EXTERNO (ARQUIVO SEPARADO)

CSS definido em **arquivo separado (.css)** e **vinculado ao HTML através da tag <link>**.

Sintaxe HTML:

```
<head>
  <link rel="stylesheet" type="text/css" href="estilos.css">
</head>
```

Arquivo estilos.css:

```
h1 {
  color: blue;
  text-align: center;
}

p {
  font-family: Arial;
  line-height: 1.5;
}
```

Vantagens: Reutilizável, facilita manutenção, melhor performance (carregamento mais rápido com cache)

Desvantagem: Requer arquivo adicional

Uso: Recomendado para site com múltiplas páginas

Para resumir, confira uma tabela comparativa com os principais prós e contras de cada método:

Método	Prós	Contras
CSS externo	Mais organizado e reutilizável	Exige mais arquivos
CSS interno	Simples e rápido de testar	Fica preso à página
CSS inline	Ágil para casos pontuais	Difícil de manter, sendo limitado para uso em larga escala

2.3. CASCATA E PRIORIDADE DE ESTILOS

O CSS funciona com base em regras, compostas por três partes principais: **seletor**, **propriedade** e **valor**. O seletor indica a quais elementos a regra se aplica. A propriedade define o que será alterado. E o valor aponta como será alterado.

2.3.1. O QUE É O CONCEITO DE CASCATA E ESPECIFICIDADE?

A palavra “**cascading**” (cascata) no CSS significa que as regras utilizadas neste tipo de linguagem de programação são aplicadas seguindo uma **hierarquia**.

O **algoritmo** em cascata é dividido em **4 estágios distintos**:

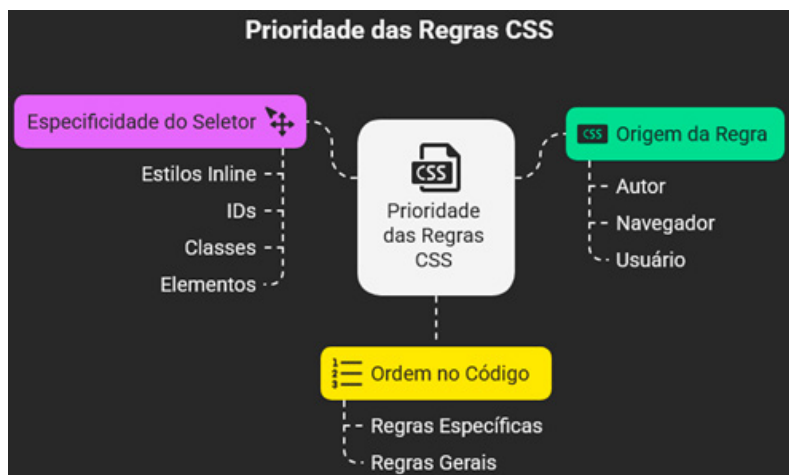
- **posição e ordem de aparência**: a ordem em que suas regras CSS aparecem;
- **especificidade**: um algoritmo que determina qual seletor CSS tem a correspondência mais forte;
- **origem**: a ordem de quando o CSS aparece e de onde vem, seja um estilo de navegador, CSS de uma extensão do navegador ou seu CSS de autoria;
- **importância**: algumas regras CSS são mais pesadas do que outras, especialmente com o tipo de regra **!important**.

Em outros termos, o CSS aplica as regras considerando **três fatores principais**:

- **Origem da regra (autor, navegador ou usuário);**
- **Especificidade do seletor;**
- **Ordem no código.**

Ordem de Prioridade (da maior para menor)

1. Estilos inline (atributo style)
2. IDs (#identificador)
3. Classes, atributos, pseudo-classes (.classe, [atributo],:hover)
4. Elementos e pseudo-elementos (p, h1,::before)
5. Estilos do navegador (padrão)



2.3.1.1. REGRA DA ÚLTIMA DECLARAÇÃO

Quando duas regras têm a mesma especificidade, a que aparece por último no código é aplicada.

Exemplo:

```
p { color: red; }
p { color: blue; } /* Esta será aplicada */
```

2.3.1.2. !IMPORTANT – QUEBRA DE PRIORIDADE

Com essa declaração, o CSS **força uma regra a ter prioridade máxima**.

Sintaxe:

```
p {
  color: red !important;
}
```

ATENÇÃO

Essa propriedade deve ser usada com cautela para não dificultar a manutenção. Um uso excessivo deles pode acabar gerando **conflitos difíceis de resolver**.

2.3.1.3. Exemplo Prático de Cascata

```
<head>
  <style>
    p { color: green; }           /* Especificidade: 1 */
    .destaque { color: blue; }   /* Especificidade: 10 */
    #principal { color: red; }   /* Especificidade: 100 */
  </style>
</head>
<body>
  <p id="principal" class="destaque" style="color: yellow;">
    Qual cor será exibida?
  </p>
</body>
```

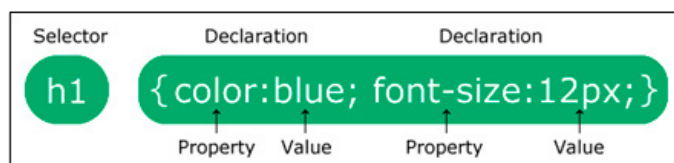
Resposta: Amarelo > style inline tem prioridade máxima.

2.4. SINTAXE BÁSICA

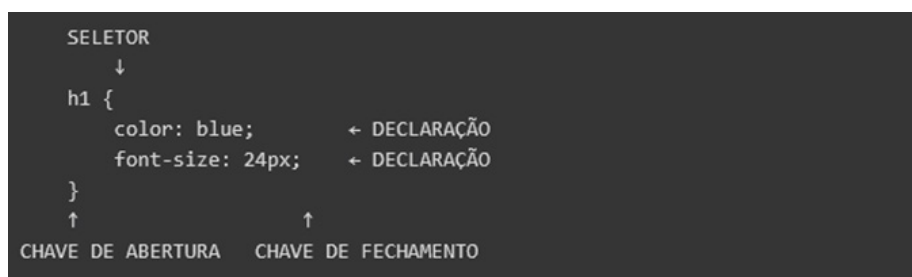
A estrutura fundamental do CSS segue **regra** é composta por:

1. **Seletor**: indica qual elemento HTML será estilizado
2. **Bloco de declaração**: conjunto de uma ou mais declarações entre chaves {}
3. **Declarações**: Cada declaração inclui um nome de propriedade CSS e um valor, separados por dois pontos. Várias declarações CSS são separadas com ponto e vírgula e declaração

2.4.1. ESQUEMA VISUAL DA SINTAXE



Fonte: <https://www.w3schools.com/css>



2.4.2. REGRAS DE SINTAXE

1. Seletor seguido de chaves {}.
2. Propriedades em minúsculas.

3. Dois pontos: separam propriedade e valor.
4. Ponto e vírgula; termina cada declaração.
5. Chaves {} delimitam o bloco de declarações.

2.4.3. EXEMPLOS DE SINTAXE CORRETA

```
/* Seletor de elemento */
p {
    color: red;
    font-size: 16px;
}

/* Seletor de classe */
.destaque {
    background-color: yellow;
    font-weight: bold;
}

/* Seletor de ID */
#cabecalho {
    text-align: center;
    margin: 20px;
}
```

Comentários em CSS

Sintaxe: **/* comentário */**

```
/* Este é um comentário de uma linha */

/*
    Este é um comentário
    de múltiplas linhas
*/

p {
    color: blue; /* Comentário inline */
}
```

2.4.4. ERROS COMUNS DE SINTAXE

Incorreto:

```
p { color blue; }           /* Falta dois pontos */
p { color: blue }          /* Falta ponto e vírgula */
p [ color: blue; ]         /* Chaves incorretas */
P { COLOR: BLUE; }         /* Maiúsculas (funciona, mas não recomendado) */
```

Correto:

```
p { color: blue; }
```


2.5. ESQUEMAS VISUAIS CONSOLIDADOS

2.5.1. MÉTODOS DE APLICAÇÃO CSS

```
CSS - MÉTODOS DE APLICAÇÃO
├── INLINE
│   ├── Atributo style no elemento
│   ├── Escopo: Elemento único
│   └── Prioridade: MÁXIMA
├── INTERNO
│   ├── Tag <style> no <head>
│   ├── Escopo: Página única
│   └── Prioridade: Média
└── EXTERNO
    ├── Arquivo .css + tag <link>
    ├── Escopo: Múltiplas páginas
    └── Prioridade: Baixa (mas RECOMENDADO)
```

2.5.2. SINTAXE BÁSICA

```
REGRA CSS
├── SELETOR (quem será estilizado)
└── BLOCO DE DECLARAÇÃO
    ├── { (chave de abertura)
    ├── DECLARAÇÕES
    │   ├── propriedade: valor;
    │   └── propriedade: valor;
    └── } (chave de fechamento)
```

2.5.3. ORDEM DE PRIORIDADE

```
PRIORIDADE CSS (maior → menor)
1. !important
2. Estilos inline (style="...")
3. IDs (#identificador)
4. Classes (.classe), Atributos ([attr]), Pseudo-classes (:hover)
5. Elementos (p, h1), Pseudo-elementos (::before)
6. Estilos do navegador (padrão)

REGRA DE DESEMPATE: Último declarado vence
```

Para Finalizar:



3. SELETORES ESSENCIAIS

3.1. INTRODUÇÃO AOS SELETORES

3.1.1. O QUE SÃO SELETORES CSS?

Seletores são padrões usados para **encontrar e selecionar elementos HTML** que receberão estilos CSS. São expressões que identificam **quais elementos HTML** serão afetados pelas **regras de estilo definidas no bloco de declaração**.

Anatomia de uma Regra CSS:

```
SELETOR
↓
h1 {
  color: blue;      + Seleciona elementos <h1>
                    + Aplica cor azul
  font-size: 24px;  + Aplica tamanho 24px
}
```

3.1.2. CATEGORIAS DE SELETORES

Os **Seletores CSS** são divididos em **5 categorias**:

Categoria	Descrição	Exemplo
Simples	Seleciona por nome, classe ou ID	p, .classe, #id
Combinador	Seleciona por relacionamento	div p, h1 + p
Pseudo-classe	Seleciona por estado	:hover, :focus
Pseudo-elemento	Seleciona parte do elemento	::before, ::after
Atributo	Seleciona por atributo	[type="text"]

3.2. SELETORES BÁSICOS

3.2.1. SELETOR DE ELEMENTO (TAG)

Seleciona todos os elementos de um **tipo específico**.

Sintaxe: nome_do_elemento

Exemplos:

```
/* Seleciona TODOS os parágrafos */
p {
  color: blue;
  font-size: 16px;
}

/* Seleciona TODOS os títulos h1 */
h1 {
  text-align: center;
  color: red;
}

/* Seleciona TODAS as divs */
div {
  margin: 10px;
  padding: 5px;
}
```

HTML Correspondente:

```
<h1>Este título será vermelho e centralizado</h1>
<p>Este parágrafo será azul, tamanho 16px</p>
<p>Este parágrafo TAMBÉM será azul, tamanho 16px</p>
<div>Esta div terá margem e padding</div>
```

Vantagem: Aplica estilo global a todos os elementos do tipo

Desvantagem: Não permite personalização específica

Uso: Estilos base e padronização geral

Especificidade: 1 (baixa)

3.2.2. SELETOR DE CLASSE

Seleciona elementos que possuem um **atributo class específico**.

Sintaxe: **.nome_da_classe** (precedido por ponto)

CSS:

```
/* Seleciona elementos com class="destaque" */
.destaque {
    background-color: yellow;
    font-weight: bold;
}

/* Seleciona elementos com class="erro" */
.erro {
    color: red;
    border: 1px solid red;
}

/* Seleciona elementos com class="sucesso" */
.sucesso {
    color: green;
    border: 1px solid green;
}
```

HTML Correspondente:

```
<p class="destaque">Parágrafo destacado em amarelo</p>
<span class="erro">Mensagem de erro em vermelho</span>
<div class="sucesso">Mensagem de sucesso em verde</div>
<p class="destaque">Outro parágrafo também destacado</p>
```

Vantagem: Reutilizável em múltiplos elementos

Flexibilidade: Um elemento pode ter múltiplas classes

Importante!

```
<p class="destaque erro">Parágrafo com duas classes</p>
```

Uso: Estilos específicos e reutilizáveis

Especificidade: 10 (Média)

ATENÇÃO



Um nome de classe não pode começar com um número!

3.2.3. SELETOR ID

Seleciona o elemento que possui um **atributo id específico**.

Sintaxe: **#nome_do_id** (precedido por hashtag/cerquilha)

CSS:

```
/* Seleciona o elemento com id="cabecalho" */
#cabecalho {
    background-color: navy;
    color: white;
    text-align: center;
}

/* Seleciona o elemento com id="menu-principal" */
#menu-principal {
    width: 100%;
    height: 50px;
}

/* Seleciona o elemento com id="rodape" */
#rodape {
    position: fixed;
    bottom: 0;
}
```

HTML Correspondente:

```
<header id="cabecalho">Cabeçalho do Site</header>
<nav id="menu-principal">Menu de Navegação</nav>
<footer id="rodape">Rodapé do Site</footer>
```

ATENÇÃO



Regra fundamental: ID deve ser único na página.

Vantagens: Alta especificidade, estilo muito específico

Desvantagem: Não é reutilizável

Uso: Elementos únicos (cabeçalho, menu, rodapé)

Especificidade: 100 (Alta)

DICA

Um nome id não pode começar com um número!

Erro Comum:

```
<!-- INCORRETO: IDs duplicados -->
<div id="box">Primeira div</div>
<div id="box">Segunda div</div> <!-- ERRO! -->

<!-- CORRETO: IDs únicos -->
<div id="box1">Primeira div</div>
<div id="box2">Segunda div</div>
```

3.2.4. SELETOR UNIVERSAL

Seleciona **todos os elementos da página**.

Sintaxe: * (asterisco)

Exemplo:

```
/* Aplica a TODOS os elementos */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* Reset CSS básico */
* {
    font-family: Arial, sans-serif;
```

Vantagem: Útil para reset CSS e estilos globais

Desvantagem: Pode impactar performance em páginas grandes

Uso: Reset de estilos padrão do navegador

Especificidade: 0 (muito baixa)

Uso prático:

```
/* Remove margens e paddings padrão de todos os elementos */
* {
    margin: 0;
    padding: 0;
}
```

3.2.5. SELETOR DE GRUPO

Aplica os **mesmos estilos a múltiplos seletores**.

Sintaxe: seletor1, seletor2, seletor3 (separados por vírgula)

Exemplo:

CSS:

```
/* Aplica o mesmo estilo a h1, h2 e h3 */
h1, h2, h3 {
    color: navy;
    font-family: Arial;
    margin-bottom: 10px;
}

/* Aplica a múltiplos seletores diferentes */
p, .destaque, #principal {
    line-height: 1.5;
    text-align: justify;
}

/* Agrupa elementos de formulário */
input, textarea, select {
    border: 1px solid #ccc;
    padding: 5px;
    font-size: 14px;
}
```

HTML Correspondente:

```
<h1>Título 1 - azul marinho</h1>
<h2>Título 2 - azul marinho</h2>
<h3>Título 3 - azul marinho</h3>
<p>Parágrafo com line-height 1.5</p>
<span class="destaque">Span com line-height 1.5</span>
```

Vantagem: Evita repetição de código

Eficiência: Menos linhas de CSS

Uso: Estilos comuns a múltiplos elementos

Especificidade: Depende de cada seletor individual

Seletor	Exemplo	Descrição
#id	#firstname	Seleciona o elemento com id="firstname"
.classe	.intro	Seleciona todos os elementos com class="intro"
* (universal)	*	Seleciona todos os elementos
elemento	p	Seleciona todos os elementos <p>
elemento,elemento,..	div, p	Seleciona todos os elementos <div> e todos os <p>

3.3. ESPECIFICIDADE BÁSICA

Especificidade é o mecanismo que determina qual **regra CSS** será aplicada quando há **conflito entre seletores** que afetam o mesmo elemento.

3.3.1. CÁLCULO DE ESPECIFICIDADE

Tipo de Seletor	Valor	Exemplo
Inline style	1000	style="color: red;"
ID	100	#cabecalho
Classe, atributo, pseudo-classe	10	.destaque, [type="text"],:hover
Elemento, pseudo-elemento	1	p, h1,::before
Universal	0	*

Exemplos de cálculos:

```
/* Especificidade = 1 */
p { color: blue; }

/* Especificidade = 10 */
.destaque { color: red; }
```

```
/* Especificidade = 100 */
#principal { color: green; }

/* Especificidade = 11 (10 + 1) */
p.destaque { color: yellow; }

/* Especificidade = 101 (100 + 1) */
div#principal { color: purple; }

/* Especificidade = 111 (100 + 10 + 1) */
div#principal.destaque { color: orange; }
```

3.3.2. REGRA DE DESEMPATE

Quando dois seletores têm a **mesma especificidade**, o que aparece por **último no CSS** é aplicado.

```
p { color: red; } /* Especificidade: 1 */
p { color: blue; } /* Especificidade: 1 - ESTE será aplicado */
```

3.3.3. EXEMPLO PRÁTICO COMPLETO

HTML:

```
<p id="principal" class="destaque">Qual cor será exibida?</p>
```

CSS:

```
p { color: black; } /* Especificidade: 1 */
.destaque { color: blue; } /* Especificidade: 10 */
#principal { color: red; } /* Especificidade: 100 */
p.destaque { color: green; } /* Especificidade: 11 */
```


3.4. ESQUEMAS VISUAIS CONSOLIDADOS

3.4.1. TIPOS DE SELETORES BÁSICOS

SELETORES CSS BÁSICOS	
ELEMENTO	- Sintaxe: elemento - Exemplo: p, h1, div - Especificidade: 1 - Uso: Estilos globais
CLASSE	- Sintaxe: .classe - Exemplo: .destaque, .erro - Especificidade: 10 - Uso: Estilos reutilizáveis
ID	- Sintaxe: #id - Exemplo: #cabecalho, #menu - Especificidade: 100 - Uso: Elementos únicos
UNIVERSAL	- Sintaxe: * - Exemplo: * { margin: 0; } - Especificidade: 0 - Uso: Reset CSS
GRUPO	- Sintaxe: sel1, sel2, sel3 - Exemplo: h1, h2, h3 - Especificidade: Individual - Uso: Estilos comuns

3.4.2. ESPECIFICIDADE CSS

ESPECIFICIDADE (maior → menor)	
1000	→ Inline styles (style="...")
100	→ IDs (#identificador)
10	→ Classes (.classe), Atributos ([attr]), Pseudo-classes (:hover)
1	→ Elementos (p, h1), Pseudo-elementos (::before)
0	→ Universal (*)
REGRA DE DESEMPATE: Último declarado vence	
QUEBRA DE REGRA: !important (força aplicação)	

3.4.3. SINTAXE DOS SELETORES

```
SINTAXE DOS SELETORES
├─ Elemento → nome_elemento
│   └─ Exemplo: p, h1, div, span
├─ Classe → .nome_classe
│   └─ Exemplo: .destaque, .erro, .sucesso
├─ ID → #nome_id
│   └─ Exemplo: #cabecalho, #menu, #rodape
├─ Universal → *
│   └─ Exemplo: * (todos os elementos)
└─ Grupo → seletor1, seletor2
    └─ Exemplo: h1, h2, h3
```

4. BOX MODEL

4.1. CONCEITOS DO BOX MODEL

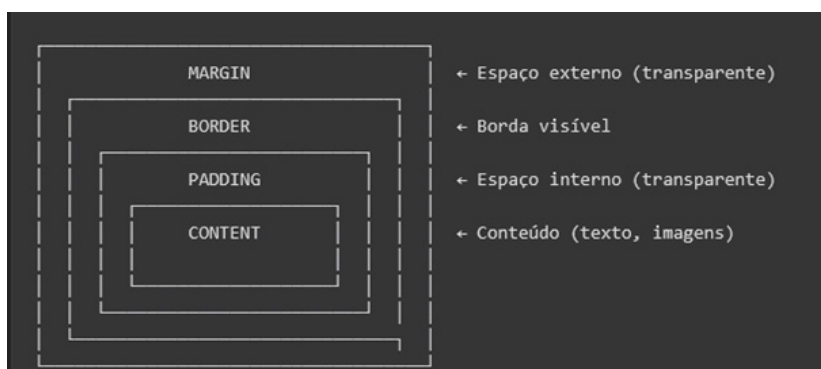
Box Model (Modelo de Caixa) **define** como cada elemento HTML é renderizado como uma caixa retangular composta por quatro áreas distintas. É o modelo que descreve **como as propriedades de dimensão, espaçamento e bordas de um elemento HTML são calculadas e exibidas**, consistindo em **content, padding, border e margin**.

Como analogia, imagine uma caixa de presente:

- **Content** = o presente dentro da caixa = onde o texto e as imagens aparecem.
- **Padding** = o papel de seda que protege o presente = limpa uma área em torno do conteúdo. O preenchimento é transparente.
- **Border** = a caixa propriamente dita = uma fronteira que gira em torno do estofamento e conteúdo.
- **Margin** = o espaço entre esta caixa e outras caixas = limpa uma área fora da fronteira. A margem é transparente.

O modelo de caixa nos permite **adicionar uma borda em torno de elementos, e definir o espaço entre os elementos**.

4.1.1. ESTRUTURA VISUAL DO BOX MODEL



4.1.2. COMPONENTES DO BOX MODEL

Componente	Descrição	Visibilidade	Afeta Dimensão
Content	Área do conteúdo (texto, imagens)	Visível	Sim
Padding	Espaço entre conteúdo e borda	Transparente	Sim
Border	Borda ao redor do padding	Visível	Sim
Margin	Espaço externo entre elementos	Transparente	Não*

* A margem não afeta a dimensão interna do elemento, mas influencia o espaçamento externo entre elementos.

4.1.3. FÓRMULA DE CÁLCULO (BOX MODEL PADRÃO)

- **Largura Total** = Largura total do elemento – largura + estofamento esquerdo + estofamento direito + borda esquerda + borda direita
- **Altura Total** = Altura total do elemento? altura + estofamento superior + preenchimento inferior + borda superior + borda inferior

DICA

1) Quando você define as propriedades de largura e altura de um Elemento com CSS, você apenas define a largura e a altura da área de conteúdo. Para a sua forma de calcular a largura total e altura de um elemento, você também deve incluir o estofamento e as bordas.

2) A propriedade de margem também afeta o espaço total que a caixa vai assumir a página, mas a margem não está incluída no tamanho real de A caixa. A largura e a altura total da caixa param na fronteira.

4.2. PROPRIEDADES DE DIMENSÃO

4.2.1. LARGURA E ALTURA (WIDTH E HEIGHT)

Definem as **dimensões da área de conteúdo**.

Sintaxe:

```
elemento {
  width: valor;
  height: valor;
}
```

Valores Aceitos:

- **Pixels:** width: 300px;
- **Percentual:** width: 50%;
- **Auto:** width: auto; (padrão)
- **Unidades relativas:** width: 20em;

Exemplo Prático:

```
.caixa {
  width: 200px;
  height: 100px;
  background-color: lightblue;
}
```

4.2.2. MIN/MAX WIDTH E HEIGHT

Estabelecem **limites mínimos e máximos para as dimensões**.

Propriedades:

```
.responsiva {
  width: 100%;
  min-width: 300px; /* Largura mínima */
  max-width: 800px; /* Largura máxima */
  min-height: 200px; /* Altura mínima */
  max-height: 500px; /* Altura máxima */
}
```

Uso Prático: Design responsivo e controle de layout.

4.2.3. DIMENSIONAMENTO DA CAIXA (BOX-SIZING)

Controla **como as dimensões são calculadas**.

Valores:

1. **content-box (padrão)** – width/height aplicam-se apenas ao content
2. **border-box** – width/height incluem padding e border

Exemplo Comparativo:

Content-box (padrão):

```
.caixa-padrao {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: content-box;
}
/* Largura total = 200 + 20 + 20 + 5 + 5 = 250px */
```

Border-box (recomendado):

```
.caixa-moderna {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  box-sizing: border-box;
}
/* Largura total = 200px (padding e border inclusos) */
```

DICA

Use box-sizing: border-box para facilitar cálculos!

Reset CSS Recomendado:

```
* {
  box-sizing: border-box;
}
```

4.3. MARGENS (MARGINS)

Margin (Margem) é o **espaço externo ao redor do elemento, fora da borda**. É transparente e cria distância entre elementos.

4.3.1. PROPRIEDADES INDIVIDUAIS

CSS tem propriedades para especificar a margem para cada O lado de um elemento:

Propriedade	Descrição
margin	Propriedade abreviada para definir todas as margens em uma única declaração
margin-bottom	Define a margem inferior de um elemento
margin-left	Define a margem esquerda de um elemento
margin-right	Define a margem direita de um elemento
margin-top	Define a margem superior de um elemento

Todas as propriedades de margem podem ter os seguintes valores:

- **Auto** – o navegador calcula a margem
- **comprimento** – especifica uma margem em px, pt, cm, etc.
- **%** – especifica uma margem em % da largura do elemento que contém
- **herdar** – especifica que a margem deve ser herdada do elemento pai

DICA

Valores negativos também são permitidos.

```
.elemento {
  margin-top: 10px;      /* Margem superior */
  margin-right: 15px;   /* Margem direita */
  margin-bottom: 20px;  /* Margem inferior */
  margin-left: 25px;    /* Margem esquerda */
}
```

4.3.2. PROPRIEDADE ABREVIADA (SHORTHAND)

Sintaxe: `margin: [top] [right] [bottom] [left]`

Variações:

- **4 valores:** `margin: 10px 15px 20px 25px;`
 - Top: 10px, Right: 15px, Bottom: 20px, Left: 25px
- **3 valores:** `margin: 10px 15px 20px;`
 - Top: 10px, Right/Left: 15px, Bottom: 20px
- **2 valores:** `margin: 10px 15px;`
 - Top/Bottom: 10px, Right/Left: 15px
- **1 valor:** `margin: 10px;`
 - Todas as margens: 10px

4.3.3. VALORES ESPECIAIS

Auto (Centralização):

```
.centralizado {
  width: 300px;
  margin: 0 auto; /* Centraliza horizontalmente */
}
```

Valores Negativos:

```
.sobreposto {
  margin-top: -10px; /* Move o elemento para cima */
}
```

Inherit (Herança):

```
.herdado {
  margin: inherit; /* Herda margin do elemento pai */
}
```

4.3.4. COLAPSO DE MARGENS (MARGIN COLLAPSE)

Quando margens verticais adjacentes se combinam em uma única margem.

Regra: A maior margem prevalece.

Exemplo:

```
.elemento1 { margin-bottom: 20px; }
.elemento2 { margin-top: 30px; }
/* Espaço entre eles = 30px (não 50px) */
```

HTML:

```
<div class="elemento1">Primeiro elemento</div>
<div class="elemento2">Segundo elemento</div>
<!-- Espaço entre eles será 30px, não 50px -->
```

ATENÇÃO



Colapso só ocorre com margens verticais (top/bottom).

4.4. BORDAS (BORDER)

Border é a linha visível que contorna o padding e o content do elemento.

4.4.1. PROPRIEDADES INDIVIDUAIS

Espessura (Border-width):

```
.elemento {
  border-top-width: 1px;
  border-right-width: 2px;
  border-bottom-width: 3px;
  border-left-width: 4px;
}
```

Valores pré-definidos:

- thin (fina)
- medium (média) – padrão
- thick (grossa)

Estilo (Border-style):

São permitidos os seguintes valores:

- **dotted:** define uma borda pontilhada;
- **dashed:** define uma fronteira tracejada;

- **solid**: define uma borda sólida;
- **double**: define uma dupla fronteira;
- **groove**: define uma fronteira render em 3D. O efeito depende do valor da borda-corte;
- **ridge**: define uma fronteira 3D. O efeito depende do valor da borda-corte;
- **inset**: define uma fronteira 3D. O efeito depende do valor da borda-corte;
- **outset**: define uma fronteira inicial em 3D. O efeito depende do valor da borda-corte;
- **none**: não define nenhuma fronteira;
- **hidden**: define uma fronteira escondida.

```
.estilos {
  border-style: solid;      /* Linha sólida */
  border-style: dashed;    /* Linha tracejada */
  border-style: dotted;    /* Linha pontilhada */
  border-style: double;    /* Linha dupla */
  border-style: groove;    /* Sulcada */
  border-style: ridge;     /* Com crista */
  border-style: inset;     /* Para dentro */
  border-style: outset;    /* Para fora */
  border-style: none;      /* Sem borda */
  border-style: hidden;    /* Escondida */
}
```

Cor (Border-color):

A cor pode ser definida por:

- **name**: especificar um nome de cor, como “vermelho”
- **HEX**: especifique um valor HEX, como “-ff0000”
- **RGB**: especificar um valor RGB, como “rgb(255,0,0)”
- **HSL**: especifique um valor HSL, como “hsl(0, 100%, 50%)”
- **transparente**

DICA

Se border color não é definido, ele herda a cor do elemento.

```
.colorida {
  border-color: red;
  border-color: #ff0000;
  border-color: rgb(255, 0, 0);
  border-color: rgba(255, 0, 0, 0.5);
}
```


4.4.2. PROPRIEDADE ABREVIADA

Sintaxe: **border: [width] [style] [color]**

```
.borda-completa {
  border: 2px solid red;
  /* Equivale a:
    border-width: 2px;
    border-style: solid;
    border-color: red; */
}
```

Bordas Específicas:

```
.bordas-especificas {
  border-top: 1px solid black;
  border-right: 2px dashed blue;
  border-bottom: 3px dotted green;
  border-left: 4px double red;
}
```

4.4.3. BORDER-RADIUS (CSS3)

Cria **cantos arredondados**.

```
.arredondado {
  border-radius: 10px;           /* Todos os cantos */
  border-radius: 10px 20px;      /* Top-Left/bottom-right: 10px, outros: 20px */
  border-radius: 10px 20px 30px; /* Top-Left: 10px, top-right/bottom-left: 20px,
  bottom-right: 30px */
  border-radius: 10px 20px 30px 40px; /* Cada canto individual */
}

/* Cantos específicos */
.cantos-especificos {
  border-top-left-radius: 10px;
  border-top-right-radius: 20px;
  border-bottom-right-radius: 30px;
  border-bottom-left-radius: 40px;
}

/* Círculo perfeito */
.circulo {
  width: 100px;
  height: 100px;
  border-radius: 50%;
}
```

4.5. PREENCHIMENTO (PADDING)

Padding é o **espaço interno entre o conteúdo e a borda**. As propriedades de preenchimento CSS são usadas **para gerar espaço em torno de conteúdo de um elemento, dentro de quaisquer fronteiras definidas**.

Com CSS, há propriedades para definir o preenchimento para cada lado de um elemento (topo, direito, inferior e esquerdo), e uma propriedade abreviação para definir todas as propriedades de preenchimento em uma Declaração.

4.5.1. PROPRIEDADES INDIVIDUAIS

CSS tem propriedades para especificar o preenchimento para cada lado de um elemento:

- **padding-top:** define o estofamento superior de um elemento;
- **padding-right:** define o estofamento direito de um elemento;
- **padding-bottom:** define o estofamento inferior de um elemento;
- **padding-left:** define o estofamento esquerdo de um elemento.

```
.elemento {
  padding-top: 10px;      /* Preenchimento superior */
  padding-right: 15px;    /* Preenchimento direito */
  padding-bottom: 20px;   /* Preenchimento inferior */
  padding-left: 25px;     /* Preenchimento esquerdo */
}
```

Todas as propriedades de preenchimento podem ter os seguintes valores:

- **Comprimento:** especifica um estofamento em px, pt, cm, etc.;
- **%:** especifica um estofamento em % da largura do elemento que contém;
- **Herança:** especifica que o preenchimento deve ser herdado do elemento pai.

Obs.: Valores negativos não são permitidos.

4.5.2. PROPRIEDADE ABREVIADA

Sintaxe: **padding:** [top] [right] [bottom] [left]

Variações (iguais ao margin):

```
/* 4 valores */
.padding-4 { padding: 10px 15px 20px 25px; }

/* 3 valores */
.padding-3 { padding: 10px 15px 20px; }

/* 2 valores */
.padding-2 { padding: 10px 15px; }

/* 1 valor */
.padding-1 { padding: 10px; }
```

4.5.3. DIFERENÇAS ENTRE MARGIN E PADDING

Aspecto	Margin	Padding
Localização	Externo (fora da borda)	Interno (dentro da borda)
Visibilidade	Transparente	Herda background do elemento
Colapso	Sim (margens verticais)	Não
Valores negativos	Permitidos	Não permitidos
Centralização	margin: 0 auto	Não aplicável

4.6. CÁLCULOS PRÁTICOS DE DIMENSÕES

4.6.1 EXEMPLO COMPLETO – CONTENT-BOX

```
.caixa {
  width: 200px;
  height: 100px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
  box-sizing: content-box; /* Padrão */
}
```

Cálculos:

- **Largura do content:** 200px
- **Largura total do elemento:** $200 + 20 + 20 + 5 + 5 = 250\text{px}$
- **Espaço total ocupado:** $250 + 10 + 10 = 270\text{px}$
- **Altura do content:** 100px
- **Altura total do elemento:** $100 + 20 + 20 + 5 + 5 = 150\text{px}$
- **Espaço total ocupado:** $150 + 10 + 10 = 170\text{px}$

4.6.2 EXEMPLO COMPLETO – BORDER-BOX

```
.caixa-moderna {
  width: 200px;
  height: 100px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
  box-sizing: border-box;
}
```

Cálculos:

- **Largura total do elemento:** 200px (fixo)
- **Largura do content:** $200 - 20 - 20 - 5 - 5 = 150\text{px}$

- **Espaço total ocupado:** $200 + 10 + 10 = 220\text{px}$
- **Altura total do elemento:** 100px (fixo)
- **Altura do content:** $100 - 20 - 20 - 5 - 5 = 50\text{px}$
- **Espaço total ocupado:** $100 + 10 + 10 = 120\text{px}$

4.7. ESQUEMAS VISUAIS CONSOLIDADOS

4.7.1. COMPONENTES DO BOX MODEL

```
BOX MODEL - COMPONENTES
├── CONTENT (Conteúdo)
│   ├── Área do texto/imagens
│   ├── Definido por width/height
│   └── Visível
├── PADDING (Preenchimento)
│   ├── Espaço interno (content ↔ border)
│   ├── Transparente (herda background)
│   └── Não aceita valores negativos
├── BORDER (Borda)
│   ├── Linha visível ao redor do padding
│   ├── Propriedades: width, style, color
│   └── border-radius para arredondamento
└── MARGIN (Margem)
    ├── Espaço externo (entre elementos)
    ├── Transparente
    ├── Aceita valores negativos
    ├── Colapso vertical
    └── margin: auto para centralização
```

4.7.2. PROPRIEDADES ABREVIADAS

```
PROPRIEDADES ABREVIADAS
├── MARGIN/PADDING
│   ├── 1 valor: margin: 10px; (todos)
│   ├── 2 valores: margin: 10px 20px; (vertical horizontal)
│   ├── 3 valores: margin: 10px 20px 30px; (top horizontal bottom)
│   └── 4 valores: margin: 10px 20px 30px 40px; (top right bottom left)
├── BORDER
│   ├── border: width style color;
│   ├── border: 2px solid red;
│   └── border-top: 1px dashed blue;
└── BORDER-RADIUS
    ├── border-radius: 10px; (todos os cantos)
    ├── border-radius: 10px 20px; (diagonal)
    └── border-radius: 10px 20px 30px 40px; (cada canto)
```

4.7.3. BOX-SIZING

```
BOX-SIZING
├── CONTENT-BOX (padrão)
│   ├── width/height = apenas content
│   ├── Dimensão total = width + padding + border
│   └── Cálculos mais complexos
└── BORDER-BOX (recomendado)
    ├── width/height = content + padding + border
    ├── Dimensão total = width (fixo)
    └── Cálculos mais simples
```

5. CORES E BACKGROUDS

5.1. SISTEMA DE CORES CSS

5.1.1. INTRODUÇÃO ÀS CORES

Cores são fundamentais no design web e CSS oferece múltiplos sistemas para defini-las. Cada sistema tem suas vantagens e usos específicos.

As propriedades que Aceitam Cores são:

- **color** (cor do texto)
- **background-color** (cor de fundo)
- **border-color** (cor da borda)
- **box-shadow** (cor da sombra)
- **text-shadow**

5.1.2. NOMES DE CORES (COLOR NAMES)

CSS possui **147** nomes de cores predefinidos.



Cores Básicas Mais Usadas:

```
.cores-basicas {
  color: red;           /* Vermelho */
  color: blue;          /* Azul */
  color: green;         /* Verde */
  color: black;         /* Preto */
  color: white;         /* Branco */
  color: gray;          /* Cinza */
  color: yellow;        /* Amarelo */
  color: orange;        /* Laranja */
  color: purple;        /* Roxo */
  color: pink;          /* Rosa */
}
```

Vantagem: Fácil de lembrar e usar

Desvantagem: Limitado a 147 cores

Uso: Prototipagem rápida e cores básicas

5.1.3. SISTEMA HEXADEXIMAL

Baseado em **6 dígitos hexadecimais (0-9, A-F) precedidos por #**.

Estrutura: #RRGGBB

- **RR = Red (vermelho)**
- **GG = Green (verde)**
- **BB = Blue (azul)**



Exemplos:

```
.cores-hex {
  color: #FF0000; /* Vermelho puro */
  color: #00FF00; /* Verde puro */
  color: #0000FF; /* Azul puro */
  color: #000000; /* Preto */
  color: #FFFFFF; /* Branco */
  color: #808080; /* Cinza médio */
  color: #FF6835; /* Laranja personalizado */
}
```

Vantagem: Amplamente suportado, compacto

Precisão: 16.777.216 cores possíveis

Uso: Design profissional, ferramentas gráficas

5.1.4. SISTEMA RGB

Sistema baseado em **Red, Green, Blue** com **valores de 0 a 255**.

Sintaxe: rgb(red, green, blue)



Exemplos:

```
.cores-rgb {
  color: rgb(255, 0, 0);      /* Vermelho puro */
  color: rgb(0, 255, 0);     /* Verde puro */
  color: rgb(0, 0, 255);     /* Azul puro */
  color: rgb(0, 0, 0);       /* Preto */
  color: rgb(255, 255, 255); /* Branco */
  color: rgb(128, 128, 128); /* Cinza médio */
  color: rgb(255, 107, 53);  /* Laranja personalizado */
}
```

Vantagem: Intuitivo, fácil de entender

Flexibilidade: Valores podem ser calculados

Uso: Quando se trabalha com valores numéricos

5.1.5. SISTEMA RGBA (CSS3)

RGB com **canal Alpha** para **transparência**.

Sintaxe: **rgba(red, green, blue, alpha)**

- Alpha: 0 (transparente) a 1 (opaco)

Exemplo:

```
.cores-rgba {
  background-color: rgba(255, 0, 0, 1);      /* Vermelho opaco */
  background-color: rgba(255, 0, 0, 0.5);   /* Vermelho 50% transparente */
  background-color: rgba(0, 0, 0, 0.8);     /* Preto 80% opaco */
  background-color: rgba(255, 255, 255, 0.2); /* Branco 20% opaco */
}
```

Vantagem: Controle de transparência

CSS3: Suporte moderno

Uso: Overlays, efeitos de transparência

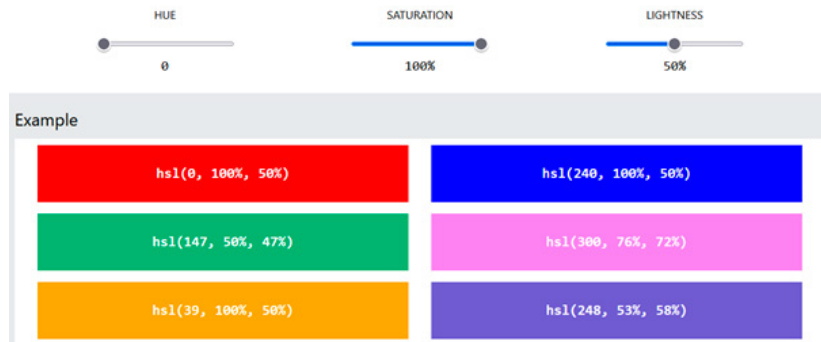
5.1.6. SISTEMA HSL (CSS3)

Hue, Saturation, Lightness – sistema mais intuitivo para designers.

Componentes:

- **Hue** (Matiz): 0–360° (roda de cores)
- **Saturation** (Saturação): 0–100% (intensidade da cor)
- **Lightness** (Luminosidade): 0–100% (claridade)

Sintaxe: **hsl(hue, saturation%, lightness%)**



Exemplo:

```
.cores-hsl {
  color: hsl(0, 100%, 50%); /* Vermelho puro */
  color: hsl(120, 100%, 50%); /* Verde puro */
  color: hsl(240, 100%, 50%); /* Azul puro */
  color: hsl(0, 0%, 0%); /* Preto */
  color: hsl(0, 0%, 100%); /* Branco */
  color: hsl(0, 0%, 50%); /* Cinza médio */
}
```

Vantagem: Intuitivo para designers

Flexibilidade: Fácil criar variações de cor

Uso: Design avançado, paletas de cores

Sistema	Sintaxe	Transparência	Suporte	Uso Recomendado
Nomes	red	Não	Universal	Prototipagem
Hex	#FF0000	Não	Universal	Design profissional
RGB	rgb(255,0,0)	Não	Universal	Cálculos numéricos
RGBA	rgba(255,0,0,0.5)	Sim	CSS3+	Transparência
HSL	hsl(0,100%,50%)	Não	CSS3+	Design intuitivo
HSLA	hsla(0,100%,50%,0.5)	Sim	CSS3+	Design + Transparência

5.2. PROPRIEDADES DE BACKGROUND

5.2.1. COR DE FUNDO (BACKGROUND-COLOR)

Define a **cor de fundo** do elemento.

Sintaxe: `background-color: cor;`

Exemplos:

```
.fundos-coloridos {
  background-color: red;           /* Nome */
  background-color: #FF0000;      /* Hex */
  background-color: rgb(255, 0, 0); /* RGB */
  background-color: rgba(255, 0, 0, 0.5); /* RGBA */
  background-color: hsl(0, 100%, 50%); /* HSL */
  background-color: transparent;   /* Transparente */
}
```

Hello World!

This page has a light blue background color!

Padrão: transparente;

Herança: Não é herdada;

Uso: Fundos sólidos, destaque de elementos.

5.2.2. IMAGEM DE FUNDO (BACKGROUND-IMAGE)

Define uma **imagem como fundo do elemento**.

Sintaxe: `background-image: url('caminho');`

Exemplos:

```
.com-imagem {
  background-image: url('imagem.jpg');
  background-image: url('https://exemplo.com/imagem.png');
  background-image: url('../images/fundo.gif');
  background-image: none; /* Remove imagem */
}
```

Hello World!

This page has an image as the background!

Formatos: JPG, PNG, GIF, SVG, WebP

URLs: Relativas ou absolutas

Uso: Texturas, padrões, imagens decorativas

5.2.3. REPETIÇÃO (BACKGROUND-REPEAT)

Controla **como a imagem de fundo é repetida**.

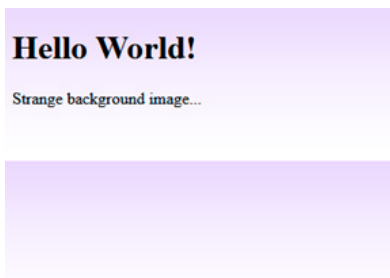
Valores:

```
.repeticoes {
  background-repeat: repeat;      /* Repete horizontal e vertical (padrão) */
  background-repeat: repeat-x;   /* Repete apenas horizontalmente */
  background-repeat: repeat-y;   /* Repete apenas verticalmente */
  background-repeat: no-repeat;  /* Não repete */
  background-repeat: space;      /* Repete com espaçamento */
  background-repeat: round;      /* Repete ajustando tamanho */
}
```

Exemplo Prático:

```
.padrao {
  background-image: url('textura.png');
  background-repeat: repeat; /* Cria padrão contínuo */
}

.logo {
  background-image: url('logo.png');
  background-repeat: no-repeat; /* Logo única */
}
```



5.2.4. POSICIONAMENTO (BACKGROUND-POSITION)

Define a **posição inicial da imagem de fundo**.

Valores por Palavras-chave:

```
.posicoes {
  background-position: left top;    /* Canto superior esquerdo */
  background-position: center;     /* Centro */
  background-position: right bottom; /* Canto inferior direito */
  background-position: left center; /* Centro esquerdo */
  background-position: center top;  /* Centro superior */
}
```

Valores Numéricos:

```
.posicoes-numericas {
  background-position: 50px 100px; /* 50px da esquerda, 100px do topo */
  background-position: 50% 25%; /* 50% horizontal, 25% vertical */
  background-position: 0 0; /* Canto superior esquerdo */
}
```

Combinações:

```
.combinacoes {
  background-position: left 20px; /* Esquerda + 20px do topo */
  background-position: 75% center; /* 75% horizontal, centro vertical */
}
```

5.2.5. BACKGROUND-SIZE (CSS3)

Controla o **tamanho da imagem de fundo**.

Valores:

```
.tamanhos {
  background-size: auto; /* Tamanho original (padrão) */
  background-size: cover; /* Cobre todo o elemento */
  background-size: contain; /* Cabe inteiro no elemento */
  background-size: 100px 200px; /* Largura e altura específicas */
  background-size: 50%; /* 50% do elemento */
  background-size: 100% auto; /* Largura 100%, altura proporcional */
}
```

Uso Prático:

```
.hero-section {
  background-image: url('hero.jpg');
  background-size: cover; /* Imagem sempre cobre toda a área */
  background-position: center;
  background-repeat: no-repeat;
}
```

5.2.6. ANEXAÇÃO (BACKGROUND-ATTACHMENT)

Define se a **imagem rola com o conteúdo ou fica fixa**.

Valores:

```
.anexacao {
  background-attachment: scroll; /* Rola com o conteúdo (padrão) */
  background-attachment: fixed; /* Fica fixa na viewport */
  background-attachment: local; /* Rola com o conteúdo do elemento */
}
```

5.2.7. PROPRIEDADE ABREVIADA (BACKGROUND)

Sintaxe: `background: [color] [image] [repeat] [attachment] [position] / [size];`

Ordem Recomendada:

```
.exemplo-completo {
  background:
    #ffffff           /* cor */
    url('imagem.jpg') /* imagem */
    no-repeat         /* repetição */
    fixed             /* anexação */
    center top        /* posição */
    / cover;          /* tamanho */
}
```

5.3. ESQUEMAS VISUAIS CONSOLIDADOS

5.3.1. SISTEMA DE CORES

```
SISTEMAS DE CORES CSS
├── NOMES
│   ├── Sintaxe: red, blue, green
│   ├── Quantidade: 147 cores
│   └── Uso: Prototipagem rápida
├── HEXADECIMAL
│   ├── Sintaxe: #RRGGBB ou #RGB
│   ├── Exemplo: #FF0000, #F00
│   └── Uso: Design profissional
├── RGB
│   ├── Sintaxe: rgb(r, g, b)
│   ├── Valores: 0-255
│   └── Uso: Cálculos numéricos
├── RGBA (CSS3)
│   ├── Sintaxe: rgba(r, g, b, a)
│   ├── Alpha: 0-1 (transparência)
│   └── Uso: Efeitos transparentes
├── HSL (CSS3)
│   ├── Sintaxe: hsl(h, s%, l%)
│   ├── H: 0-360°, S/L: 0-100%
│   └── Uso: Design intuitivo
└── HSLA (CSS3)
    ├── Sintaxe: hsla(h, s%, l%, a)
    ├── Com transparência
    └── Uso: Design avançado
```

5.3.2. PROPRIEDADES DE BACKGROUND

```

PROPRIEDADES BACKGROUND
├── background-color
│   └── Cor sólida de fundo
├── background-image
│   └── Imagem de fundo (url)
├── background-repeat
│   ├── repeat (padrão)
│   ├── no-repeat
│   ├── repeat-x
│   └── repeat-y
├── background-position
│   ├── Palavras: left, center, right, top, bottom
│   └── Valores: px, %, combinações
├── background-size (CSS3)
│   ├── auto (padrão)
│   ├── cover (cobre tudo)
│   ├── contain (cabe inteiro)
│   └── valores específicos
├── background-attachment
│   ├── scroll (padrão)
│   ├── fixed (efeito parallax)
│   └── local
└── background (abreviada)
    └── color image repeat attachment position / size
    
```

6. TIPOGRAFIA E TEXTO

6.1. PROPRIEDADES DE TEXTO

6.1.1. COR DO TEXTO (COLOR)

Define a **cor do texto** do elemento.

Sintaxe: **color: valor;**

Exemplos:

```

.cores-texto {
  color: red;                /* Nome da cor */
  color: #FF0000;           /* Hexadecimal */
  color: rgb(255, 0, 0);     /* RGB */
  color: rgba(255, 0, 0, 0.8); /* RGBA com transparência */
  color: hsl(0, 100%, 50%);  /* HSL */
  color: currentColor;       /* Cor atual do elemento */
  color: inherit;            /* Herda do elemento pai */
}
    
```

This is heading 1

This is heading 2

This is a paragraph. Notice that this text is blue. The default text color for a page is defined in the body selector.

Another paragraph.

Herança: Propriedade herdada pelos elementos filhos

Padrão: Geralmente preto (#000000)

Uso:

6.1.2. ALINHAMENTO HORIZONTAL (TEXT-ALIGN)

Controla o **alinhamento horizontal do texto** dentro do elemento.

Valores:

```
.alinhamentos {
  text-align: left;      /* Esquerda (padrão para LTR) */
  text-align: right;     /* Direita */
  text-align: center;    /* Centralizado */
  text-align: justify;    /* Justificado */
  text-align: start;     /* Início (depende da direção) */
  text-align: end;       /* Final (depende da direção) */
}
```

Exemplo Prático:

```
.titulo {
  text-align: center;    /* Título centralizado */
}

.paragrafo {
  text-align: justify;    /* Texto justificado */
}

.assinatura {
  text-align: right;     /* Assinatura à direita */
}
```

Herança: Herdada pelos elementos filhos

Aplicação: Apenas em elementos de bloco

Uso: Alinhamento de títulos, parágrafos, textos

6.1.3. DECORAÇÃO DO TEXTO (TEXT-DECORATION)

Adiciona **linhas decorativas** ao texto.

Valores:

```
.decoracoes {
  text-decoration: none;    /* Sem decoração (padrão) */
  text-decoration: underline; /* Sublinhado */
  text-decoration: overline; /* Linha acima */
  text-decoration: line-through; /* Riscado */
  text-decoration: blink;   /* Piscante (obsoleto) */
}
```

Uso Prático -Links:

```
/* Remove sublinhado padrão dos links */
a {
    text-decoration: none;
}

/* Adiciona sublinhado no hover */
a:hover {
    text-decoration: underline;
}

/* Texto riscado para preços antigos */
.preco-antigo {
    text-decoration: line-through;
    color: #999;
}
```

CSS# – Propriedades Expandidas:

```
.decoracao-avancada {
    text-decoration-line: underline;      /* Tipo de linha */
    text-decoration-color: red;          /* Cor da linha */
    text-decoration-style: wavy;         /* Estilo da linha */
    text-decoration-thickness: 2px;      /* Espessura da linha */
}
```

Overline text decoration

Line-through text decoration

Underline text decoration

Overline and underline text decoration.

Note: It is not recommended to underline text that is not a link, as this often confuses the reader.

6.1.4. TRANSFORMAÇÃO DO TEXTO (TEXT-TRANSFORMATION)

Controla a **capitalização** do texto.

Valores:

```
.transformacoes {
    text-transform: none;      /* Sem transformação (padrão) */
    text-transform: uppercase; /* MAIÚSCULAS */
    text-transform: lowercase; /* minúsculas */
    text-transform: capitalize; /* Primeira Letra Maiúscula */
}
```


Exemplos Práticos:

```
.titulo-secao {
  text-transform: uppercase; /* "SOBRE NÓS" */
}

.nome-usuario {
  text-transform: capitalize; /* "João Silva" */
}

.email {
  text-transform: lowercase; /* "usuario@email.com" */
}
```

Using the text-transform property

THIS TEXT IS TRANSFORMED TO UPPERCASE.

this text is transformed to lowercase.

This Text Is Capitalized.

6.1.5. RECUO DA PRIMEIRA LINHA (TEXT-INDENT)

Define o **recuo da primeira linha** de um parágrafo.

Sintaxe: `text-indent: valor;`

Exemplos:

```
.paragrafo-recuado {
  text-indent: 2em;          /* Recuo de 2 vezes o tamanho da fonte */
  text-indent: 50px;        /* Recuo fixo de 50 pixels */
  text-indent: 10%;         /* Recuo de 10% da largura do elemento */
  text-indent: -20px;       /* Recuo negativo (hanging indent) */
}
```

Uso Prático:

```
.artigo p {
  text-indent: 1.5em;      /* Recuo tradicional de parágrafo */
}

.lista-pendente {
  text-indent: -20px;      /* Hanging indent para listas */
  padding-left: 20px;
}
```

6.1.6. ESPAÇAMENTO ENTRE CARACTERES (LETTER-SPACING)

Controla o **espaço entre caracteres** (*tracking*).

Sintaxe: `letter-spacing: valor;`

Exemplos:

```
.espacamentos {
  letter-spacing: normal; /* Espaçamento padrão */
  letter-spacing: 2px; /* Espaçamento aumentado */
  letter-spacing: -1px; /* Espaçamento reduzido */
  letter-spacing: 0.1em; /* Espaçamento relativo */
}
```

Uso Prático:

```
.titulo-elegante {
  letter-spacing: 3px; /* Títulos com espaçamento */
  text-transform: uppercase;
}

.texto-compacto {
  letter-spacing: -0.5px; /* Texto mais compacto */
}
```

Using letter-spacing

This is heading 1

This is heading 2

6.1.7. ESPAÇAMENTO ENTRE PALAVRAS (WORD-SPACING)

Controla o **espaço entre palavras**.

Sintaxe: `word-spacing: valor;`

Exemplos:

```
.espacamento-palavras {
  word-spacing: normal; /* Espaçamento padrão */
  word-spacing: 5px; /* Espaçamento aumentado */
  word-spacing: -2px; /* Espaçamento reduzido */
  word-spacing: 0.2em; /* Espaçamento relativo */
}
```

6.1.8. ALTURA DA LINHA (LINE-HEIGHT)

Define o **espaço vertical entre linhas** de texto.

Sintaxe: `line-height: valor;`

Valores:

```
.alturas-linha {
  line-height: normal;      /* Altura padrão (≈1.2) */
  line-height: 1.5;        /* 1.5 vezes o tamanho da fonte */
  line-height: 24px;       /* Altura fixa em pixels */
  line-height: 150%;       /* 150% do tamanho da fonte */
}
```

DICA

Line-height sem unidade é recomendado pois é herdado como multiplicador.

6.2. PROPRIEDADES DE FONTE

6.2.1. FAMÍLIA DA FONTE (FONT-FAMILY)

Especifica a **família de fontes** a ser usada.

Sintaxe: **font-family: fonte1, fonte2, família-genérica;**

Famílias Genéricas:

```
.familias-genericas {
  font-family: serif;      /* Fontes com serifa */
  font-family: sans-serif; /* Fontes sem serifa */
  font-family: monospace;  /* Fontes monoespaciaadas */
  font-family: cursive;    /* Fontes cursivas */
  font-family: fantasy;    /* Fontes decorativas */
}
```

Fontes Seguras (Web Safe):

```
.fontes-seguras {
  /* Serif */
  font-family: "Times New Roman", Times, serif;
  font-family: Georgia, serif;

  /* Sans-serif */
  font-family: Arial, Helvetica, sans-serif;
  font-family: "Trebuchet MS", sans-serif;
  font-family: Verdana, Geneva, sans-serif;

  /* Monospace */
  font-family: "Courier New", Courier, monospace;
  font-family: "Lucida Console", Monaco, monospace;
}
```

Stack de Fontes Modernas:

```
.fontes-modernas {
  /* Sistema operacional */
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;

  /* Google Fonts */
  font-family: "Roboto", Arial, sans-serif;
  font-family: "Open Sans", Helvetica, sans-serif;
}
```

CSS font-family

This is a paragraph, shown in the Times New Roman font.

This is a paragraph, shown in the Arial font.

This is a paragraph, shown in the Lucida Console font.

6.2.2. TAMANHO DA FONTE (FONT-SIZE)

Define o **tamanho da fonte**.

Unidades Absolutas:

```
.tamanhos-absolutos {
  font-size: 16px;      /* Pixels (mais comum) */
  font-size: 12pt;      /* Points (impressão) */
  font-size: 1cm;       /* Centímetros */
}
```

Unidades Relativas:

```
.tamanhos-relativos {
  font-size: 1em;       /* Relativo ao elemento pai */
  font-size: 1.2rem;     /* Relativo ao elemento raiz */
  font-size: 120%;       /* Percentual do elemento pai */
  font-size: 2ex;        /* Altura do "x" da fonte */
}
```

Palavras-chave:

```
.tamanhos-palavras {
  font-size: xx-small;   /* Muito pequeno */
  font-size: x-small;    /* Pequeno */
  font-size: small;      /* Pequeno */
  font-size: medium;     /* Médio (padrão) */
  font-size: large;      /* Grande */
  font-size: x-large;     /* Muito grande */
  font-size: xx-large;    /* Extremamente grande */
}
```

Tamanhos Recomendados:

```
.hierarquia-tipografica {
  /* Corpo do texto */
  font-size: 16px;           /* Base recomendada */

  /* Títulos */
  font-size: 2.5rem;         /* H1 - 40px */
  font-size: 2rem;           /* H2 - 32px */
  font-size: 1.5rem;         /* H3 - 24px */
  font-size: 1.25rem;        /* H4 - 20px */

  /* Textos menores */
  font-size: 0.875rem;       /* Small - 14px */
  font-size: 0.75rem;        /* Caption - 12px */
}
```

Como escolher a unidade certa?

- Para o controle fixo e preciso, px pode ser usado.
- Com px, a página web não vai escalar bem em diferentes tamanhos de tela ou com preferências do usuário.
- O tamanho absoluto pode ser útil quando o tamanho físico da saída é conhecido.
- Para webs de design escalável, use em ou a rem; eles permitem que os usuários ajustem o tamanho do texto em suas configurações do navegador.
- Porcentagem (%) pode ser útil para ajustar os tamanhos de fonte com base no pai Elementos.

Tamanhos Recomendados:

```
.hierarquia-tipografica {
  /* Corpo do texto */
  font-size: 16px;           /* Base recomendada */

  /* Títulos */
  font-size: 2.5rem;         /* H1 - 40px */
  font-size: 2rem;           /* H2 - 32px */
  font-size: 1.5rem;         /* H3 - 24px */
  font-size: 1.25rem;        /* H4 - 20px */

  /* Textos menores */
  font-size: 0.875rem;       /* Small - 14px */
  font-size: 0.75rem;        /* Caption - 12px */
}
```

This is heading 1

This is heading 2

This is a paragraph.

Specifying the font-size in rem allows all major browsers to resize the text.

6.2.3. PESO DA FONTE (FONTE-WEIGHT)

Controla a **espessura (peso) da fonte**.

Valores por Palavras:

```
.pesos-palavras {
  font-weight: normal;      /* Peso normal (400) */
  font-weight: bold;        /* Negrito (700) */
  font-weight: bolder;      /* Mais pesado que o pai */
  font-weight: lighter;     /* Mais leve que o pai */
}
```

Valores Numéricos:

```
.pesos-numericos {
  font-weight: 100;         /* Thin */
  font-weight: 200;         /* Extra Light */
  font-weight: 300;         /* Light */
  font-weight: 400;         /* Normal */
  font-weight: 500;         /* Medium */
  font-weight: 600;         /* Semi Bold */
  font-weight: 700;         /* Bold */
  font-weight: 800;         /* Extra Bold */
  font-weight: 900;         /* Black */
}
```

Uso prático:

```
.hierarquia-peso {
  /* Títulos */
  font-weight: 700;         /* Negrito para títulos */

  /* Texto normal */
  font-weight: 400;         /* Peso normal */

  /* Destaque sutil */
  font-weight: 500;         /* Medium para destaque */

  /* Texto secundário */
  font-weight: 300;         /* Light para textos secundários */
}
```

6.2.4. ESTILO DA FONTE (FONT-STYLE)

Define o **estilo da fonte (normal, itálico, oblíquo)**.

Valores:

```
.estilos-fonte {
  font-style: normal;       /* Estilo normal (padrão) */
  font-style: italic;       /* Itálico */
  font-style: oblique;      /* Oblíquo */
}
```

Uso Prático:

```
.citacao {
    font-style: italic;      /* Citações em itálico */
}

.enfase {
    font-style: italic;      /* Ênfase em palavras */
}

.normal {
    font-style: normal;      /* Remove itálico herdado */
}
```

The font-style property

This is a paragraph in normal style.

This is a paragraph in italic style.

This is a paragraph in oblique style.

6.2.5. VARIANTE DA FONTE (FONT-VARIANT)

Controla **variações da fonte, como small-caps**.

Valores:

```
.variantes {
    font-variant: normal;     /* Normal (padrão) */
    font-variant: small-caps; /* Versaletes */
}
```

Exemplo:

```
.titulo-elegante {
    font-variant: small-caps; /* "TÍTULO ELEGANTE" em versaletes */
    letter-spacing: 1px;
}
```

6.2.6. PROPRIEDADE ABREVIADA

Sintaxe: **font:** [style] [variant] [weight] size[/line-height] family;

Exemplos:

```
.font-abreviado {
    /* Mínimo obrigatório: size e family */
    font: 16px Arial;
```

```
/* Completo */
font: italic small-caps bold 16px/1.5 "Times New Roman", serif;

/* Comum */
font: 400 18px/1.6 "Helvetica Neue", sans-serif;
}
```

DICA

A propriedade font reseta todas as outras propriedades de fonte.

6.3. RECURSOS CSS3 PARA TIPOGRAFIA

6.3.1. SOMBRA DO TEXTO (TEXT-SHADOW)

Adiciona **sombra ao texto**.

Sintaxe: `text-shadow: offset-x offset-y blur-radius color;`

Exemplos:

```
.sombbras-texto {
  /* Sombra simples */
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);

  /* Sombra sem blur */
  text-shadow: 1px 1px 0 #000;

  /* Múltiplas sombras */
  text-shadow:
    1px 1px 0 #000,
    2px 2px 0 #333,
    3px 3px 0 #666;

  /* Sombra colorida */
  text-shadow: 0 0 10px #ff0080;
}
```

6.3.2. WEN FONTS (@FONT-FACE)

Permite usar **fontes** customizadas **não instaladas no sistema**.

Sintaxe Básica:

```
@font-face {
  font-family: "MinhaFonte";
  src: url("minhafonte.woff2") format("woff2"),
       url("minhafonte.woff") format("woff"),
       url("minhafonte.ttf") format("truetype");
  font-weight: normal;
  font-style: normal;
}
```



```
.com-fonte-customizada {
  font-family: "MinhaFonte", Arial, sans-serif;
}
```

6.3.3. UNIDADES RESPONSIVAS

Em vs Rem:

```
.unidades-responsivas {
  /* Em - relativo ao elemento pai */
  font-size: 1.2em;          /* 1.2 x tamanho do pai */

  /* Rem - relativo ao elemento raiz */
  font-size: 1.2rem;         /* 1.2 x tamanho do html */

  /* Viewport units */
  font-size: 4vw;            /* 4% da largura da viewport */
  font-size: 3vh;            /* 3% da altura da viewport */
  font-size: 2vmin;          /* 2% da menor dimensão */
  font-size: 2vmax;          /* 2% da maior dimensão */
}
```

Tipografia Responsiva:

```
/* Base */
html {
  font-size: 16px;          /* 1rem = 16px */
}

/* Títulos responsivos */
h1 {
  font-size: clamp(1.5rem, 4vw, 3rem); /* Min: 24px, Max: 48px */
}

/* Media queries para tipografia */
@media (max-width: 768px) {
  html {
    font-size: 14px;        /* 1rem = 14px em mobile */
  }
}
```


6.4. ESQUEMAS VISUAIS CONSOLIDADOS

6.4.1. PROPRIEDADES DE TEXTO

```

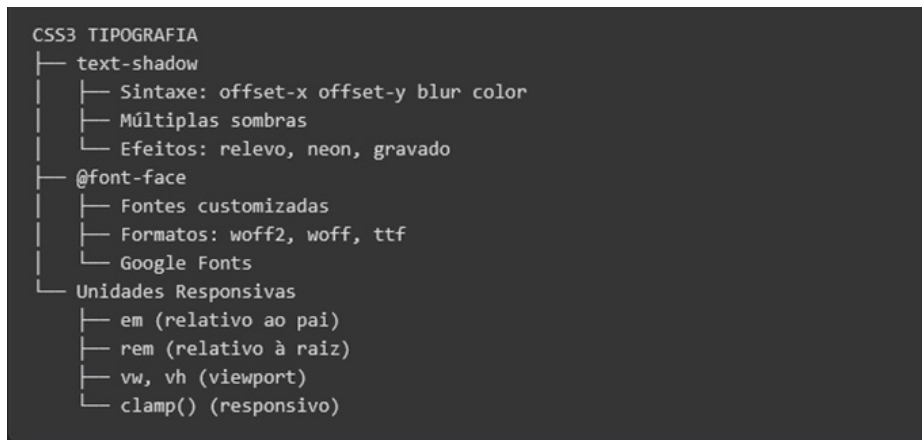
PROPRIEDADES DE TEXTO
├── color
│   └── Cor do texto (herdada)
├── text-align
│   ├── left, right, center, justify
│   └── Alinhamento horizontal
├── text-decoration
│   ├── none, underline, overline, line-through
│   └── Linhas decorativas
├── text-transform
│   ├── none, uppercase, lowercase, capitalize
│   └── Transformação de capitalização
├── text-indent
│   └── Recuo da primeira linha
├── letter-spacing
│   └── Espaçamento entre caracteres
├── word-spacing
│   └── Espaçamento entre palavras
└── line-height
    └── Altura da linha (espaçamento vertical)
    
```

6.4.2. PROPRIEDADES DE FONTE

```

PROPRIEDADES DE FONTE
├── font-family
│   ├── Família de fontes
│   ├── Fallbacks: serif, sans-serif, monospace
│   └── Web safe fonts
├── font-size
│   ├── Unidades: px, em, rem, %
│   ├── Palavras-chave: small, medium, large
│   └── Tamanho da fonte
├── font-weight
│   ├── Palavras: normal, bold, bolder, lighter
│   ├── Números: 100-900
│   └── Peso/espessura da fonte
├── font-style
│   ├── normal, italic, oblique
│   └── Estilo da fonte
├── font-variant
│   ├── normal, small-caps
│   └── Variações da fonte
└── font (abreviada)
    └── [style] [variant] [weight] size[/line-height] family
    
```

6.4.3. CSS3 TIPOGRAFIA



7. DISPLAY E POSICIONAMENTO

7.1. PROPRIEDADE DISPLAY

Display é uma das propriedades mais importantes do CSS, pois define **como** um elemento **é exibido** e **como se comporta** no *layout* da página.

A propriedade display determina o **tipo de caixa de renderização** usada para um elemento, controlando seu **comportamento** no fluxo do documento e como interage com outros elementos.

7.1.1. DISPLAY BLOCK

Elementos de bloco **ocupam toda a largura disponível e quebram linha**.

Características:

- **Largura:** 100% do container pai (por padrão)
- **Altura:** Determinada pelo conteúdo
- **Quebra de linha:** Antes e depois do elemento
- **Aceita:** width, height, margin, padding
- **Empilhamento:** Vertical (um abaixo do outro)

Elementos Block por Padrão:

```

<div>, <p>, <h1>-<h6>, <ul>, <ol>, <li>, <section>, <article>, <header>, <footer>,
<nav>, <main>, <aside>, <blockquote>, <form>, <fieldset>, <table>, <hr>
    
```

Exemplo Prático:

CSS:

```
.bloco {
  display: block;
  width: 300px;      /* Largura específica */
  height: 100px;     /* Altura específica */
  background: lightblue;
  margin: 10px;      /* Margem em todos os lados */
  padding: 20px;     /* Padding em todos os lados */
}
```

HTML:

```
<div class="bloco">Elemento de bloco 1</div>
<div class="bloco">Elemento de bloco 2</div>
<!-- Cada div ocupa uma linha inteira -->
```

7.1.2. DISPLAY INLINE

Elementos inline **fluem horizontalmente e não quebram linha.**

Características:

- **Largura:** Determinada pelo conteúdo
- **Altura:** Determinada pelo conteúdo
- **Quebra de linha:** Não quebra linha
- **Width/Height:** Não aceita width e height
- **Margin/Padding:** Apenas horizontal (left/right)
- **Empilhamento:** Horizontal (lado a lado)

Elementos Inline por Padrão:

```
<span>, <a>, <strong>, <em>, <b>, <i>, <u>, <small>, <sub>, <sup>, <code>, <kbd>,
<var>, <samp>, <time>, <mark>, <del>, <ins>
```

Exemplo Prático:

CSS:

```
.inline {
  display: inline;
  background: yellow;
  margin: 10px;      /* Apenas left/right funcionam */
  padding: 5px;      /* Apenas left/right funcionam */
  /* width: 200px;    IGNORADO em elementos inline */
  /* height: 100px;   IGNORADO em elementos inline */
}
```

HTML:

```
<span class="inline">Texto 1</span>
<span class="inline">Texto 2</span>
<span class="inline">Texto 3</span>
<!-- Todos ficam na mesma linha -->
```

7.1.3. DISPLAY INLINE-BLOCK

Combina características de **inline e block**.

Características:

- **Fluxo:** Horizontal como inline
- **Dimensões:** Aceita width e height como block
- **Margin/Padding:** Funciona em todas as direções
- **Quebra de linha:** Não quebra linha automaticamente
- **Empilhamento:** Horizontal com controle de dimensões

Exemplo Prático:

CSS:

```
.inline-block {
  display: inline-block;
  width: 150px;      /* Funciona! */
  height: 100px;     /* Funciona! */
  background: lightgreen;
  margin: 10px;      /* Funciona em todas as direções */
  padding: 15px;     /* Funciona em todas as direções */
  vertical-align: top; /* Controla alinhamento vertical */
}
```

HTML:

```
<div class="inline-block">Caixa 1</div>
<div class="inline-block">Caixa 2</div>
<div class="inline-block">Caixa 3</div>
<!-- Ficam lado a lado, mas com dimensões controláveis -->
```

7.1.4. DISPLAY NONE

Remove completamente o elemento do layout.

Características:

- **Visibilidade:** Elemento não é exibido
- **Espaço:** Não ocupa espaço no layout
- **Interação:** Não pode ser clicado ou focado
- **DOM:** Elemento permanece no DOM

Exemplo:

CSS:

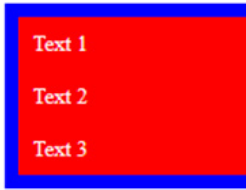
```
.escondido {
    display: none;      /* Elemento desaparece completamente */
}

.visivel {
    display: block;     /* Elemento volta a aparecer */
}
```

Diferença: Display: none vs Visibility: hidden

Propriedade	Visibilidade	Ocupa Espaço	Interação
display: none	Invisível	Não ocupa	Não interage
visibility: hidden	Invisível	Ocupa espaço	Não interage
opacity: 0	Invisível	Ocupa espaço	Interage

7.1.5. OUTROS VALORES DE DISPLAY

Valor	Descrição	
inline	Exibe um elemento como um elemento em linha	display: inline: 
block	Exibe um elemento como um elemento em bloco	display: block: 
contents	Faz o contêiner desaparecer, tornando seus elementos filhos filhos do elemento do próximo nível no DOM	
flex	Exibe um elemento como um contêiner flexível em nível de bloco	display: flex: 
grid	Exibe um elemento como um contêiner de grade em nível de bloco	display: grid: 

inline-block	Exibe um elemento como um contêiner de bloco em nível de linha. O elemento em si é formatado como um elemento em linha, mas você pode aplicar altura, largura, preenchimento e margens	display: inline-block: 
none	O elemento fica completamente oculto do fluxo do documento (não ocupa nenhum espaço)	

7.2. PROPRIEDADE POSICIONAMENTO (POSITION)

Position controla **como um elemento é posicionado no documento** e como se relaciona com outros elementos.

Propriedades de Offset (usadas com position):

- **top** – distância do topo
- **right** – distância da direita
- **bottom** – distância de baixo
- **left** – distância da esquerda

7.2.1. POSITION STATIC (PADRÃO)

Posicionamento normal no fluxo do documento.

Características:

- **Padrão:** Valor padrão de todos os elementos
- **Offset:** Propriedades top, right, bottom, left são ignoradas
- **Fluxo:** Segue o fluxo normal do documento
- **Z-index:** Não aceita z-index

Exemplo:

```
.estatico {
  position: static;      /* Valor padrão */
  /* top: 50px;          IGNORADO */
  /* left: 100px;        IGNORADO */
}
```

7.2.2. POSITION RELATIVE

Posicionado relativo à sua posição original no fluxo normal.

Características:

- **Referência:** Sua posição original no fluxo
- **Offset:** Aceita top, right, bottom, left
- **Espaço:** Mantém o espaço original no layout

- **Z-index:** Aceita z-index
- **Contexto:** Cria contexto de empilhamento

Exemplo:

CSS:

```
.relativo {
  position: relative;
  top: 20px;      /* Move 20px para baixo */
  left: 30px;     /* Move 30px para direita */
  background: lightcoral;
}
```

7.2.3. POSITION ABSOLUTE

Posicionado relativo ao ancestral posicionado mais próximo.

Características:

- **Referência:** Ancestral com position ≠ static
- **Offset:** Aceita top, right, bottom, left
- **Fluxo:** Removido do fluxo normal
- **Espaço:** Não ocupa espaço no layout
- **Z-index:** Aceita z-index
- **Sobreposição:** Pode sobrepor outros elementos

Exemplo:

CSS:

```
.absoluto {
  position: absolute;
  top: 50px;      /* 50px do topo do container posicionado */
  right: 20px;    /* 20px da direita do container posicionado */
  width: 200px;
  height: 100px;
  background: lightgreen;
}
```

7.2.4. POSITION FIXED

Posicionado relativo à viewport (janela do navegador).

Características:

- **Referência:** Viewport (janela do navegador)
- **Offset:** Aceita top, right, bottom, left
- **Fluxo:** Removido do fluxo normal
- **Scroll:** Não rola com a página
- **Z-index:** Aceita z-index
- **Fixo:** Permanece fixo na tela

Exemplo – Header Fixo:

```
.header-fixa {
  position: fixed;
  top: 0;           /* Grudado no topo da tela */
  left: 0;          /* Grudado na esquerda */
  width: 100%;      /* Largura total */
  height: 60px;
  background: #333;
  z-index: 1000;    /* Acima de outros elementos */
}
```

Exemplo –Botão Flutuante:

```
.botao-flutuante {
  position: fixed;
  bottom: 20px;     /* 20px do fundo da tela */
  right: 20px;      /* 20px da direita da tela */
  width: 60px;
  height: 60px;
  border-radius: 50%;
  background: #007bff;
  z-index: 999;
}
```

7.2.5. POSITION STICKY (CSS3)

Combina relative e fixed baseado na posição de scroll.

Características:

- **Híbrido:** Relative até atingir threshold, depois fixed
- **Threshold:** Definido por top, right, bottom, left
- **Container:** Limitado ao container pai
- **Scroll:** Comportamento baseado no scroll
- **Z-index:** Aceita z-index

Exemplo – Menu Sticky:

```
.menu-sticky {
  position: sticky;
  top: 0;           /* Fica fixo quando chegar no topo */
  background: white;
  border-bottom: 1px solid #ddd;
  z-index: 100;
}
```

7.3. Z-INDEX E EMPILHAMENTO

Z-index controla a **ordem de empilhamento** de elementos posicionados no eixo Z (profundidade).

Características:

- **Aplicação:** Apenas em elementos posicionados (position ≠ static)
- **Valores:** Números inteiros (positivos, negativos, zero)
- **Padrão:** auto (equivale a 0)
- **Maior valor:** Fica por cima

7.3.1. COMO FUNCIONA O Z-INDEX

Regras Básicas:

```
.atras {
  position: relative;
  z-index: 1;      /* Atrás */
  background: red;
}

.meio {
  position: relative;
  z-index: 2;      /* No meio */
  background: green;
}

.frente {
  position: relative;
  z-index: 3;      /* Na frente */
  background: blue;
}
```

Valores Comuns:

```
.modal-overlay {
  z-index: 1000;   /* Overlay de modal */
}

.modal {
  z-index: 1001;   /* Modal acima do overlay */
}

.tooltip {
  z-index: 1002;   /* Tooltip acima do modal */
}

.dropdown {
  z-index: 100;    /* Dropdown menu */
}

.header-fixa {
  z-index: 50;     /* Header fixa */
}
```

7.3.2. CONCEITO DE EMPILHAMENTO

Elementos que **criam um novo “contexto” de empilhamento local.**

Criam Contexto de Empilhamento:

- position: relative/absolute/fixed/sticky com z-index ≠ auto
- opacity < 1
- transform ≠ none
- filter ≠ none
- isolation: isolate

Exemplo de Problema:

```
.pai1 {
  position: relative;
  z-index: 1;
}

.filho1 {
  position: relative;
  z-index: 9999; /* Não pode ficar acima de .pai2 */
}

.pai2 {
  position: relative;
  z-index: 2; /* Maior que .pai1 */
}

.filho2 {
  position: relative;
  z-index: 1;
}
```

Resultado: .filho2 fica acima de .filho1 porque .pai2 tem z-index maior que .pai1.

7.4. FLOAT E CLEAR

Float era usado para layout antes do Flexbox/Grid, mas ainda é útil para **texto ao redor de imagens. Remove o elemento do fluxo normal e o “flutua” para um lado.**

7.4.1. PROPRIEDADE FLOAT

Valores:

```
.flutuante {
  float: left; /* Flutua para esquerda */
  float: right; /* Flutua para direita */
  float: none; /* Não flutua (padrão) */
}
```

O que é **float** Propriedade específica como um elemento deve flutuar dentro do seu recipiente.

Ele coloca um elemento no lado esquerdo ou direito do seu recipiente, permitindo que o texto e os elementos em linha o enrolem.

A imagem dentro deste ?div? é flutuada para a direita.

Se você fizer a tela menor, verá que o texto envolve o Imagem.



7.4.2. PROPRIEDADE CLEAR

Clear **impede que um elemento fique ao lado de elementos flutuantes.**

Valores:

```
.limpar {
  clear: left;      /* Não permite float à esquerda */
  clear: right;     /* Não permite float à direita */
  clear: both;      /* Não permite float em nenhum lado */
  clear: none;      /* Permite float (padrão) */
}
```



7.5. ESQUEMAS VISUAIS CONSOLIDADOS

7.5.1. VALORES DE DISPLAY

```
PROPRIEDADE DISPLAY
├── BLOCK
│   ├── Largura: 100% do container
│   ├── Aceita: width, height, margin, padding
│   ├── Quebra linha: antes e depois
│   └── Exemplos: div, p, h1-h6, section
├── INLINE
│   ├── Largura: determinada pelo conteúdo
│   ├── Não aceita: width, height
│   ├── Margin/padding: apenas horizontal
│   └── Exemplos: span, a, strong, em
├── INLINE-BLOCK
│   ├── Fluxo: horizontal como inline
│   ├── Aceita: width, height, margin, padding
│   ├── Uso: botões, caixas lado a lado
│   └── Híbrido: melhor dos dois mundos
└── NONE
    ├── Remove do layout
    ├── Não ocupa espaço
    └── Diferente de visibility: hidden
```

7.5.2. TIPOS DE POSITION

```

PROPRIEDADE POSITION
├── STATIC (padrão)
│   ├── Fluxo normal do documento
│   ├── Ignora: top, right, bottom, left
│   └── Não aceita z-index
├── RELATIVE
│   ├── Relativo à posição original
│   ├── Mantém espaço no layout
│   ├── Aceita offset e z-index
│   └── Cria contexto de empilhamento
├── ABSOLUTE
│   ├── Relativo ao ancestral posicionado
│   ├── Removido do fluxo normal
│   ├── Não ocupa espaço
│   └── Aceita offset e z-index
├── FIXED
│   ├── Relativo à viewport
│   ├── Não rola com a página
│   ├── Removido do fluxo normal
│   └── Uso: headers fixos, botões flutuantes
└── STICKY (CSS3)
    ├── Híbrido: relative + fixed
    ├── Baseado na posição de scroll
    ├── Limitado ao container pai
    └── Uso: menus que grudam no topo
    
```

7.5.3. Z-INDEX E EMPILHAMENTO

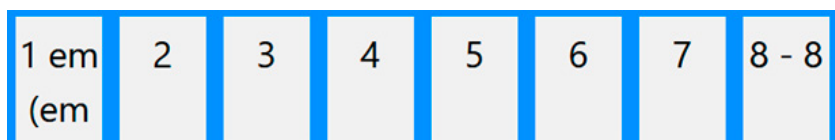
```

Z-INDEX E EMPILHAMENTO
├── APLICAÇÃO
│   ├── Apenas elementos posicionados
│   ├── position ≠ static
│   └── Valores: números inteiros
├── REGRAS
│   ├── Maior valor = mais acima
│   ├── Padrão: auto (≈ 0)
│   ├── Negativos: atrás do contexto pai
│   └── Positivos: acima do contexto pai
├── CONTEXTO DE EMPILHAMENTO
│   ├── position + z-index ≠ auto
│   ├── opacity < 1
│   ├── transform ≠ none
│   └── Cria "camada" local
└── VALORES COMUNS
    ├── Modal: 1000+
    ├── Dropdown: 100-999
    ├── Header fixo: 50-99
    └── Elementos normais: 1-49
    
```

8. FLEXBOX E GRID

8.1. FLEXBOX – LAYOUT UNIDIMENSIONAL

Flexbox é um sistema de **layout unidimensional** (linha OU coluna) que facilita **alinhamento e distribuição de espaço**.



Ativação:

```
.container {
  display: flex;          /* Ativa flexbox */
  display: inline-flex;   /* Flexbox inline */
}
```

Eixos do Flexbox:

- Main Axis (Principal): Direção dos itens
- Cross Axis (Cruzado): Perpendicular ao principal

8.1.1. PROPRIEDADES DO CONTAINER FLEX

Direção dos Itens:

```
.container {
  flex-direction: row;      /* Horizontal → (padrão) */
  flex-direction: column;   /* Vertical ↓ */
  flex-direction: row-reverse; /* Horizontal ← */
  flex-direction: column-reverse; /* Vertical ↑ */
}
```

Alinhamento Principal (justify-content):

```
.container {
  justify-content: flex-start; /* Início (padrão) */
  justify-content: center;     /* Centro */
  justify-content: flex-end;   /* Final */
  justify-content: space-between; /* Espaço entre */
  justify-content: space-around; /* Espaço ao redor */
  justify-content: space-evenly; /* Espaço igual */
}
```

Alinhamento Cruzado (align-items):

```
.container {
  align-items: stretch; /* Estica (padrão) */
  align-items: flex-start; /* Início */
  align-items: center; /* Centro */
  align-items: flex-end; /* Final */
  align-items: baseline; /* Linha de base */
}
```

Quebra de Linha:

```
.container {
  flex-wrap: nowrap; /* Não quebra (padrão) */
  flex-wrap: wrap; /* Quebra linha */
  flex-wrap: wrap-reverse; /* Quebra reversa */
}
```

8.1.2. PROPRIEDADES DOS ITENS FLEX

Crescimento e Encolhimento:

```
.item {
  flex-grow: 1; /* Cresce proporcionalmente */
  flex-shrink: 1; /* Encolhe proporcionalmente */
  flex-basis: auto; /* Tamanho base */

  /* Propriedade abreviada */
  flex: 1; /* flex: 1 1 0% */
  flex: auto; /* flex: 1 1 auto */
  flex: none; /* flex: 0 0 auto */
}
```

Alinhamento Individual:

```
.item {
  align-self: center; /* Sobrescreve align-items */
  order: 2; /* Ordem visual */
}
```

8.1.3. PADRÕES FLEXBOX ESSENCIAIS

Centralização Perfeita:

```
.centro {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Layout 3 Colunas:

```
.layout {
  display: flex;
}
.sidebar { flex: 0 0 200px; } /* Fixa 200px */
.content { flex: 1; } /* Flexível */
.aside { flex: 0 0 150px; } /* Fixa 150px */
```

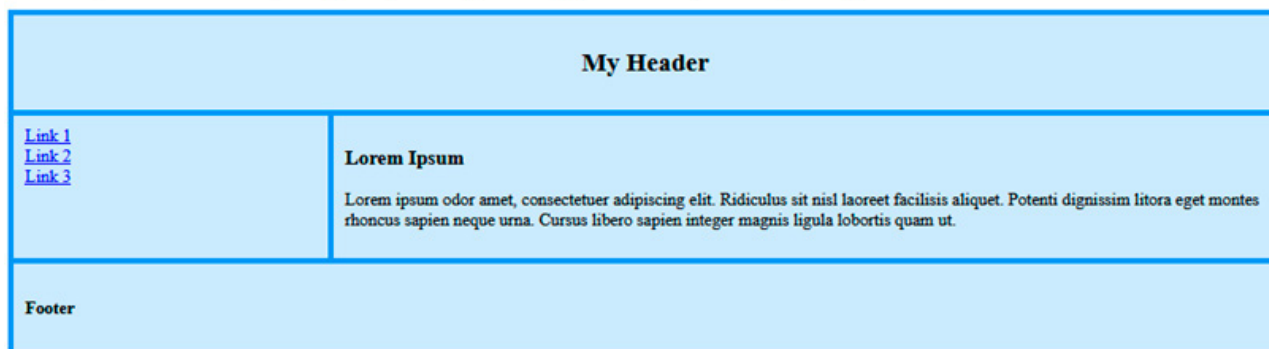
8.2. GRID – LAYOUT BIDIMENSIONAL

CSS Grid é um sistema de **layout bidimensional (linhas E colunas)** para layouts complexos.

CSS Grid Layout

The Grid Layout Module offers a grid-based layout system, with rows and columns.

The Grid Layout Module makes it easy to design complex and responsive web pages without using floats and positioning:



Ativação:

```
.container {
  display: grid; /* Ativa grid */
  display: inline-grid; /* Grid inline */
}
```

Terminologia:

- **Grid Container:** Elemento **pai** com display: grid
- **Grid Items:** **Filhos** diretos do container
- **Grid Lines:** Linhas que dividem o grid
- **Grid Tracks:** Espaço entre duas linhas (row/column)
- **Grid Cells:** Interseção de row e column
- **Grid Areas:** Retângulo formado por múltiplas células

8.2.1. DEFININDO O GRID

```
.container {
  /* Colunas */
  grid-template-columns: 200px 1fr 100px;      /* 3 colunas */
  grid-template-columns: repeat(3, 1fr);        /* 3 colunas iguais */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* Responsivo */

  /* Linhas */
  grid-template-rows: 100px auto 50px;         /* 3 linhas */
  grid-template-rows: repeat(3, 100px);        /* 3 linhas de 100px */
}
```

Unidades Grid:

```
.container {
  grid-template-columns:
    1fr 2fr 1fr;      /* Frações (1:2:1) */
    200px auto 100px; /* Fixo + flexível + fixo */
    minmax(100px, 1fr); /* Mínimo 100px, máximo flexível */
    fit-content(200px); /* Ajusta ao conteúdo, máx 200px */
}
```

Espaçamento:

```
.container {
  gap: 20px;          /* Espaço entre itens */
  row-gap: 20px;      /* Espaço entre linhas */
  column-gap: 30px;   /* Espaço entre colunas */
}
```

8.2.2. POSICIONAMENTO DE ITENS

Por Linhas:

```
.item {
  grid-column-start: 1;
  grid-column-end: 3;      /* Ocupa colunas 1-2 */

  /* Abreviado */
  grid-column: 1 / 3;      /* Colunas 1-2 */
  grid-row: 2 / 4;         /* Linhas 2-3 */

  /* Span (extensão) */
  grid-column: span 2;      /* Ocupa 2 colunas */
  grid-row: span 3;         /* Ocupa 3 linhas */
}
```


Por Áreas Nomeadas:

```
.container {
  grid-template-areas:
    "header header header"
    "sidebar main aside"
    "footer footer footer";
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.aside { grid-area: aside; }
.footer { grid-area: footer; }
```

8.2.3. ALINHAMENTO NO GRID

Alinhamento de Itens:

```
.container {
  /* Alinhamento horizontal */
  justify-items: start; /* início, center, end, stretch */

  /* Alinhamento vertical */
  align-items: center; /* início, center, end, stretch */

  /* Abreviado */
  place-items: center; /* align-items + justify-items */
}
```

Alinhamento do Grid:

```
.container {
  /* Alinhamento horizontal do grid */
  justify-content: center; /* start, center, end, space-between, etc. */

  /* Alinhamento vertical do grid */
  align-content: center; /* start, center, end, space-between, etc. */

  /* Abreviado */
  place-content: center; /* align-content + justify-content */
}
```

8.2.4. GRID RESPONSIVO

Auto-fit e Auto-fill:

```
.container {
  /* Auto-fit: colapsa tracks vazias */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));

  /* Auto-fill: mantém tracks vazias */
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
}
```

Grid Implícito:

```
.container {
  grid-auto-rows: 200px; /* Altura de linhas implícitas */
  grid-auto-columns: 1fr; /* Largura de colunas implícitas */
  grid-auto-flow: row; /* Direção: row, column, dense */
}
```

8.3. FLEXBOX VS GRID

8.3.1. COMPARAÇÃO PRÁTICA

Aspecto	Flexbox	Grid
Dimensão	Unidimensional (linha O U coluna)	Bidimensional (linhas E colunas)
Uso Principal	Componentes, alinhamento	Layouts de página
Controle	Baseado no conteúdo	Baseado no container
Responsividade	Flexível por natureza	Requer configuração
Complexidade	Simples	Mais complexo

8.3.2. CASOS DE USO

Use Flexbox para:

- Navegação horizontal
- Centralização de elementos
- Distribuição de espaço em uma direção
- Componentes flexíveis (cards, botões)
- Alinhamento de itens

Use Grid para:

- Layouts de página completos
- Grids de conteúdo (galerias, produtos)
- Layouts complexos bidimensionais
- Posicionamento preciso de elementos
- Sobreposição de elementos

8.3.3. COMBINANDO FLEXBOX E GRID

```
/* Grid para layout geral */
.page {
  display: grid;
  grid-template-areas:
    "header"
    "main"
    "footer";
  min-height: 100vh;
}

/* Flexbox para componentes */
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 2rem;
}

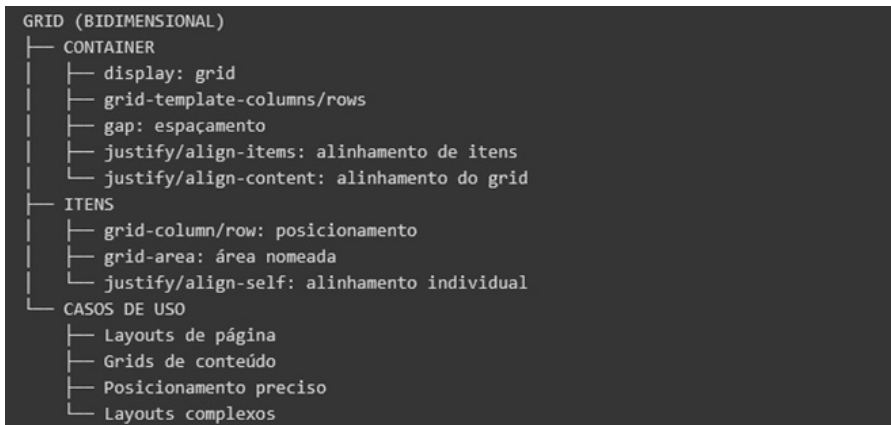
.card {
  display: flex;
  flex-direction: column;
}
```

8.4. ESQUEMAS VISUAIS CONSOLIDADOS

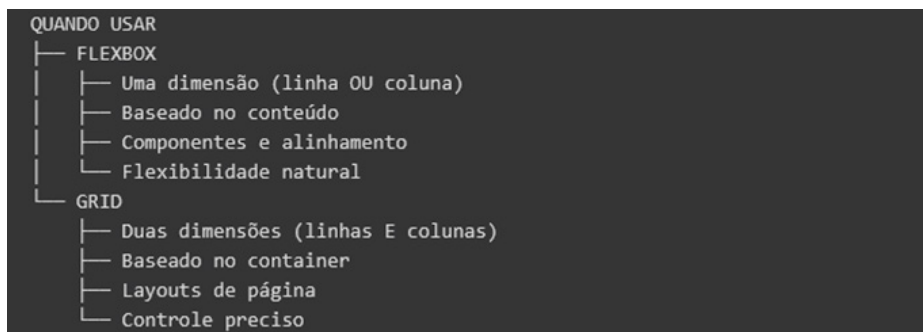
8.4.1. FLEXBOX ESSENCIAL

```
FLEXBOX (UNIDIMENSIONAL)
├─ CONTAINER
│  ├── display: flex
│  ├── flex-direction: row/column
│  ├── justify-content: alinhamento principal
│  ├── align-items: alinhamento cruzado
│  └── flex-wrap: quebra de linha
├─ ITENS
│  ├── flex: grow shrink basis
│  ├── align-self: alinhamento individual
│  └── order: ordem visual
└─ CASOS DE USO
   ├── Navegação horizontal
   ├── Centralização
   ├── Distribuição de espaço
   └── Componentes flexíveis
```

8.4.2. GRID ESSENCIAL



8.4.3. FLEXBOX VS GRID



9. RESPONSIVIDADE

9.1. CONCEITOS FUNDAMENTAIS

9.1.1. O QUE É DESIGN RESPONSIVO?

Design Responsivo é uma abordagem que faz com que **páginas web se adaptem automaticamente a diferentes tamanhos de tela e dispositivos**. Consiste em uma técnica de desenvolvimento web que utiliza layouts flexíveis, imagens adaptáveis e media queries CSS para criar uma experiência otimizada em qualquer dispositivo.



9.1.2. PILARES DO DESIGN RESPONSIVO

9.1.2.1. LAYOUT FLEXÍVEL

O **layout flexível** em um site responsivo busca considerar os **diferentes pontos de interatividade** e as **variadas dimensões de tela**, de modo a criar uma unidade visual que se adapte de forma adequada a cada contexto.

```
.container {
  width: 100%;
  max-width: 1200px;
  margin: 0 auto;
}
```

9.1.2.2. MÍDIAS FLEXÍVEIS

Outro pilar do design responsivo são as **mídias flexíveis**, como imagens e vídeos, que **devem adaptar-se a diferentes dispositivos**. O CSS auxilia nesse processo, controlando dimensões e tornando as mídias responsivas conforme o tipo e tamanho de tela.

```
img {
  max-width: 100%;
  height: auto;
}
```

9.1.2.3. MEDIA QUERIES

Media Queries são **regras** de CSS que permitem **definir estilos específicos para diferentes dispositivos**, ajustando o design do site conforme suas características. Com isso, é possível adaptar dimensões e estilos em um único arquivo, garantindo maior flexibilidade e responsividade.

```
@media (max-width: 768px) {
  .container {
    padding: 10px;
  }
}
```

9.1.3. VIEWPORT META TAG

A **Meta Tag Viewport**, inserida no **<head>** do site, **orienta** os navegadores sobre como **ajustar as dimensões** da página em **diferentes dispositivos**, especialmente móveis. Essencial para o design responsivo, garante a exibição correta do conteúdo sem zoom ou rolagem lateral, melhorando a experiência do usuário.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Propriedades do Viewport:

- **width=device-width** – largura igual ao dispositivo
- **initial-scale=1.0** – zoom inicial 1:1
- **maximum-scale=5.0** – zoom máximo
- **minimum-scale=0.5** – zoom mínimo
- **user-scalable=yes** – permite zoom do usuário



Without the viewport meta tag



With the viewport meta tag

9.2. MEDIA QUERIES

Media query é uma técnica CSS introduzida no CSS3. Ele usa o @media **regra** para **incluir um bloco de propriedades CSS somente se um Uma condição é verdadeira**.

9.2.1. SINTAXE BÁSICA

Estrutura Geral:

```
@media media-type and (media-feature) {  
  /* CSS para essa condição */  
}
```

Exemplo Prático:

```
@media screen and (max-width: 768px) {  
  .sidebar {  
    display: none;  
  }  
}
```

9.2.2. MEDIA TYPES

Tipos de Mídia:

```
@media screen { } /* Telas (padrão) */
@media print { } /* Impressão */
@media speech { } /* Leitores de tela */
@media all { } /* Todos os tipos */
```

Mais Usado:

```
/* Screen é o padrão, pode ser omitido */
@media (max-width: 768px) { }
```

9.2.3. MEDIA FEATURES ESSENCIAIS

Largura (Width)

```
@media (width: 768px) { } /* Exatamente 768px */
@media (min-width: 768px) { } /* 768px ou mais */
@media (max-width: 767px) { } /* 767px ou menos */
```

Altura (Height):

```
@media (height: 600px) { } /* Exatamente 600px */
@media (min-height: 600px) { } /* 600px ou mais */
@media (max-height: 599px) { } /* 599px ou menos */
```

Orientação:

```
@media (orientation: portrait) { } /* Retrato (altura > largura) */
@media (orientation: landscape) { } /* Paisagem (largura > altura) */
```



```
@media (resolution: 300dpi) { } /* Resolução específica */
@media (min-resolution: 2dppx) { } /* Telas de alta densidade */
```

9.2.4. OPERADORES LÓGICOS

AND (E):

```
@media screen and (min-width: 768px) and (max-width: 1024px) {
  /* Entre 768px e 1024px */
}
```

OR (Vírgula):

```
@media (max-width: 768px), (orientation: portrait) {  
    /* Até 768px OU orientação retrato */  
}
```

NOT (não):

```
@media not screen and (max-width: 768px) {  
    /* Não tela OU não até 768px */  
}
```

ONLY (Apenas):

```
@media only screen and (max-width: 768px) {  
    /* Apenas telas até 768px */  
}
```

9.3. BREAKPOINTS PADRÃO

9.3.1. BREAKPOINTS COMUNS

Sistema Bootstrap (Muito Usado):

```
/* Extra Small (xs) */  
@media (max-width: 575.98px) { }  
  
/* Small (sm) */  
@media (min-width: 576px) { }  
@media (min-width: 576px) and (max-width: 767.98px) { }  
  
/* Medium (md) */  
@media (min-width: 768px) { }  
@media (min-width: 768px) and (max-width: 991.98px) { }  
  
/* Large (lg) */  
@media (min-width: 992px) { }  
@media (min-width: 992px) and (max-width: 1199.98px) { }  
  
/* Extra Large (xl) */  
@media (min-width: 1200px) { }  
  
/* Extra Extra Large (xxl) */  
@media (min-width: 1400px) { }
```

Sistema Simplificado:

```
/* Mobile First */  
/* Mobile: 0-767px (padrão) */  
  
/* Tablet */  
@media (min-width: 768px) { }
```



```
/* Desktop */
@media (min-width: 1024px) { }

/* Large Desktop */
@media (min-width: 1200px) { }
```

9.3.2. DISPOSITIVOS TÍPICOS

Dispositivo	Largura Típica	Breakpoint
Smartphone	320px – 767px	max-width: 767px
Tablet	768px – 1023px	min-width: 768px
Desktop	1024px – 1199px	min-width: 1024px
Large Desktop	1200px+	min-width: 1200px

9.4. MOBILE-FIRST APPROACH

9.4.1. CONCEITO MOBILE-FIRST

O conceito de **Mobile First** propõe que, ao desenvolver um site ou outro produto, o **mobile seja levado em conta primeiro**. Ao contrário do que se fazia antes, as adaptações devem ser feitas de mobile para desktop.

Vantagens:

- **Performance:** Carrega menos CSS em mobile
- **Priorização:** Foca no essencial primeiro
- **Progressivo:** Adiciona recursos conforme espaço
- **Moderno:** Abordagem atual da indústria

9.4.2. MOBILE-FIRST VS DESKTOP-FIRST

Desktop-First (Antigo):

```
/* CSS padrão para desktop */
.container {
  width: 1200px;
  display: flex;
}

/* Adaptações para mobile */
@media (max-width: 768px) {
  .container {
    width: 100%;
    display: block;
  }
}
```

Mobile-First (Moderno):

```
/* CSS padrão para mobile */
.container {
  width: 100%;
  display: block;
}

/* Melhorias para desktop */
@media (min-width: 768px) {
  .container {
    width: 1200px;
    display: flex;
  }
}
```

9.5. UNIDADES RESPONSIVAS

9.5.1. UNIDADE VIEWPORT

A **viewport** é a **área visível da página da web** para o usuário. Essa área varia conforme o dispositivo: em um smartphone, por exemplo, a janela de visualização é menor do que em um computador.

Antes dos dispositivos móveis, as páginas tinham design fixo para desktop. Com celulares e tablets, esse modelo não se adaptava, e os navegadores passaram a **redimensionar automaticamente as páginas para caber na tela**.

No HTML5 foi criado o atributo **viewport**, configurado por meio da **tag <meta>**, para que o desenvolvedor pudesse controlar como a página será exibida em diferentes dispositivos (celulares, tablets, desktops). Esse código deve ser colocado **dentro da seção <head>** da página:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- **width=device-width:** Diz ao navegador que a largura da página deve se ajustar exatamente à largura da tela do dispositivo.

EXEMPLO

Se a tela tem 375px de largura (como em muitos smartphones), a página será renderizada com 375px de largura também.

- **initial-scale=1.0:** Define o nível de zoom inicial.
 - O valor 1.0 significa que a página será exibida sem zoom (escala real). Se fosse 0.5, a página apareceria reduzida pela metade; se fosse 2.0, apareceria com zoom de 200%.

9.5.2. UNIDADES RELATIVAS

Em vs Rem:

```
/* Em: relativo ao pai */
.parent {
  font-size: 20px;
}

.child {
  font-size: 1.5em; /* 30px (20px * 1.5) */
}

/* Rem: relativo à raiz */
html {
  font-size: 16px;
}

.element {
  font-size: 1.5rem; /* 24px (16px * 1.5) */
}
```

Percentuais:

```
.container {
  width: 90%; /* 90% do pai */
  max-width: 1200px; /* Máximo 1200px */
  margin: 0 auto; /* Centralizado */
}
```

9.5.3. TIPOGRAFIA RESPONSIVA

Tipografia responsiva é um estilo de design que **ajusta automaticamente a apresentação do texto** para melhorar a legibilidade em diferentes dispositivos. Isso inclui mudanças no tamanho da fonte, espaçamento entre letras e linhas, e outras propriedades tipográficas, dependendo das necessidades específicas de cada tela.

Clamp() – CSS Moderno:

```
.title {
  font-size: clamp(1.5rem, 4vw, 3rem);
  /* Mínimo: 1.5rem, Ideal: 4vw, Máximo: 3rem */
}
```

Media Queries Tradicionais:

```
.title {
  font-size: 1.5rem; /* Mobile */
}

@media (min-width: 768px) {
  .title {
    font-size: 2rem; /* Tablet */
  }
}
```

```
@media (min-width: 1024px) {
  .title {
    font-size: 2.5rem; /* Desktop */
  }
}
```

9.6. TÉCNICAS RESPONSIVAS AVANÇADAS

9.6.1. CONTAINER QUERIES

```
.card-container {
  container-type: inline-size;
}

@container (min-width: 300px) {
  .card {
    display: flex;
  }
}
```

9.6.2. IMAGENS RESPONSIVAS

Srcset e Sizes:

```

```

Picture Element:

```
<picture>
  <source media="(min-width: 1024px)" srcset="desktop.jpg">
  <source media="(min-width: 768px)" srcset="tablet.jpg">
  
</picture>
```

CSS Object-fit:

```
.responsive-image {
  width: 100%;
  height: 200px;
  object-fit: cover; /* cover, contain, fill, scale-down */
  object-position: center; /* Posicionamento */
}
```

9.6.3. GRIDS RESPONSIVOS

CSS Grid Auto-fit:

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 2rem;
}
```

Flexbox Responsivo:

```
.flex-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 2rem;
}

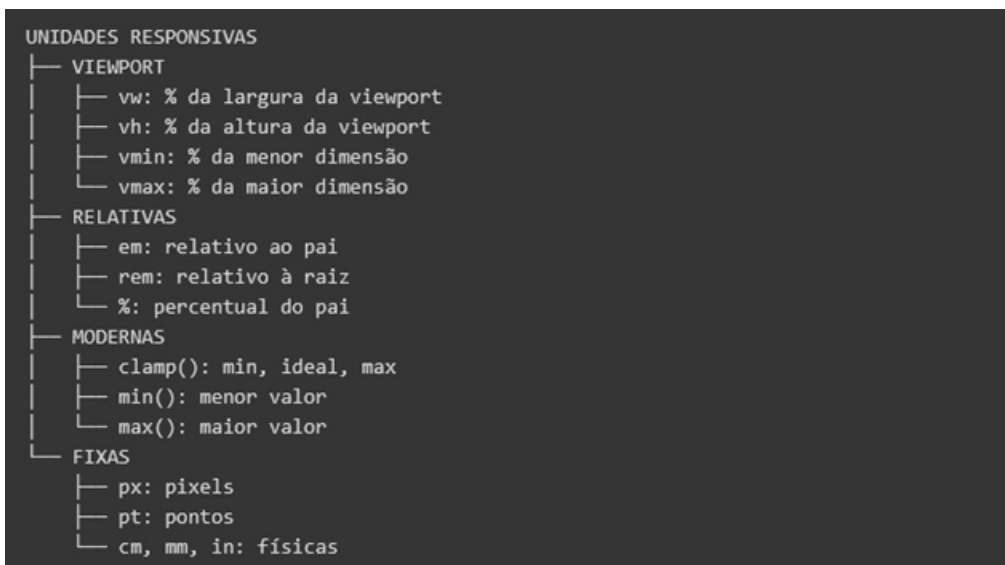
.flex-item {
  flex: 1 1 calc(33.333% - 2rem);
  min-width: 250px;
}
```

9.7. ESQUEMAS VISUAIS CONSOLIDADOS

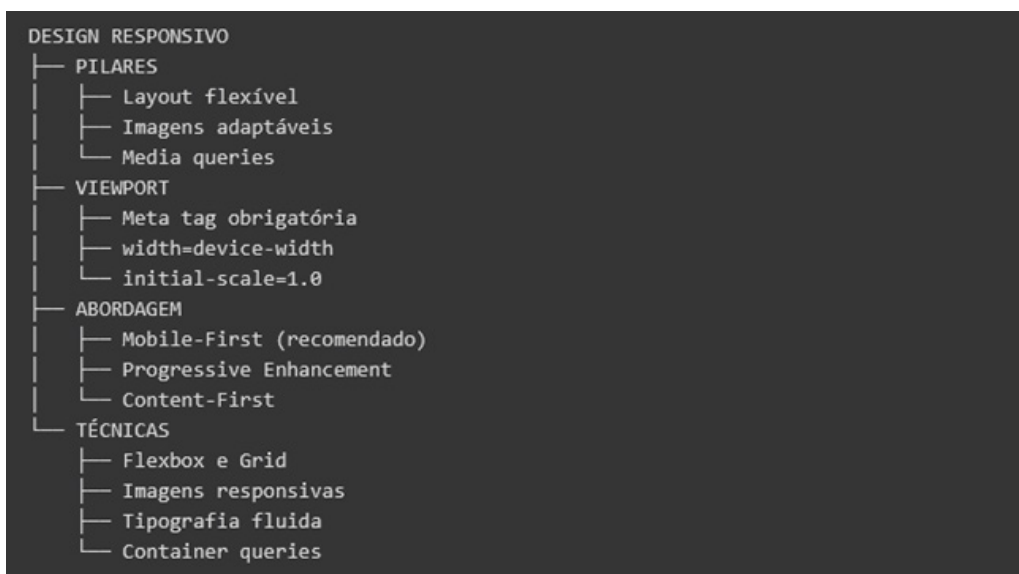
9.7.1. MEDIA QUERIES

```
MEDIA QUERIES
├── SINTAXE
│   ├── @media media-type and (feature) { }
│   ├── Operadores: and, or (,), not, only
│   └── Features: width, height, orientation
├── BREAKPOINTS
│   ├── Mobile: 0-767px
│   ├── Tablet: 768-1023px
│   ├── Desktop: 1024-1199px
│   └── Large: 1200px+
├── ABORDAGENS
│   ├── Mobile-First: min-width (moderno)
│   └── Desktop-First: max-width (antigo)
└── TIPOS
    ├── screen (padrão)
    ├── print (impressão)
    └── all (todos)
```

9.7.2. UNIDADES RESPONSIVAS



9.7.3. DESIGN RESPONSIVO



10. SELETORES AVANÇADOS

10.1. COMBINADORES

Combinadores CSS são operadores que determinam a **relação hierárquica ou posicional entre elementos HTML**, possibilitando a seleção de elementos com base em sua localização relativa na estrutura do documento.

Sintaxe: **pai descendente**

A tabela abaixo mostra os diferentes **combinadores em CSS**:

Nome	Símbolo	Exemplo	Resultado do Exemplo
Combinador de filho	>	div > p	Seleciona todo elemento <p> que é filho direto de um elemento <div>
Combinador de descendente	(espaço)	div p	Seleciona todos os elementos <p> dentro de elementos <div>
Separador de namespace		Ns h2	Seleciona todos os elementos <h2> no namespace ns
Combinador de próximo-irmão	+	div + p	Seleciona o primeiro elemento <p> que vem imediatamente após um elemento <div>
Lista de seletores	,	div, p	Seleciona todos os elementos <div> e todos os elementos <p>
Combinador de irmãos gerais	~	p ~ ul	Seleciona todos os elementos que são precedidos por um elemento <p>

10.1.1. EXEMPLOS PRÁTICOS DE COMBINADORES

```
.navbar > .menu > li {
  position: relative;
  display: inline-block;
}

.navbar > .menu > li > .dropdown {
  position: absolute;
  top: 100%;
  left: 0;
  display: none;
  background: white;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.navbar > .menu > li:hover > .dropdown {
  display: block;
}

.navbar .dropdown > li {
  display: block;
  width: 200px;
}

.navbar .dropdown > li > a {
  padding: 10px 15px;
  border-bottom: 1px solid #eee;
}

.navbar .dropdown > li:last-child > a {
  border-bottom: none;
}
```

10.2. PSEUDO-CLASSES

Uma pseudo-classe CSS é uma palavra-chave que pode ser adicionada a um seletor, para **definir um estilo para um estado especial de um elemento**.

Uso comum para pseudo-classes:

- Estilo de um elemento quando um usuário move o mouse sobre ele;
- Links visitados e não visitados de forma diferente;
- Estilo de um elemento quando ele se concentra;
- Elementos de forma válidos/inválidos/necessários/opcionais;
- Estilo de um elemento que é o primeiro filho de seu pai.

As pseudo-classes **iniciam com símbolo: (dois pontos) e são geralmente aplicadas a algum seletor CSS mais simples**, como um seletor de classe ou um seletor de tag, permitindo formatações mais específicas conforme o estado do elemento.

A tabela abaixo lista todas as **palavras-chave de pseudoclassem em CSS**:

Pseudo-classe	Exemplo	Descrição traduzida
:active	a:active	Seleciona o link ativo
:any-link	a:any-link, area:any-link	Seleciona qualquer <a> ou <area> com atributo href
:autofill	input:autofill	Seleciona qualquer <input> preenchido automaticamente pelo navegador
:checked	input:checked, option:checked	Seleciona qualquer <input> ou <option> marcado
:default	input:default, button:default, option:default	Seleciona elementos de formulário que são padrão em um grupo
:defined	:defined	Seleciona qualquer elemento que tenha sido definido (padrão ou personalizado)
:dir()	:dir(ltr),:dir rtl)	Seleciona elementos com a direção de texto especificada
:disabled	input:disabled, option:disabled	Seleciona elementos desabilitados (principalmente formulários)
:empty	div:empty	Seleciona elementos sem filhos (incluindo nós de texto)
:enabled	input:enabled	Seleciona elementos habilitados (principalmente formulários)
:first	@page:first	Representa a primeira página de um documento impresso
:first-child	p:first-child, li:first-child	Seleciona o primeiro filho do pai
:first-of-type	p:first-of-type, li:first-of-type	Seleciona o primeiro elemento de seu tipo entre os irmãos
:focus	input:focus, select:focus	Seleciona o elemento que recebe foco

Pseudo-classe	Exemplo	Descrição traduzida
:focus-visible	button:focus-visible	Seleciona elementos focados via teclado (não mouse)
:focus-within	form:focus-within, label:focus-within	Seleciona elementos se eles ou seus descendentes estiverem em foco
:fullscreen	:fullscreen	Seleciona qualquer elemento em modo tela cheia
:has()	h2:has(+p)	Seleciona <h2> seguido imediatamente de <p>
:hover	a:hover, p:hover	Seleciona elementos quando o mouse passa sobre eles
:in-range	input:in-range	Seleciona <input> com valor dentro do limite especificado
:indeterminate	input:indeterminate	Seleciona elementos de formulário em estado indeterminado
:invalid	input:invalid, fieldset:invalid	Seleciona elementos de formulário inválidos
:is()	:is(ul, ol)	Seleciona todos os e
:lang()	p:lang(it)	Seleciona <p> com atributo lang="it"
:last-child	li:last-child	Seleciona o último filho de seu pai
:last-of-type	p:last-of-type	Seleciona o último <p> filho de seu pai
:left	@page:left	Representa páginas à esquerda em documentos impressos
:link	a:link	Seleciona links não visitados
:modal	:modal	Seleciona o elemento que está em estado modal
:not()	:not(p)	Seleciona elementos que não sejam <p>
:nth-child()	p:nth-child(2)	Seleciona o <p> que é o segundo filho do pai
:nth-last-child()	p:nth-last-child(2)	Seleciona o <p> que é o segundo filho do pai, contando de trás para frente
:nth-last-of-type()	p:nth-last-of-type(2)	Seleciona o segundo <p> filho do pai, contado do final
:nth-of-type()	p:nth-of-type(2)	Seleciona o segundo <p> filho do pai
:only-child	p:only-child	Seleciona <p> que é filho único
:only-of-type	p:only-of-type	Seleciona <p> que é o único <p> filho do pai
:optional	input:optional, select:optional, textarea:optional	Seleciona elementos sem atributo required
:out-of-range	input:out-of-range	Seleciona <input> com valor fora do limite
:placeholder-shown	input:placeholder-shown, textarea:placeholder-shown	Seleciona elementos exibindo texto placeholder

Pseudo-classe	Exemplo	Descrição traduzida
:popover-open	:popover-open	Seleciona elementos em estado de popover ativo
:read-only	input:read-only	Seleciona <input> com atributo readonly
:read-write	input:read-write	Seleciona <input> editável
:required	input:required	Seleciona <input> com atributo required
:right	@page:right	Representa páginas à direita em documentos impressos
:root	:root	Seleciona o elemento raiz do documento
:scope	:scope	Seleciona elementos que servem de referência para seletores
:state()	:state()	Seleciona elementos personalizados com estado definido
:target	:target	Seleciona o alvo ativo atual
:user-invalid	:user-invalid	Seleciona formulário inválido após interação do usuário
:user-valid	:user-valid	Seleciona formulário válido após interação do usuário
:valid	input:valid	Seleciona <input> com valor válido
:visited	a:visited, area:visited	Seleciona links visitados
:where()	:where(ol, ul)	Seleciona todos e

10.2.1. EXEMPLOS PRÁTICOS DE PSEUDO-CLASSES

Nth-child:

```
/* Tabela zebra */
tr:nth-child(even) {
    background: #f8f9fa;
}

/* Primeiros 3 itens em destaque */
.item:nth-child(-n+3) {
    font-size: 1.1em;
    color: #007bff;
}

/* Cada terceiro item */
.card:nth-child(3n) {
    margin-right: 0;
}

/* A partir do segundo item */
.menu-item:nth-child(n+2) {
    border-top: 1px solid #eee;
}
```

Empty:

```
:empty {
  /* Elementos vazios (sem conteúdo ou filhos) */
}

/* Exemplos */
p:empty {
  display: none;
}

.alert:empty {
  padding: 0;
  border: none;
}

.container:empty::before {
  content: "Nenhum conteúdo disponível";
  color: #999;
  font-style: italic;
}
```

10.3. PSEUDO-ELEMENTOS

Pseudo-elementos permitem **estilizar partes específicas de um elemento ou inserir conteúdo virtual no documento**.

Sintaxe Moderna::pseudo-elemento (CSS3)

Sintaxe Antiga::pseudo-elemento (CSS2 – ainda funciona)

Pode ser usado para:

- Estilo da primeira letra ou linha de um elemento;
- Inserir conteúdo antes ou depois de um elemento;
- Estilo dos marcadores de itens da lista;
- Estilo da caixa de visualização atrás de uma caixa de diálogo.

A tabela abaixo mostra os diferentes **pseudo-elementos em CSS**:

Pseudo-element	Example	Descrição em português
::after	p::afterdiv::after	Inserir algo após o conteúdo do elemento especificado
::backdrop	dialog::backdrop	Estiliza a área atrás de uma caixa de diálogo ou elemento popover
::before	p::beforediv::before	Inserir algo antes do conteúdo do elemento especificado
::file-selector-button	::file-selector-button	Seleciona qualquer botão do tipo <input type="file">
::first-letter	p::first-letter	Seleciona a primeira letra de cada elemento <p>
::first-line	p::first-line	Seleciona a primeira linha de cada elemento <p>

::grammar-error	::grammar-error	Estiliza um texto que o navegador marcou como gramaticalmente incorreto
::highlight()	::highlight(custom-name)	Seleciona um destaque personalizado
::marker	::marker	Seleciona os marcadores de itens de lista
::placeholder	input::placeholder, textarea::placeholder	Estiliza o texto do placeholder de elementos <input> ou <textarea>
::selection	::selection	Estiliza o texto selecionado pelo usuário
::spelling-error	::spelling-error	Estiliza um texto que o navegador marcou como escrito incorretamente
::view-transition	::view-transition	Representa a raiz da sobreposição de transições de visualização, que contém todas as transições na página
::view-transition-group	::view-transition-group	Representa um único grupo de captura de transição de visualização
::view-transition-image-pair	::view-transition-image-pair	Representa um contêiner para os estados “antigo” e “novo” de uma transição de visualização – antes e depois da transição
::view-transition-new	::view-transition-new	Representa o estado “novo” de uma transição de visualização
::view-transition-old	::view-transition-old	Representa o estado “antigo” de uma transição de visualização

10.3.1. EXEMPLOS PRÁTICOS DE PSEUDO-ELEMENTOS

Before e After:

```
/* Ícones com ::before */
.icone-estrela::before {
  content: "★";
  color: gold;
  margin-right: 5px;
}

.icone-usuario::before {
  content: "👤";
  margin-right: 8px;
}

/* Aspas decorativas */
.citacao::before {
  content: "“";
  font-size: 2em;
  color: #ccc;
  line-height: 0;
  margin-right: 5px;
}
```

```
.citacao::after {
  content: "";
  font-size: 2em;
  color: #ccc;
  line-height: 0;
  margin-left: 5px;
}

/* Links externos */
a[href^="http"]:not([href="meusite.com"]>::after {
  content: " />";
  color: #007bff;
  font-size: 0.8em;
}

/* Clearfix clássico */
.clearfix::after {
  content: "";
  display: table;
  clear: both;
}
```

First-letter:

```
::first-letter {
  /* Primeira letra do elemento */
}

/* Drop cap clássico */
p::first-letter {
  font-size: 3em;
  font-weight: bold;
  float: left;
  margin: 0.1em 0.1em 0 0;
  line-height: 0.8;
  color: #007bff;
}

.artigo::first-letter {
  font-family: "Times New Roman", serif;
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}
```

First-line:

```

::first-line {
    /* Primeira linha do elemento */
}

/* Primeira linha em destaque */
p::first-line {
    font-weight: bold;
    color: #333;
    font-variant: small-caps;
}

.intro::first-line {
    font-size: 1.1em;
    letter-spacing: 1px;
}

```

10.4. SELETORES DE ATRIBUTO

Seletores de atributo são padrões CSS que permitem **selecionar elementos HTML baseados na presença ou valor de seus atributos HTML**.

Sintaxe: **[atributo]**

Seletor	Exemplo	Descrição do Exemplo
[attribute]	[target]	Seleciona todos os elementos com um atributo target
[attribute=value]	[target="_blank"]	Seleciona todos os elementos com target="_blank"
[attribute~=value]	[title~="flower"]	Seleciona todos os elementos com um atributo title que contenha uma lista de palavras separadas por espaço, sendo uma delas "flower"
[attribute =value]	[lang="en"]	Seleciona todos os elementos com um atributo lang cujo valor comece com "en" .
[attribute^=value]	a[href^="https"]	Seleciona todos os elementos <a> com atributo href cujo valor comece com "https"
[attribute\$=value]	a[href\$=".pdf"]	Seleciona todos os elementos <a> com atributo href cujo valor termine em ".pdf"
[attribute*=value]	a[href*="w3schools"]	Seleciona todos os elementos <a> com atributo href que contenha a substring "concurso"

10.4.1. EXEMPLOS PRÁTICOS DE SELETORES DE ATRIBUTOS

```

/* Links internos */
a[href^="/"], a[href^="#"] {
    color: #007bff;
}

```

```
/* Links externos */
a[href^="http"]:not([href*="meusite.com"]) {
    color: #28a745;
}

a[href^="http"]:not([href*="meusite.com"]):after {
    content: " ↗";
    font-size: 0.8em;
}

/* Links para email */
a[href^="mailto:"] {
    color: #6f42c1;
}

a[href^="mailto:"]::before {
    content: "✉ ";
}

/* Links para telefone */
a[href^="tel:"] {
    color: #fd7e14;
}

a[href^="tel:"]::before {
    content: "☎ ";
}
```

10.5. ESPECIFICIDADE AVANÇADA

A **especificidade avançada** no CSS envolve um conjunto de técnicas para **definir qual regra de estilo prevalece quando múltiplas regras são aplicadas a um mesmo elemento**, especialmente com o uso de seletores complexos.

10.5.1. CÁLCULO DA ESPECIFICIDADE

A especificidade é calculada como uma pontuação, que pode ser representada por uma **tríade de três componentes, na notação (A, B, C)**. A regra com a maior pontuação de especificidade é a que será aplicada.

- **Componente A (IDs):** Conta a quantidade de seletores de ID (#meu-id) na sua regra. Cada ID adiciona **1** a este componente. IDs têm o maior peso na especificidade.
- **Componente B (Classes, Atributos e Pseudoclasses):** Conta o número de seletores de classe (.minha-classe), seletores de atributo ([href]) e pseudoclasses (:hover,:focus,:active). Cada um deles adiciona **1** a este componente.
- **Componente C (Elementos e Pseudoelementos):** Conta o número de seletores de elemento (p, h1, div) e pseudoelementos (::before,::after). Cada um deles adiciona **1** a este componente.

Hierarquia de Especificidade:

- !important: ∞ (sempre vence)
- Inline styles: 1000 pontos
- IDs: 100 pontos
- Classes/Atributos: 10 pontos
- Elementos: 1 ponto
- Universal (*): 0 pontos

Exemplo de Cálculo:

```
/* 1 ponto */
p { color: black; }

/* 1 ponto */
div { color: blue; }

/* 10 pontos */
.classe { color: red; }

/* 10 pontos */
[type="text"] { color: green; }

/* 10 pontos */
:hover { color: yellow; }

/* 11 pontos (10 + 1) */
p.classe { color: purple; }

/* 11 pontos (10 + 1) */
div[type="text"] { color: orange; }

/* 12 pontos (10 + 1 + 1) */
div.classe p { color: pink; }

/* 20 pontos (10 + 10) */
.classe1.classe2 { color: brown; }

/* 21 pontos (10 + 10 + 1) */
.classe1.classe2 p { color: gray; }

/* 100 pontos */
#id { color: navy; }

/* 101 pontos (100 + 1) */
#id p { color: teal; }

/* 110 pontos (100 + 10) */
#id .classe { color: lime; }

/* 111 pontos (100 + 10 + 1) */
#id .classe p { color: cyan; }
```



```
/* 200 pontos (100 + 100) */
#id1 #id2 { color: magenta; }

/* 1000 pontos */
<p style="color: gold;">

/* ∞ - SEMPRE VENCE */
p { color: silver !important; }
```

10.5.2. REGRAS DE DESEMPATE

Ordem de Prioridade:

1. **!important (sempre vence)**
2. Especificidade (maior pontuação vence)
3. Ordem no CSS (último declarado vence em empate)
4. Herança (propriedades herdadas têm especificidade 0)

Exemplo de Desempate:

```
/* Especificidade: 11 */
p.texto { color: red; }

/* Especificidade: 11 - EMPATE */
.texto p { color: blue; }

/* O último declarado vence: blue */
```

10.6. ESQUEMAS VISUAIS CONSOLIDADOS

10.6.1. COMBINADORES

```
COMBINADORES CSS
├── DESCENDENTE (espaço)
│   ├── pai descendente
│   ├── Seleciona: qualquer descendente
│   └── Exemplo: .menu a
├── FILHO DIRETO (>)
│   ├── pai > filho
│   ├── Seleciona: apenas filhos diretos
│   └── Exemplo: .menu > li
├── IRMÃO ADJACENTE (+)
│   ├── elemento + irmão
│   ├── Seleciona: próximo irmão imediato
│   └── Exemplo: h2 + p
└── IRMÃOS GERAIS (~)
    ├── elemento ~ irmãos
    ├── Seleciona: todos irmãos seguintes
    └── Exemplo: h2 ~ p
```

10.6.2. PSEUDO-CLASSES

```
PSEUDO-CLASSES CSS
├── ESTRUTURAIS
│   ├── :first-child, :last-child
│   ├── :nth-child(n), :nth-last-child(n)
│   ├── :first-of-type, :last-of-type
│   └── :nth-of-type(n)
├── ESTADO
│   ├── :hover, :focus, :active
│   ├── :visited, :link
│   └── :checked, :disabled, :enabled
├── VALIDAÇÃO
│   ├── :valid, :invalid
│   └── :required, :optional
└── ESPECIAIS
    ├── :not(seletor)
    ├── :empty
    └── :only-child
```

10.6.3. SELETORES DE ATRIBUTOS

```
SELETORES DE ATRIBUTO
├── PRESENÇA
│   └── [atributo] - possui o atributo
├── VALOR EXATO
│   └── [atributo="valor"] - valor exato
├── INICIA COM
│   └── [atributo^="início"] - valor inicia com
├── TERMINA COM
│   └── [atributo$="final"] - valor termina com
├── CONTÉM SUBSTRING
│   └── [atributo*="sub"] - contém substring
├── CONTÉM PALAVRA
│   └── [atributo~="palavra"] - palavra separada
└── IDIOMA
    └── [atributo|= "valor"] - valor ou "valor-"
```

10.6.4. ESPECIFICIDADE

```
ESPECIFICIDADE CSS
├── PONTUAÇÃO
│   ├── !important: ∞ (sempre vence)
│   ├── Inline: 1000 pontos
│   ├── IDs: 100 pontos
│   ├── Classes/Atributos: 10 pontos
│   ├── Elementos: 1 ponto
│   └── Universal: 0 pontos
├── DESEMPATE
│   ├── 1º: !important
│   ├── 2º: Maior especificidade
│   ├── 3º: Ordem no CSS
│   └── 4º: Herança (0 pontos)
└── CÁLCULO
    ├── Some todos os pontos
    ├── Compare totais
    └── Maior vence
```

RESUMO

FUNDAMENTOS DO CSS

Definições Essenciais

- **CSS** = Linguagem de estilo para apresentação visual
- **Cascata** = Mecanismo de prioridade entre regras conflitantes
- **Separação** = HTML (estrutura) + CSS (apresentação)

Métodos de Aplicação

1. Inline – style="..." (maior prioridade)
2. Interno – <style> no <head>
3. Externo – arquivo.css + <link> (recomendado)

```
seletor {
    propriedade: valor;
}
```

Ordem de Prioridade

1. !important
2. Inline styles
3. IDs (#)
4. Classes (.), atributos, pseudo-classes
5. Elementos, pseudo-elementos
6. Padrão do navegador

SELETORES BÁSICOS

5 Tipos de Seletores Básicos

1. Elemento – p (especificidade: 1)
2. Classe – .classe (especificidade: 10)
3. ID – #id (especificidade: 100)
4. Universal – * (especificidade: 0)
5. Grupo – h1, h2, h3 (especificidade: individual)

Regras de Especificidade

- ID (100) > Classe (10) > Elemento (1) > Universal (0)
- Desempate: Último declarado vence
- Quebra: !important força aplicação

Sintaxe Essencial

- Classe: ponto antes do nome (.classe)
- ID: hashtag antes do nome (#id)
- Grupo: vírgula entre seletores (h1, h2)

Regras Importantes

- ID deve ser único na página
- Elemento pode ter múltiplas classes
- Seletor universal afeta todos

BOX MODEL**Box Model = 4 Componentes**

- 1.Content – Área do conteúdo (width/height)
- 2.Padding – Espaço interno (transparente)
- 3.Border – Borda visível (width, style, color)
- 4.Margin – Espaço externo (transparente)

Fórmulas de Cálculo

- Content-box: Dimensão total = width + padding + border
- Border-box: Dimensão total = width (fixo)

Propriedades Abreviadas

- 1 valor: Todos os lados
- 2 valores: Vertical, horizontal
- 3 valores: Top, horizontal, bottom
- 4 valores: Top, right, bottom, left (TRouBLe)

Conceitos Importantes

- Box-sizing: border-box facilita cálculos
- Margin: 0 auto centraliza horizontalmente
- Colapso de margens apenas vertical
- Border-radius

CORES E BACKGROUNDS**6 Sistemas de Cores**

- 1.Nomes – red (147 cores)
- 2.Hex – #FF0000 (16M+ cores)
- 3.RGB – rgb(255,0,0) (valores 0-255)
- 4.RGBA – rgba(255,0,0,0.5) (com transparência)
- 5.HSL – hsl(0,100%,50%) (intuitivo)
- 6.HSLA – hsla(0,100%,50%,0.5) (HSL + transparência)

Background Properties

- background-color – cor sólida
- background-image – imagem de fundo
- background-repeat – repetição (repeat, no-repeat, repeat-x/y)
- background-position – posicionamento
- background-size – tamanho (cover, contain, auto)
- background-attachment – anexação (scroll, fixed)
- background – propriedade abreviada

Conceitos Importantes

- Cover vs Contain – cobre tudo vs cabe inteiro
- Fixed attachment – efeito parallax
- RGBA vs Opacity – transparência seletiva vs geral

TIPOGRAFIA E TEXTO**Propriedades de Texto Essenciais**

- color – cor do texto (herdada)
- text-align – alinhamento (left, center, right, justify)
- text-decoration – linhas decorativas (none, underline, line-through)
- text-transform – capitalização (uppercase, lowercase, capitalize)
- text-indent – recuo da primeira linha
- letter-spacing – espaçamento entre caracteres
- word-spacing – espaçamento entre palavras
- line-height – altura da linha (use sem unidade)

Propriedades de Fonte Essenciais

- font-family – família de fontes (com fallbacks)
- font-size – tamanho (px, em, rem, %)
- font-weight – peso (normal/400, bold/700, 100-900)
- font-style – estilo (normal, italic, oblique)
- font – propriedade abreviada

CSS3 Moderno

- text-shadow – sombra no texto
- @font-face – fontes customizadas
- Google Fonts – fontes web
- em vs rem – unidades relativas
- Viewport units – tipografia responsiva

Conceitos Importantes

- Herança tipográfica – color e line-height são herdadas
- Fallbacks de fonte – sempre incluir família genérica
- Line-height sem unidade – melhor para herança
- Hierarquia tipográfica

DISPLAY E POSICIONAMENTO**Display Essencial**

- block – linha inteira, aceita width/height
- inline – fluxo horizontal, não aceita width/height
- inline-block – híbrido (fluxo horizontal + dimensões)
- none – remove completamente do layout

Position Fundamental

- static – fluxo normal (padrão)
- relative – relativo à posição original
- absolute – relativo ao ancestral posicionado
- fixed – relativo à viewport (não rola)
- sticky – híbrido relative/fixed (CSS3)

Z-index e Empilhamento

- Apenas elementos posicionados (position ≠ static)
- Maior valor = mais acima
- Contexto de empilhamento cria “camadas” locais

Float e Clear

- float – remove do fluxo, flutua para um lado
- clear – impede elementos ao lado de floats
- clearfix – solução para colapso de container

Conceitos Importantes

- Fluxo normal vs removido do fluxo.
- Viewport vs container posicionado.
- Ocupa espaço vs não ocupa espaço.
- Contexto de empilhamento.

FLEXBOX E GRID**Flexbox (Unidimensional)**

- display: flex – ativa flexbox
- justify-content – alinhamento principal
- align-items – alinhamento cruzado

- flex: 1 – cresce para ocupar espaço
- Uso: componentes, alinhamento, navegação

Grid (Bidimensional)

- display: grid – ativa grid
- grid-template-columns – define colunas
- grid-template-areas – áreas nomeadas
- gap – espaçamento entre itens
- Uso: layouts de página, grids complexos

Quando Usar

- Flexbox: Uma direção, componentes, alinhamento
- Grid: Duas direções, layouts, posicionamento preciso
- Combinados: Grid para estrutura, Flexbox para componentes

Conceitos Importantes

- fr = fração do espaço disponível (Grid)
- repeat() = repetição de padrões (Grid)
- auto-fit/auto-fill = responsividade automática (Grid)
- justify vs align

RESPONSIVIDADE

Design Responsivo = 3 Pilares

- Layout flexível – Grid, Flexbox, unidades relativas
- Imagens adaptáveis – max-width: 100%, object-fit
- Media queries – adaptação por dispositivo

Media Queries Essenciais

- Sintaxe: @media (feature) { }
- Min-width: mobile-first (recomendado)
- Max-width: desktop-first (antigo)
- Orientation: portrait vs landscape

Breakpoints Padrão

- Mobile: 0-767px
- Tablet: 768-1023px
- Desktop: 1024-1199px
- Large: 1200px+

Unidades Responsivas

- vw/vh – percentual da viewport
- rem – relativo à raiz

- % – percentual do pai
- clamp() – min, ideal, max

Mobile-First Approach

- Base: CSS para mobile
- Expansão: min-width para telas maiores
- Vantagens: performance, priorização, modernidade

Viewport Meta Tag:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

SELETORES AVANÇADOS

Combinadores (Relacionamento)

- (espaço) – qualquer descendente
- > – filho direto
- + – irmão adjacente (próximo)
- ~ – irmãos gerais (todos seguintes)

Pseudo-classes

- Estruturais::first-child, :nth-child(n), :last-child
- Estado::hover, :focus, :active, :checked
- Validação::valid, :invalid, :required
- Especiais::not(), :empty, :only-child

Pseudo-elementos

- ::before, ::after – conteúdo virtual (requer content)
- ::first-letter – primeira letra
- ::first-line – primeira linha
- ::placeholder – texto placeholder
- ::selection – texto selecionado

Seletores de Atributo

- [atributo] – possui o atributo
- [atributo="valor"] – valor exato
- [atributo^="início"] – inicia com
- [atributo\$="final"] – termina com
- [atributo="substring"]* – contém substring
- [atributo~="palavra"] – contém palavra separada

Especificidade

- !important: ∞ (sempre vence)
- Inline: 1000 pontos
- IDs: 100 pontos
- Classes/Atributos: 10 pontos
- Elementos: 1 ponto

Conceitos Importantes

- nth-child(2n+1) = ímpares, nth-child(2n) = pares
- :first-child ≠ :first-of-type (diferença crucial)
- ::before/::after obrigatoriamente precisam de content
- Especificidade: soma de pontos, maior vence.

Para treinar CSS brincando:**Flexbox Defense**

No estilo Tower Defense, você deve impedir que seus inimigos atravessem suas defesas. Porém aqui, você usa as propriedades do CSS Flexbox para posicionar suas torres de defesa.

<http://www.flexboxdefense.com/>

Flexbox Froggy

Aqui temos outro jogo que se propõe a ensinar as propriedades do flexbox, onde seu objetivo é posicionar o sapo para cima da vitória-régia.

<https://flexboxfroggy.com/#pt-br>

Grid Garden

Neste jogo, você deve regar suas plantações de cenoura e envenenar as ervas daninhas. Como o nome já sugere, a ideia aqui é aprender sobre GRID Layout.

<https://cssgridgarden.com/>

QUESTÕES DE CONCURSO

001. (INSTITUTO AOCP/IF-MS/PROFESSOR EBT/INFORMÁTICA/2025) O IFMS está em processo de modernização de seu portal institucional, com foco na criação de uma interface mais interativa e responsiva para os alunos e professores. Como parte desse projeto, o time de Desenvolvimento Web foi responsável por melhorar o design da página de cursos, utilizando CSS3 para criar efeitos dinâmicos e melhorar a experiência do usuário. Durante o desenvolvimento, o time utilizou pseudoclasses do CSS3 para aplicar estilos em elementos de acordo com a interação do usuário. Um dos requisitos era alterar o estilo de elementos de listas de cursos, que não possuíam subitens, para dar destaque àqueles que são mais simples e diretos.

Com base na situação exposta, assinale a alternativa que apresenta a pseudoclassee do CSS3 usada para estilizar elementos que não possuem filhos, incluindo nós de texto.

- a) :empty
- b) :required
- c) :visited
- d) :focus
- e) :hover

002. (INSTITUTO AOCP/IF-MS/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) As pseudoclasses, no contexto de Criação de Folhas de Estilo em Cascata (CSS – Cascading Style Sheets), são utilizadas para aplicar estilos específicos a elementos em determinadas condições ou estados. Elas permitem que a aparência de elementos seja alterada dinamicamente, dependendo de interações do usuário, como passar o mouse sobre o elemento, clicar nele ou quando ele recebe foco. Considerando essas interações, assinale a alternativa que apresenta a pseudoclassee do CSS responsável por aplicar um estilo quando o usuário passa o cursor sobre o elemento.

- a) :disabled
- b) :empty
- c) :required
- d) :hover
- e) :checked

003. (FCC/TRF – 4ª REGIÃO/TÉCNICO JURIDICIÁRIO – ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE: DESENVOLVIMENTO DE SISTEMA DA INFORMAÇÃO/2025) Em um site de uma organização construído com HTML e CSS, deseja-se aplicar um estilo especial a todos os links externos (com target=” blank”) que não pertencem à classe patrocinado. O seletor CSS3 que atende a esse requisito é:

- a) a[target="_blank"]:!(.patrocinado)
- b) a [target="_blank"]:!([class="patrocinado"])
- c) a[link{target="_blank"}]:not(.patrocinado)
- d) a [target="_blank"]:not(.patrocinado)
- e) a [target="_blank"]:not([class*=".patrocinado"])

004. (FCC/SEFAZ-PI/ANALISTA DO TESOIRO ESTADUAL – ÁREA DE CONHECIMENTO: TECNOLOGIA DA INFORMAÇÃO/2025) Considere o comando CSS3:

@media (max-width: 600px) {.container {flex-direction: column;}}

Esse comando:

- a) altera a propriedade flex-direction para column sempre que a altura do.container estiver abaixo de 600px, independentemente da largura.
- b) garante o layout de coluna no.container para qualquer dispositivo móvel, sem levar em conta a largura da tela.
- c) aplica a propriedade flex-direction: column ao.container, fazendo com que seus elementos sejam alinhados verticalmente quando a largura do dispositivo estiver em até 600px.
- d) define a largura do.container como exatamente 600px a partir do momento em que a tela atinge ou fica abaixo desse valor, ignorando qualquer modificação posterior de largura.
- e) altera a direção do flex para column apenas quando a largura do dispositivo exceder 600px, sem afetar resoluções menores.

005. (QUADRIX/CREMESE/ANALISTA DE SISTEMAS/2025) O desenvolvimento de sistemas é uma das principais atividades da área de Tecnologia da Informação e Comunicação (TIC) nas organizações. As metodologias de desenvolvimento compõem o conjunto de conhecimentos essenciais para analistas e programadores. Com base nessas informações, julgue o item a seguir.

“CSS pode ser usado para acessar bancos de dados e para manipular conteúdo dinâmico.”

006. (CONSULPAM/CONAB/ANALISTA – TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código em HTML5+CSS3 construído por um desenvolvedor:

```
<!DOCTYPE html> <html> <head> <style> div.container > p { color: red; }.container p.highlight { color: blue; } p { color: green!important; } </style> </head> <body> <div class="container"> <p class="highlight">Texto de exemplo</p> </div> </body> </html>
```

Diante do exposto, assinale a alternativa que apresenta a cor final que será aplicada ao texto exibido na tela.

- a) Azul.
- b) Verde.
- c) Branco.
- d) Vermelho.

007. (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao estruturar um layout de página usando CSS moderno, você decide utilizar uma abordagem flexível e responsiva. A propriedade CSS a ser utilizada para criar um container que permita o alinhamento flexível de seus itens internos é

- a) 'display: flex;'
- b) 'display: show-flex;'
- c) 'display: flex-wrap;'
- d) 'display: wrap;'

008. (QUADRIX/CRP-SP/ANALISTA DE DESENVOLVIMENTO/2025) Quanto às tecnologias para o desenvolvimento *web*, julgue o item seguinte.

"*Bootstrap* é um *framework* CSS que acelera o desenvolvimento de interfaces responsivas e compatíveis com múltiplos dispositivos, fornecendo componentes prontos, como botões, menus e grades."

009. (CESPE-CEBRASPE/BANRISUL/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO II – DESENVOLVIMENTO DE SOFTWARE/2025) Assinale a opção correspondente ao caractere utilizado em CSS3 como seletor universal. * B: C @ D. E #

- a) *
- b) :
- c) @
- d) .
- e) #

010. (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Em um momento em que os dispositivos de acesso à internet variam em tamanho, torna-se essencial adaptar o layout e o design dos sites para proporcionar uma experiência otimizada para todos os usuários. Para melhorar a responsividade de um site, qual técnica CSS deve ser usada para definir estilos específicos conforme o tamanho da tela do dispositivo?

- a) '@font-face'
- b) '@keyframes'
- c) '@import'
- d) '@media'

011. (FGV/DPE-RO/ANALISTA DE SISTEMAS – CLASSE B/2025) O design responsivo é uma abordagem essencial no desenvolvimento web moderno, garantindo que páginas sejam exibidas de forma adequada em dispositivos com diferentes tamanhos de tela, desde smartphones até desktops. Qual a técnica central utilizada para garantir que uma página web seja exibida adequadamente em dispositivos de diferentes tamanhos de tela?

- a) Definição de tamanhos fixos para elementos.
- b) Implementação de *scripts server-side*.
- c) Uso de tabelas para *layout*.
- d) Aplicação de CSS responsivo.
- e) Utilização de imagens fixas.

012. (UEM/UEM/ANALISTA DE INFORMÁTICA I/2025) Considere o seguinte código:

```
a.botao {  
background-color: orange;  
color: white;  
padding: 10px;
```

Com base no código acima, assinale a alternativa **correta**.

- a) Apenas o link com a classe botao será estilizado pelo seletor a.botao
- b) Todos os links receberão o mesmo estilo, pois links são definidos com a tag <a>
- c) A classe botao só funciona se for aplicada em um botão real (<button>)
- d) O seletor está incorreto e deveria ser a.botao
- e) Para aplicar estilos corretamente, o seletor deveria ser #botao

013. (IDCAP/IPJB/TECNOLOGISTA – TECNOLOGIA DA INFORMAÇÃO APLICADA A CONSERVAÇÃO DA BIODIVERSIDADE/2025) No desenvolvimento web, é comum o uso de fontes customizadas para enriquecer o design de uma página. Qual regra do CSS permite a importação de fontes personalizadas, possibilitando especificar a localização do arquivo de fonte, além de configurar estilos como peso e estilo da fonte, ampliando as opções tipográficas além das fontes padrão dos navegadores?

- a) @font-face
- b) @font-family
- c) @font-link
- d) @font-style
- e) @font-import

014. (FGV/MPU/ANALISTA DO MPU-DESENVOLVIMENTO DE SISTEMAS/2025) (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao ajustar o design de um site, percebe-se a necessidade de aumentar o espaço entre o conteúdo de um elemento e suas bordas. A propriedade CSS a ser modificada para atender essa necessidade é:

- a) 'margin'
- b) 'border'
- c) 'padding'
- d) 'background'

015. (FADESP/UNIFESSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO-ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Considerando o código CSS3:

é correto afirmar que:

- a) *display: flex* define a direção dos *flex* itens que por padrão é *column*.
- b) *flex-wrap: wrap* é o valor que define que não é permitida a quebra de linha.
- c) *display: flex* torna o elemento um *flex* container, automaticamente transformando todos os seus filhos diretos em *flex* itens.
- d) *flex-wrap* é a propriedade que define se os itens devem quebrar ou não a linha; por padrão, há quebra de linha.
- e) *display: flex* torna o elemento flexível, automaticamente transformando o elemento em colunas, para evitar a sobreposição.

016. (FCC/TRT-15ª REGIÃO/TÉCNICO JUDICIÁRIO – ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Uma equipe de desenvolvimento está criando um site responsivo e acessível que utiliza, entre outros recursos, HTML5, CSS3, JSF e jQuery. A meta é garantir que o site seja adaptável a diferentes dispositivos e tamanhos de tela, atenda aos padrões de acessibilidade (WCAG 2.1) e possua menus dinâmicos e conteúdo interativo carregado de forma assíncrona. A estratégia que melhor implementa os requisitos é usar:

- a) jQuery e CSS3 para carregar dinamicamente todo o conteúdo e criar menus interativos, deixando HTML5 apenas para estruturação básica.
- b) JSF para criar os menus dinâmicos e jQuery para ajustar automaticamente o *layout* com base no tamanho da tela.
- c) HTML5 para criar menus interativos, CSS3 para ajustar dinamicamente os tamanhos de tela, e JSF para manipular eventos e carregar conteúdo assíncrono.
- d) JSF para ajustar automaticamente o *layout* responsivo, HTML5 para validação acessível, e CSS3 para estilização básica sem *media queries*.
- e) CSS3 para *media queries* e animações responsivas, HTML5 para estruturar elementos semânticos acessíveis, e jQuery para carregar conteúdo assíncrono e manipular menus dinâmicos.

017. (CESPE-CEBRASPE/EMBRAPA/ANALISTA – ÁREA: GESTÃO DA INFORMAÇÃO – SUBÁREA: ENGENHARIA DE DADOS/2025) A respeito das linguagens Python, JavaScript e CSS, e dos conceitos de orientação a objetos, julgue o item seguinte.

“Na linguagem CSS, a propriedade *padding*, no Box Model, define a distância entre a borda de um elemento e os elementos vizinhos na página.”

018. (IBADE/UNIVESP/DESIGN GRÁFICO E DE INTERFACE/2025) No desenvolvimento web, HTML, CSS e JavaScript desempenham funções essenciais e complementares na construção de interfaces digitais. Considerando suas especificidades, qual das alternativas a seguir descreve corretamente as diferenças entre essas tecnologias?

- a) HTML estrutura semanticamente o conteúdo de uma página web, CSS define sua apresentação estética e JavaScript adiciona interatividade e comportamento dinâmico, proporcionando uma experiência mais rica ao usuário.
- b) HTML é uma linguagem de programação utilizada para criar páginas web, CSS é responsável exclusivamente pela responsividade e JavaScript é uma tecnologia voltada para o design gráfico.
- c) HTML define a interação do usuário com a página, CSS controla a lógica da aplicação e JavaScript estrutura o conteúdo textual e multimídia.
- d) HTML, CSS e JavaScript são linguagens de programação que atuam de maneira idêntica, sendo intercambiáveis no desenvolvimento de aplicações web.
- e) HTML e CSS são ferramentas visuais empregadas apenas para design, enquanto JavaScript é a única tecnologia capaz de estruturar, estilizar e interagir com elementos de uma página web.

019. (IBAM/PRODESAN-SP/ANALISTA DE DESENVOLVIMENTO (MOBILE)/2025) HTML e CSS são tecnologias fundamentais para a construção de páginas web, permitindo a estruturação e estilização de conteúdos de forma eficiente. Acerca de suas características, marque (V), para as afirmativas verdadeiras, e (F), para as falsas.

- () O elemento em HTML é um contêiner genérico que pode ser estilizado ou manipulado com CSS e JavaScript, mas não possui semântica específica.
- () A unidade de medida relativa no CSS é sempre equivalente ao tamanho da fonte padrão do navegador, independentemente do contexto em que é utilizada.
- () A propriedade z-index no CSS determina a sobreposição de elementos na página, sendo que valores negativos são permitidos e podem afetar a ordem de empilhamento.
- () A utilização do atributo alt em imagens no HTML é essencial para acessibilidade, pois fornece uma descrição textual do conteúdo da imagem para tecnologias assistivas.

A sequência está correta em:

- a) V – F – F – F.
- b) V – V – V – V.
- c) F – F – V – V.
- d) V – F – V – V.

020. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) O setor de TI da SEMCONGER está desenvolvendo uma interface web interativa para consulta de relatórios fiscais. Para isso, os programadores precisam garantir responsividade e usabilidade da aplicação, utilizando HTML, CSS e JavaScript. A equipe de desenvolvimento discutiu diferentes abordagens para melhorar o desempenho da interface, levantar requisitos e otimizar a manipulação do DOM; analise-as.

I – O *JavaScript* permite manipular dinamicamente os elementos HTML por meio da API *Document Object Model* (DOM).

II – No CSS, a propriedade *display: inline-block*; pode ser utilizada para criar elementos alinháveis horizontalmente sem ocupar toda a largura disponível.

III – O DHTML é um conjunto de tecnologias que inclui HTML, CSS e *JavaScript* para criar páginas dinâmicas e interativas.

IV – O AJAX permite que páginas web façam requisições assíncronas ao servidor, evitando recarregamentos completos.

V – A tag `<script>` no HTML deve ser sempre posicionada dentro do `<head>` para garantir que os scripts sejam carregados antes da renderização da página.

Está correto o que se afirma apenas em:

- a) I, II e IV.
- b) I, III e IV.
- c) II, III e V.
- d) I, II, III e IV.

021. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) O desenvolvedor da SEMCONGER precisa criar uma página dinâmica utilizando HTML, CSS e *JavaScript* para um sistema interno de gerenciamento de contratos. Ele deseja utilizar CSS para definir o layout responsivo e *JavaScript* para interatividade. A seguir está um trecho do código HTML com CSS embutido:

```
<!DOCTYPE html> <html lang="pt-br"> <head> <meta charset="UTF-8"> <meta
name="viewport" content="width=device-width, initial-scale=1.0"> <title>Contratos</
title> <style>.container { display: _____; justify-content: center; align-items: center;
height: 100vh; } </style> </head> <body> <div class="container"> <h1>Gerenciamento de
Contratos</h1> </div> </body> </html>
```

Assinale a alternativa que preenche corretamente a lacuna no código CSS, para que o div. container centralize o conteúdo tanto na horizontal quanto na vertical.

- a) flex
- b) grid
- c) block
- d) inline-block

022. (AVANÇA SP/UNITAU/PROGRAMADOR PLENO/2025) Em uma aplicação que utiliza CSS-in-JS com styled-components, qual abordagem é mais adequada para implementar temas dinâmicos?

- a) Usar variáveis CSS globais
- b) Implementar ThemeProvider com contexto React
- c) Definir estilos inline

- d) Criar múltiplos arquivos CSS
- e) Aplicar classes CSS diretamente no JSX

023. (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM GESTÃO DE SISTEMAS COMPUTACIONAIS/2024) CSS é chamado de linguagem Cascading Style Sheet e é usado para estilizar elementos escritos em uma linguagem de marcação como HTML. O CSS separa o conteúdo da representação visual do site. Pense na decoração da sua página. Utilizando o CSS, é possível alterar a cor do texto e do fundo, fonte e espaçamento entre parágrafos. Também pode criar tabelas, usar variações de layouts, ajustar imagens para suas respectivas telas. São características da Cascading Style Sheets (CSS):

- a) definir cores, fontes e layout de apresentação do documento.
- b) descrever o estilo de documentos estruturados utilizando Java, Python e C++.
- c) permitir o aumento do código que determina o conteúdo do que vai ser apresentado na página.
- d) possibilitar criar novas folhas de estilo sem permitir a herança de outra folha de estilo já existente.

024. (FGV/TCE-PA/AUDITOR DE CONTROLE EXTERNO – ÁREA DE INFORMÁTICA – WEB DESIGN/2024) No Cascading Style Sheets (CSS), uma área de um documento que emprega *flexbox* é chamada de flex container. Para criar um contêiner flexível, é necessário definir uma determinada propriedade como *flex* ou *inline-flex*. Dessa forma, os elementos filhos do contêiner tornam-se itens do tipo *flex*.

Nesse caso, assinale a referida propriedade do CSS.

- a) *flex-wrap*.
- b) *display*.
- c) *container*.
- d) *flex-flow*.
- e) *flex-basis*.

025. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT – ÁREA 36: INFORMAÇÃO E COMUNICAÇÃO III/2025) Em CSS 3 é possível criar ou complementar conteúdos que irão compor a página do site. Uma forma de complementar um elemento com conteúdos é por meio da utilização de pseudoelementos, como *before* e *after*, na declaração da regra CSS, e da utilização da propriedade *content* em seu conteúdo.

Considerando o pseudoelemento *before*, qual opção apresenta um exemplo de aplicação para elementos *p*, com a sintaxe correta?

- a) *p:before*
- b) *p>before*
- c) *p::before*
- d) *p before*

026. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT – ÁREA 36: INFORMAÇÃO E COMUNICAÇÃO III/2025) A função CSS *linear-gradient* é capaz de criar imagens com gradientes em páginas HTML, devendo-se informar a direção e as cores que farão parte do efeito.

Qual opção apresenta um exemplo de sintaxe correta para sua utilização?

- a) background-color: linear-gradient(to bottom #4377FA #0537B7)
- b) background-color: linear-gradient(to bottom, #4377FA, #0537B7)
- c) background-image: linear-gradient(to bottom #4377FA #0537B7)
- d) background-image: linear-gradient(to bottom, #4377FA, #0537B7)

027. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) Em CSS, a propriedade que é usada para definir a largura de um elemento é denominada

- a) Length.
- b) Size.
- c) Breadth.
- d) Width.
- e) Measure.

028. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) A propriedade CSS que é usada para alterar a cor de fundo de um elemento é denominada:

- a) background-color.
- b) bgcolor.
- c) color.
- d) background-style.
- e) color-background.

029. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) Assinale a opção que corresponde ao seletor CSS que pode ser utilizado para selecionar todos os elementos <p> que são filhos diretos de <div>.

- a) div p
- b) div > p
- c) div + p
- d) div.p
- e) #div p

030. (CESPE-CEBRASPE/TCE-AC/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO – ÁREA: SISTEMAS DE INFORMAÇÃO/2024) Em relação ao desenvolvimento de sistemas *web*, julgue o próximo item.

“O uso extensivo de pseudoelementos no CSS3 pode melhorar a estrutura semântica de um documento HTML.”

GABARITO

1. a
2. d
3. d
4. c
5. E
6. b
7. a
8. C
9. a
10. d
11. d
12. a
13. a
14. c
15. c
16. e
17. E
18. a
19. d
20. d
21. a
22. b
23. a
24. b
25. c
26. d
27. d
28. a
29. b
30. E

GABARITO COMENTADO

001. (INSTITUTO AOCP/IF-MS/PROFESSOR EBTT/INFORMÁTICA/2025) O IFMS está em processo de modernização de seu portal institucional, com foco na criação de uma interface mais interativa e responsiva para os alunos e professores. Como parte desse projeto, o time de Desenvolvimento Web foi responsável por melhorar o design da página de cursos, utilizando CSS3 para criar efeitos dinâmicos e melhorar a experiência do usuário. Durante o desenvolvimento, o time utilizou pseudoclasses do CSS3 para aplicar estilos em elementos de acordo com a interação do usuário. Um dos requisitos era alterar o estilo de elementos de listas de cursos, que não possuíam subitens, para dar destaque àqueles que são mais simples e diretos.

Com base na situação exposta, assinale a alternativa que apresenta a pseudoclasse do CSS3 usada para estilizar elementos que não possuem filhos, incluindo nós de texto.

- a) :empty
- b) :required
- c) :visited
- d) :focus
- e) :hover



a) Certa. A pseudoclasse:empty seleciona elementos que não possuem nenhum conteúdo, incluindo filhos (outros elementos) e nós de texto. Um elemento é considerado “vazio” se não tiver nem mesmo um espaço em branco ou quebras de linha. Essa pseudoclasse é ideal para o caso descrito, pois o requisito é estilizar elementos que não possuem subitens (ou seja, sem filhos, incluindo texto).

b) Errada. A pseudoclasse:required é usada para estilizar elementos de formulário (como <input>, <select> ou <textarea>) que possuem o atributo required, indicando que o preenchimento é obrigatório. Não tem relação com a presença ou ausência de filhos em um elemento, como listas.

c) Errada. A pseudoclasse:visited é aplicada a links (<a>) que já foram visitados pelo usuário, com base no histórico do navegador. Não tem qualquer relação com a estrutura de elementos ou a presença de filhos.

d) Errada. A pseudoclasse:focus estiliza elementos que recebem o foco do usuário, como campos de formulário ou links, quando clicados ou selecionados via teclado. Não está relacionada à estrutura de filhos ou subitens de um elemento.

e) Errada. A pseudoclasse:hover aplica estilos a um elemento quando o cursor do mouse passa sobre ele. É usada para interações visuais baseadas no movimento do mouse, não para verificar a presença ou ausência de filhos.

Letra a.

002. (INSTITUTO AOCP/IF-MS/TÉCNICO DE TECNOLOGIA DA INFORMAÇÃO/2025) As pseudoclasses, no contexto de Criação de Folhas de Estilo em Cascata (CSS – Cascading Style Sheets), são utilizadas para aplicar estilos específicos a elementos em determinadas condições ou estados. Elas permitem que a aparência de elementos seja alterada dinamicamente, dependendo de interações do usuário, como passar o mouse sobre o elemento, clicar nele ou quando ele recebe foco. Considerando essas interações, assinale a alternativa que apresenta a pseudoclasse do CSS responsável por aplicar um estilo quando o usuário passa o cursor sobre o elemento.

- a) :disabled
- b) :empty
- c) :required
- d) :hover
- e) :checked



- a) Errada. Esta pseudoclasse é usada para selecionar elementos de formulário que estão desabilitados, como um botão ou campo de entrada de texto. O estilo é aplicado para indicar que o elemento não pode ser clicado ou editado.
- b) Errada. Esta pseudoclasse seleciona elementos que não possuem filhos (incluindo texto). É útil, por exemplo, para estilizar uma div vazia de uma maneira diferente.
- c) Errada. Esta pseudoclasse é usada para selecionar elementos de formulário que são obrigatórios. Os navegadores geralmente exibem um asterisco ao lado desses campos para indicar que eles devem ser preenchidos.
- d) Certa. Esta é a pseudoclasse que você usa para aplicar estilos quando o usuário passa o mouse sobre um elemento. É muito comum para criar efeitos de “mouse-over” em links, botões e outros elementos interativos.
- e) Errada. Esta pseudoclasse seleciona elementos de formulário que estão em um estado “marcado” ou “selecionado”, como caixas de seleção (checkboxes) e botões de rádio (radio buttons). O estilo é aplicado quando o usuário clica para marcar a opção.

Letra d.

003. (FCC/TRF – 4ª REGIÃO/TÉCNICO JURIDICIÁRIO – ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE: DESENVOLVIMENTO DE SISTEMA DA INFORMAÇÃO/2025) Em um site de uma organização construído com HTML e CSS, deseja-se aplicar um estilo especial a todos os links externos (com target=” blank”) que não pertencem à classe patrocinado. O seletor CSS3 que atende a esse requisito é:

- a) a[target=”_blank”]:!(.patrocinado)
- b) a [target=”_blank”]:!([class=”patrocinado”])

- c) `a[link{target="_blank"}]:not(.patrocinado)`
- d) `a [target="_blank"]:not(.patrocinado)`
- e) `a [target="_blank"]:not([class*=".patrocinado"])`



- a) Errada. `a[target="_blank"]:!(.patrocinado)`: A sintaxe `!` não é usada para negação em CSS. A forma correta é a pseudoclasse `:not()`.
- b) Errada. `a [target="_blank"]:!([class="patrocinado"])`: Novamente, o `!` está sintaticamente incorreto. Além disso, `[class="patrocinado"]` seleciona apenas elementos cuja única classe seja "patrocinado", o que pode não ser o caso (um elemento pode ter várias classes). O seletor de classe `.patrocinado` é a forma padrão para selecionar elementos que tenham essa classe, independentemente de outras.
- c) Errada. `a[link{target="_blank"}]:not(.patrocinado)`: A sintaxe `{}` não é válida para seletores de atributos em CSS. A forma correta é `[]`.
- d) Certa. Este seletor combina três partes para atingir o resultado desejado:
 - `a`: Seleciona todos os elementos de link (`<a>`).
 - `[target="_blank"]`: É um seletor de atributo que filtra os links, selecionando apenas aqueles que possuem o atributo `target` com o valor `"_blank"`.
 - `:not(.patrocinado)`: É a pseudoclasse de negação. Ela exclui da seleção todos os links que pertencem à classe `patrocinado`.
- e) Errada. `a [target="_blank"]:not([class*=".patrocinado"])`: O seletor de atributo `[class*=".patrocinado"]` busca por qualquer classe que contenha a string `".patrocinado"`, o que não é o que se deseja. Além disso, a sintaxe com o ponto `(.)` dentro do valor do atributo está incorreta. A maneira correta de negar uma classe é com o seletor de classe padrão dentro do `:not()`, como na alternativa d.

Letra d.

004. (FCC/SEFAZ-PI/ANALISTA DO TESOIRO ESTADUAL – ÁREA DE CONHECIMENTO: TECNOLOGIA DA INFORMAÇÃO/2025) Considere o comando CSS3:

```
@media (max-width: 600px) { .container { flex-direction: column; } }
```

Esse comando:

- a) altera a propriedade `flex-direction` para `column` sempre que a altura do `.container` estiver abaixo de 600px, independentemente da largura.
- b) garante o layout de coluna no `.container` para qualquer dispositivo móvel, sem levar em conta a largura da tela.
- c) aplica a propriedade `flex-direction: column` ao `.container`, fazendo com que seus elementos sejam alinhados verticalmente quando a largura do dispositivo estiver em até 600px.

- d) define a largura do.container como exatamente 600px a partir do momento em que a tela atinge ou fica abaixo desse valor, ignorando qualquer modificação posterior de largura.
- e) altera a direção do flex para column apenas quando a largura do dispositivo exceder 600px, sem afetar resoluções menores.



- a) Errada. A condição (max-width: 600px) se refere à largura (width), e não à altura (height).
- b) Errada. A regra não se aplica a qualquer dispositivo móvel, mas sim especificamente àqueles com largura de tela de até 600px. Dispositivos móveis com telas maiores (como alguns tablets) não serão afetados por essa regra.
- c) Certa. Descreve com precisão a função do comando. Ele aplica o estilo de layout vertical (flex-direction: column) para o elemento.container quando a largura do dispositivo ou janela do navegador for de até 600px.
- d) Errada. O comando não altera a largura do.container. Ele apenas aplica um estilo de layout diferente, sem definir um tamanho fixo. A largura continua sendo a do dispositivo (viewport).
- e) Errada. A regra max-width se aplica a larguras iguais ou menores que 600px, e não a larguras que excedem esse valor. A regra para exceder seria min-width.

Letra c.

005. (QUADRIX/CREMESE/ANALISTA DE SISTEMAS/2025) O desenvolvimento de sistemas é uma das principais atividades da área de Tecnologia da Informação e Comunicação (TIC) nas organizações. As metodologias de desenvolvimento compõem o conjunto de conhecimentos essenciais para analistas e programadores. Com base nessas informações, julgue o item a seguir.

“CSS pode ser usado para acessar bancos de dados e para manipular conteúdo dinâmico.”



O **CSS (Cascading Style Sheets)** é uma linguagem de folhas de estilo utilizada para descrever a apresentação de um documento escrito em uma linguagem de marcação, como o **HTML**. Sua única função é definir como os elementos de um documento serão exibidos na tela, no papel ou em outras mídias.

Suas principais responsabilidades incluem:

- **Estilização:** Definir cores, fontes, tamanhos, espaçamento e layout dos elementos.
- **Apresentação:** Controlar a forma como o conteúdo é visualizado pelo usuário, como a responsividade em diferentes dispositivos.

Já as tarefas de **acessar bancos de dados** e **manipular conteúdo dinâmico** são funções de linguagens de programação e tecnologias do lado do servidor (back-end) ou do lado do cliente (front-end), como:

- **Linguagens de back-end:** PHP, Python, Java, Node.js.
- **Linguagens de front-end:** JavaScript.

O **JavaScript**, por exemplo, é a linguagem responsável por manipular o **DOM (Document Object Model)**, o que permite criar, alterar e remover elementos HTML de forma dinâmica, além de interagir com APIs para buscar dados de um banco de dados ou de outras fontes. O CSS, por si só, não tem capacidade de processamento ou de interação com dados, ele apenas recebe as instruções de estilo e as aplica.

Errado.

006. (CONSULPAM/CONAB/ANALISTA – TECNOLOGIA DA INFORMAÇÃO/2025) Considere o seguinte trecho de código em *HTML5+CSS3* construído por um desenvolvedor:

```
<!DOCTYPE html> <html> <head> <style> div.container > p { color: red; }.container p.highlight { color: blue; } p { color: green!important; } </style> </head> <body> <div class="container"> <p class="highlight">Texto de exemplo</p> </div> </body> </html>
```

Diante do exposto, assinale a alternativa que apresenta a cor final que será aplicada ao texto exibido na tela.

- a) Azul.
- b) Verde.
- c) Branco.
- d) Vermelho.



- a) Errada. A regra `div.container p.highlight { color: blue; }` tem alta especificidade e seria a vencedora em uma disputa normal. No entanto, `!important` anula sua prioridade.
- b) Certa. A regra `p { color: green!important; }` é a menos específica, mas o modificador `!important` força sua aplicação, ignorando a especificidade das outras regras. Por isso, a cor final é verde.
- c) Errada. Nenhuma das regras de estilo define a cor branca.
- d) Errada. A regra `div.container > p { color: red; }` tem alta especificidade, mas é superada tanto pela regra azul (que vem depois) quanto pelo `!important`.

Letra b.

007. (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao estruturar um layout de página usando CSS moderno, você decide utilizar uma abordagem flexível e responsiva. A propriedade CSS a ser utilizada para criar um container que permita o alinhamento flexível de seus itens internos é

- a) `'display: flex;'`
- b) `'display: show-flex;'`

- c) 'display: flex-wrap;'
- d) 'display: wrap;'



- a) Certa. Ao aplicá-la a um elemento, ele se torna um container flexível (flex container), permitindo que seus filhos diretos (flex items) sejam alinhados e distribuídos de forma dinâmica e responsiva.
- b) Errada. Esta propriedade não existe no CSS padrão.
- c) Errada. A propriedade flex-wrap é utilizada para controlar se os itens flexíveis devem ou não quebrar para a próxima linha quando o espaço no container não for suficiente. Ela não é a propriedade que define o container flexível em si.
- d) Errada. Esta propriedade não existe no CSS padrão.

Letra a.

008. (QUADRIX/CRP-SP/ANALISTA DE DESENVOLVIMENTO/2025) Quanto às tecnologias para o desenvolvimento *web*, julgue o item seguinte.

"*Bootstrap* é um *framework* CSS que acelera o desenvolvimento de interfaces responsivas e compatíveis com múltiplos dispositivos, fornecendo componentes prontos, como botões, menus e grades."



Essas três tecnologias são, de fato, a base do desenvolvimento front-end e possuem papéis bem definidos e complementares:

- **HTML (HyperText Markup Language):** É a linguagem de marcação que define a **estrutura** e o conteúdo da página. Ela cria a espinha dorsal do site, usando elementos como cabeçalhos, parágrafos, imagens e links.
- **CSS (Cascading Style Sheets):** É a linguagem de estilo que cuida da **aparência visual**. Ela é usada para estilizar o HTML, controlando cores, fontes, layouts e espaçamento, transformando a estrutura básica em um design visualmente atraente.
- **JavaScript:** É a linguagem de programação que adiciona **interatividade e dinamismo**. Ele permite criar funcionalidades como animações, validações de formulários, manipulação de conteúdo em tempo real e a comunicação com o servidor, tornando a página mais responsiva e interativa para o usuário.

Certo.

009. (CESPE-CEBRASPE/BANRISUL/TÉCNICO EM TECNOLOGIA DA INFORMAÇÃO II – DESENVOLVIMENTO DE SOFTWARE/2025) Assinale a opção correspondente ao caractere utilizado em CSS3 como seletor universal. * B: C @ D. E #

- a) *
- b) :
- c) @
- d) .
- e) #



- a) Certa. O caractere **asterisco *** é o **seletor universal** em CSS3. Ele seleciona todos os elementos de uma página HTML.
- b) Errada. O caractere **:** é usado para definir pseudoclasses (como `:hover`, `:focus`) e pseudoelementos (como `::before`, `::after`). Eles selecionam elementos com base em seu estado ou em partes específicas de um elemento.
- c) Errada. O caractere **@** inicial para as chamadas regras-at (@-rules) em CSS, como `@media` (para regras de mídia), `@import` (para importar folhas de estilo externas) ou `@font-face` (para definir fontes personalizadas).
- d) Errada. O ponto **.** é usado para selecionar elementos por sua classe. Por exemplo, `.minha-classe` seleciona todos os elementos que têm `class="minha-classe"`.
- e) Errada. O sinal de cerquilha **#** é usado para selecionar elementos por seu ID. Por exemplo, `#meu-id` seleciona o elemento com o atributo `id="meu-id"`.

Letra a.

010. (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Em um momento em que os dispositivos de acesso à internet variam em tamanho, torna-se essencial adaptar o layout e o design dos sites para proporcionar uma experiência otimizada para todos os usuários. Para melhorar a responsividade de um site, qual técnica CSS deve ser usada para definir estilos específicos conforme o tamanho da tela do dispositivo?

- a) '@font-face'
- b) '@keyframes'
- c) '@import'
- d) '@media'



- a) Errada. Esta regra CSS é utilizada para carregar fontes personalizadas em um site, não para a criação de layouts responsivos.
- b) Errada. Esta regra define o comportamento de uma animação CSS, não tendo relação com a adaptação do layout a diferentes tamanhos de tela.

c) Errada. Esta regra é usada para importar folhas de estilo de outros arquivos CSS. Embora possa ser usada para importar estilos responsivos, ela não é a técnica que, por si só, adapta o layout ao tamanho da tela.

d) Certa. Conhecidas como “media queries”, as regras @media permitem aplicar estilos CSS apenas quando certas condições são verdadeiras, como a largura da tela do dispositivo, a altura, a orientação (paisagem ou retrato), etc. É a ferramenta fundamental do CSS para tornar sites responsivos.

Letra d.

011. (FGV/DPE-RO/ANALISTA DE SISTEMAS – CLASSE B/2025) O design responsivo é uma abordagem essencial no desenvolvimento web moderno, garantindo que páginas sejam exibidas de forma adequada em dispositivos com diferentes tamanhos de tela, desde smartphones até desktops. Qual a técnica central utilizada para garantir que uma página web seja exibida adequadamente em dispositivos de diferentes tamanhos de tela?

a) Definição de tamanhos fixos para elementos.

b) Implementação de *scripts server-side*.

c) Uso de tabelas para *layout*.

d) Aplicação de CSS responsivo.

e) Utilização de imagens fixas.



a) Errada. Isso é o oposto do design responsivo. Usar tamanhos fixos (em pixels, por exemplo) faria com que o layout “quebrasse” em telas menores, pois os elementos não se ajustariam.

b) Errada. Scripts do lado do servidor (como PHP ou Node.js) são usados para processar dados e gerar conteúdo, mas não para controlar a forma como a página se adapta visualmente ao tamanho da tela. Isso é uma função do front-end.

c) Errada. Embora tenha sido uma técnica comum no passado, usar tabelas para layout é considerado uma prática desatualizada e não é eficaz para o design responsivo, pois as tabelas não são flexíveis por natureza.

d) Certa. O CSS responsivo engloba um conjunto de práticas e recursos, como:

- **Media Queries:** São regras CSS que aplicam estilos específicos com base nas características do dispositivo, como a largura da tela. Por exemplo, é possível definir que um layout de três colunas em um desktop se torne uma única coluna em um celular.
- **Grid e Flexbox:** São módulos de layout do CSS que permitem a criação de layouts flexíveis e dinâmicos, que se ajustam automaticamente ao espaço disponível.
- **Unidades de Medida Relativas:** O uso de unidades como porcentagem (%), em, rem e vw/vh (viewport width/height) faz com que os elementos se adaptem ao tamanho do contêiner ou da tela, em vez de terem tamanhos fixos em pixels.

e) Errada. Similar aos tamanhos fixos, usar imagens com dimensões absolutas pode causar problemas em telas menores, levando a barras de rolagem horizontais ou a imagens cortadas. O ideal é usar imagens fluidas que se redimensionam com o layout.

Letra d.

012. (UEM/UEM/ANALISTA DE INFORMÁTICA I/2025) Considere o seguinte código:

```
a. botao {
background-color: orange;
color: white;
padding: 10px;
```

Com base no código acima, assinale a alternativa **correta**.

- a) Apenas o link com a classe botao será estilizado pelo seletor a.botao
- b) Todos os links receberão o mesmo estilo, pois links são definidos com a tag <a>
- c) A classe botao só funciona se for aplicada em um botão real (<button>)
- d) O seletor está incorreto e deveria ser a.botao
- e) Para aplicar estilos corretamente, o seletor deveria ser #botao



a) Certa. O seletor a.botao é um seletor de classe combinado com um seletor de tipo. Ele é muito específico e faz a seguinte seleção:

- a: Seleciona todos os elementos do tipo <a> (links).
- .botao: Seleciona todos os elementos que têm a classe botao.

Quando combinados, como em a.botao, o seletor busca por elementos que são links (<a>) E que têm a classe botao. Isso significa que a regra de estilo só será aplicada a links que possuam class="botao".

b) Errada. O seletor não é apenas a. A presença de .botao restringe a aplicação do estilo apenas aos links que pertencem a essa classe.

c) Errada. Classes CSS podem ser aplicadas a qualquer elemento HTML, como div, p, span e, neste caso, a. O nome da classe não a restringe a um tipo de elemento específico, a menos que o seletor faça essa restrição, como é o caso de a.botao.

d) Errada. A sintaxe a.botao não é válida em CSS para o que se propõe. A forma correta de combinar um seletor de tipo e um de classe é tipo.classe, como em a.botao.

e) Errada. O seletor #botao é para selecionar elementos por ID, não por classe. Para funcionar, o HTML precisaria ter um elemento com id="botao". O código CSS fornecido usa um seletor de classe (.botao), que corresponde ao atributo class.

Letra a.

013. (IDCAP/IPJB/TECNOLOGISTA – TECNOLOGIA DA INFORMAÇÃO APLICADA A CONSERVAÇÃO DA BIODIVERSIDADE/2025) No desenvolvimento web, é comum o uso de fontes customizadas para enriquecer o design de uma página. Qual regra do CSS permite a importação de fontes personalizadas, possibilitando especificar a localização do arquivo de fonte, além de configurar estilos como peso e estilo da fonte, ampliando as opções tipográficas além das fontes padrão dos navegadores?

- a) @font-face
- b) @font-family
- c) @font-link
- d) @font-style
- e) @font-import



- a) Certa. @font-face é a regra correta. Ela define uma nova fonte para ser usada em uma página web, permitindo que você especifique o nome da fonte (font-family) e a localização do arquivo da fonte (src), entre outras propriedades como peso (font-weight) e estilo (font-style).
- b) Errada. @font-family não é uma regra CSS, mas sim uma propriedade dentro de um seletor que define qual família de fonte será usada para um elemento, como `p { font-family: 'MinhaFonte', sans-serif; }`.
- c) Errada. @font-link não é uma regra CSS válida para importar fontes.
- d) Errada. @font-style é uma propriedade CSS que define o estilo da fonte (normal, itálico, etc.), mas não é uma regra para importar fontes.
- e) Errada. @font-import não é uma regra CSS válida para importar fontes. Embora a regra @import possa ser usada para importar folhas de estilo inteiras, ela não é a forma correta de importar arquivos de fontes.

Letra a.

014. (FGV/MPU/ANALISTA DO MPU-DESENVOLVIMENTO DE SISTEMAS/2025) (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM DESENVOLVIMENTO FRONT-END/2024) Ao ajustar o design de um site, percebe-se a necessidade de aumentar o espaço entre o conteúdo de um elemento e suas bordas. A propriedade CSS a ser modificada para atender essa necessidade é:

- a) 'margin'
- b) 'border'
- c) 'padding'
- d) 'background'



- a) Errada. A margem cria espaço fora da borda de um elemento, separando-o de outros elementos ao seu redor.
- b) Errada. A borda é a linha que circunda o elemento, ficando entre o padding e a margin. Ela não cria espaço entre o conteúdo e a borda, mas sim define a própria borda.
- c) Certa. A propriedade CSS correta para aumentar o espaço entre o conteúdo de um elemento e suas bordas é a padding.
- d) Errada. A propriedade background define a cor ou imagem de fundo de um elemento. Ela preenche a área do conteúdo e do padding, mas não cria o espaço interno em si.

Letra c.

015. (FADESP/UNIFEISSPA/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO-ÁREA DESENVOLVIMENTO DE SOFTWARE/2025) Considerando o código CSS3:

é correto afirmar que:

- a) *display: flex* define a direção dos *flex* itens que por padrão é *column*.
- b) *flex-wrap: wrap* é o valor que define que não é permitida a quebra de linha.
- c) *display: flex* torna o elemento um *flex* container, automaticamente transformando todos os seus filhos diretos em *flex* itens.
- d) *flex-wrap* é a propriedade que define se os itens devem quebrar ou não a linha; por padrão, há quebra de linha.
- e) *display: flex* torna o elemento flexível, automaticamente transformando o elemento em colunas, para evitar a sobreposição.



- a) Errada. A propriedade *display: flex* torna o elemento um contêiner flexível, mas a direção padrão é a linha (row), não a coluna. A direção é controlada pela propriedade *flex-direction*.
- b) Errada. O valor *wrap* permite a quebra de linha quando os itens não cabem em uma única linha. O valor que impede a quebra de linha é o *nowrap*, que é o padrão.
- c) Certa. Esta é a função principal da propriedade *display: flex*. Quando você a aplica a um elemento, ele se torna um “contêiner flexível” (*flex container*), e todos os seus elementos filhos diretos se tornam “itens flexíveis” (*flex items*), que podem ser alinhados e distribuídos usando as outras propriedades do Flexbox.
- d) Errada. A propriedade *flex-wrap* realmente define a quebra de linha, mas o valor padrão é o *nowrap*, que impede a quebra, forçando todos os itens a permanecerem em uma única linha.
- e) Errada. Conforme mencionado no item “a”, o padrão de direção é a linha, não a coluna. A sobreposição não é o principal problema que o Flexbox resolve, mas sim o alinhamento e a distribuição de espaço entre os itens.

Letra c.

016. (FCC/TRT-15ª REGIÃO/TÉCNICO JUDICIÁRIO – ÁREA APOIO ESPECIALIZADO – ESPECIALIDADE TECNOLOGIA DA INFORMAÇÃO/2025) Uma equipe de desenvolvimento está criando um site responsivo e acessível que utiliza, entre outros recursos, HTML5, CSS3, JSF e jQuery. A meta é garantir que o site seja adaptável a diferentes dispositivos e tamanhos de tela, atenda aos padrões de acessibilidade (WCAG 2.1) e possua menus dinâmicos e conteúdo interativo carregado de forma assíncrona. A estratégia que melhor implementa os requisitos é usar:

- a) jQuery e CSS3 para carregar dinamicamente todo o conteúdo e criar menus interativos, deixando HTML5 apenas para estruturação básica.
- b) JSF para criar os menus dinâmicos e jQuery para ajustar automaticamente o *layout* com base no tamanho da tela.
- c) HTML5 para criar menus interativos, CSS3 para ajustar dinamicamente os tamanhos de tela, e JSF para manipular eventos e carregar conteúdo assíncrono.
- d) JSF para ajustar automaticamente o *layout* responsivo, HTML5 para validação acessível, e CSS3 para estilização básica sem *media queries*.
- e) CSS3 para *media queries* e animações responsivas, HTML5 para estruturar elementos semânticos acessíveis, e jQuery para carregar conteúdo assíncrono e manipular menus dinâmicos.



Vamos analisar por que essa é a melhor estratégia, considerando as responsabilidades de cada tecnologia mencionada:

- **HTML5 (Estrutura e Semântica):** O HTML5 é a base do site. Para um site acessível e semântico, ele deve ser usado para estruturar o conteúdo corretamente, utilizando tags como <header>, <nav>, <main>, <article> e <footer>. Isso não apenas organiza o conteúdo, mas também facilita a navegação por tecnologias assistivas, como leitores de tela.
- **CSS3 (Estilo e Responsividade):** O CSS3 é ideal para a parte visual e responsiva. As *media queries* do CSS3 são a ferramenta principal para criar um layout que se adapta a diferentes tamanhos de tela. Além disso, o CSS3 é usado para estilos, animações e transições, tornando a interface mais atraente.
- **jQuery (Interatividade e Dinamismo):** O jQuery, uma biblioteca JavaScript, é excelente para adicionar interatividade. Ele simplifica a manipulação do DOM (Document Object Model), permitindo a criação de menus dinâmicos e o carregamento de conteúdo assíncrono (via AJAX), o que melhora a performance e a experiência do usuário.
- **JSF (JavaServer Faces):** O JSF é um framework de servidor (back-end) e não uma tecnologia de front-end. Ele lida com a lógica da aplicação, a renderização de componentes na web e a comunicação entre o cliente e o servidor. No contexto do problema, que foca no front-end (layout, menus, conteúdo dinâmico), JSF não é a ferramenta mais adequada para realizar as tarefas citadas.

- a) Errada. Usar jQuery para carregar *todo* o conteúdo dinamicamente pode ser ineficiente e não é a função principal do HTML5. O HTML5 deve ser a base estrutural.
- b) Errada. JSF é um framework de back-end, não é a ferramenta ideal para criar menus dinâmicos ou ajustar o layout no lado do cliente.
- c) Errada. HTML5 não é usado para criar menus interativos (isso é trabalho do JavaScript/ jQuery) nem para ajustar dinamicamente tamanhos de tela (isso é feito pelo CSS3).
- d) Errada. JSF não é a ferramenta para layout responsivo. CSS3 não é apenas para estilização básica; suas media queries são essenciais para a responsividade.
- e) Certa. Essa é a alternativa.

Letra e.

017. (CESPE-CEBRASPE/EMBRAPA/ANALISTA – ÁREA: GESTÃO DA INFORMAÇÃO – SUBÁREA: ENGENHARIA DE DADOS/2025) A respeito das linguagens Python, JavaScript e CSS, e dos conceitos de orientação a objetos, julgue o item seguinte.

“Na linguagem CSS, a propriedade *padding*, no Box Model, define a distância entre a borda de um elemento e os elementos vizinhos na página.”



No CSS Box Model, cada elemento HTML é tratado como uma caixa retangular, composta por quatro camadas:

- **Conteúdo (Content):** A área onde o conteúdo real do elemento (como texto ou imagens) é exibido.
- **Preenchimento (Padding):** A área de espaço transparente que fica **dentro da borda** do elemento e ao redor do conteúdo. O padding adiciona espaço interno e faz parte da “caixa” do elemento. Ele afeta o tamanho total da caixa.
- **Borda (Border):** A linha que envolve o preenchimento e o conteúdo. Ela separa o padding da margem.
- **Margem (Margin):** O espaço transparente **fora da borda** do elemento. A margem é a propriedade que define a distância entre a borda do elemento e os elementos vizinhos.

Portanto, a afirmação está incorreta porque a propriedade padding controla o espaço interno (entre o conteúdo e a borda), enquanto a propriedade margin é a responsável por definir o espaço externo, ou seja, a distância entre a borda de um elemento e os elementos ao seu redor.

Errado.

018. (IBADE/UNIVESP/DESIGN GRÁFICO E DE INTERFACE/2025) No desenvolvimento web, HTML, CSS e JavaScript desempenham funções essenciais e complementares na construção de interfaces digitais. Considerando suas especificidades, qual das alternativas a seguir descreve corretamente as diferenças entre essas tecnologias?

- a) HTML estrutura semanticamente o conteúdo de uma página web, CSS define sua apresentação estética e JavaScript adiciona interatividade e comportamento dinâmico, proporcionando uma experiência mais rica ao usuário.
- b) HTML é uma linguagem de programação utilizada para criar páginas web, CSS é responsável exclusivamente pela responsividade e JavaScript é uma tecnologia voltada para o design gráfico.
- c) HTML define a interação do usuário com a página, CSS controla a lógica da aplicação e JavaScript estrutura o conteúdo textual e multimídia.
- d) HTML, CSS e JavaScript são linguagens de programação que atuam de maneira idêntica, sendo intercambiáveis no desenvolvimento de aplicações web.
- e) HTML e CSS são ferramentas visuais empregadas apenas para design, enquanto JavaScript é a única tecnologia capaz de estruturar, estilizar e interagir com elementos de uma página web.



a) Certa. Essa alternativa descreve de forma precisa e completa as funções de cada uma dessas tecnologias no desenvolvimento web:

- **HTML (HyperText Markup Language):** É a espinha dorsal de qualquer página web. Sua função principal é estruturar o conteúdo. Pense no HTML como o esqueleto do site, onde você define o que é um título, um parágrafo, uma imagem, uma lista, etc.
- **CSS (Cascading Style Sheets):** O CSS é o responsável pelo visual. É ele que define cores, fontes, layouts, espaçamentos e a forma como o conteúdo será apresentado ao usuário. Se o HTML é o esqueleto, o CSS é a pele, a roupa e a maquiagem.
- **JavaScript:** O JavaScript traz a página à vida. Ele é uma linguagem de programação que permite adicionar interatividade, como validação de formulários, animações, manipulação de conteúdo em tempo real e qualquer outro tipo de comportamento dinâmico que o usuário possa interagir. É o sistema nervoso da página.

b) Errada. HTML não é uma linguagem de programação e CSS não é responsável apenas pela responsividade (que é uma parte do design).

c) Errada. As funções estão completamente invertidas. O HTML não define interação, o CSS não controla a lógica e o JavaScript não estrutura o conteúdo.

d) Errada. As três tecnologias não são idênticas nem intercambiáveis. Elas trabalham de forma complementar.

e) Errada. HTML e CSS não são apenas ferramentas visuais e o JavaScript não é a única tecnologia capaz de estruturar, estilizar e interagir, pois cada uma tem seu papel específico.

Letra a.

019. (IBAM/PRODESAN-SP/ANALISTA DE DESENVOLVIMENTO (MOBILE)/2025) HTML e CSS são tecnologias fundamentais para a construção de páginas web, permitindo a estruturação e estilização de conteúdos de forma eficiente. Acerca de suas características, marque (V), para as afirmativas verdadeiras, e (F), para as falsas.

- () O elemento em HTML é um contêiner genérico que pode ser estilizado ou manipulado com CSS e JavaScript, mas não possui semântica específica.
- () A unidade de medida relativa no CSS é sempre equivalente ao tamanho da fonte padrão do navegador, independentemente do contexto em que é utilizada.
- () A propriedade z-index no CSS determina a sobreposição de elementos na página, sendo que valores negativos são permitidos e podem afetar a ordem de empilhamento.
- () A utilização do atributo alt em imagens no HTML é essencial para acessibilidade, pois fornece uma descrição textual do conteúdo da imagem para tecnologias assistivas.

A sequência está correta em:

- a) V - F - F - F.
- b) V - V - V - V.
- c) F - F - V - V.
- d) V - F - V - V.



(V) O elemento em HTML é um contêiner genérico que pode ser estilizado ou manipulado com CSS e JavaScript, mas não possui semântica específica: O elemento `<div>` é um contêiner genérico em HTML, amplamente usado para agrupar conteúdos e aplicar estilos via CSS ou manipulações via JavaScript. Ele não carrega significado semântico, ao contrário de elementos como `<article>` ou `<section>`.

(F) A unidade de medida relativa no CSS é sempre equivalente ao tamanho da fonte padrão do navegador, independentemente do contexto em que é utilizada: No CSS, unidades de medida relativas, como `rem`, `em`, `vw`, `vh`, `%`, entre outras, não são sempre equivalentes ao tamanho da fonte padrão do navegador. Por exemplo, `rem` é relativo ao tamanho da fonte do elemento raiz (`<html>`), enquanto `em` é relativo ao tamanho da fonte do elemento pai ou do próprio elemento. Outras unidades, como `vw` e `vh`, são relativas às dimensões da janela de visualização, não à fonte padrão do navegador.

(V) A propriedade z-index no CSS determina a sobreposição de elementos na página, sendo que valores negativos são permitidos e podem afetar a ordem de empilhamento: A propriedade z-index controla a ordem de empilhamento de elementos na página (eixo Z). Valores maiores posicionam o elemento acima de outros com valores menores. Valores negativos são permitidos e podem fazer com que o elemento fique abaixo de outros elementos ou até mesmo do plano de fundo, dependendo do contexto de empilhamento.

(V) A utilização do atributo alt em imagens no HTML é essencial para acessibilidade, pois fornece uma descrição textual do conteúdo da imagem para tecnologias assistivas: O atributo alt em elementos `<img>` é fundamental para acessibilidade, pois fornece uma

descrição textual da imagem que é lida por tecnologias assistivas, como leitores de tela, permitindo que pessoas com deficiência visual compreendam o conteúdo da imagem. Além disso, o alt é usado quando a imagem não carrega.

Letra d.

020. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) O setor de TI da SEMCONGER está desenvolvendo uma interface web interativa para consulta de relatórios fiscais. Para isso, os programadores precisam garantir responsividade e usabilidade da aplicação, utilizando HTML, CSS e JavaScript. A equipe de desenvolvimento discutiu diferentes abordagens para melhorar o desempenho da interface, levantar requisitos e otimizar a manipulação do DOM; analise-as.

I – O *JavaScript* permite manipular dinamicamente os elementos HTML por meio da API *Document Object Model* (DOM).

II – No CSS, a propriedade *display: inline-block*; pode ser utilizada para criar elementos alinháveis horizontalmente sem ocupar toda a largura disponível.

III – O DHTML é um conjunto de tecnologias que inclui HTML, CSS e *JavaScript* para criar páginas dinâmicas e interativas.

IV – O AJAX permite que páginas web façam requisições assíncronas ao servidor, evitando recarregamentos completos.

V – A tag `<script>` no HTML deve ser sempre posicionada dentro do `<head>` para garantir que os scripts sejam carregados antes da renderização da página.

Está correto o que se afirma apenas em:

- a) I, II e IV.
- b) I, III e IV.
- c) II, III e V.
- d) I, II, III e IV.



I – Certa. O *JavaScript* permite manipular dinamicamente os elementos HTML por meio da API *Document Object Model* (DOM): O JavaScript interage com a API do DOM para acessar, modificar, adicionar ou remover elementos HTML de forma dinâmica, permitindo a criação de interfaces interativas. Por exemplo, a manipulação do DOM é essencial para atualizar relatórios fiscais em tempo real sem recarregar a página.

II – Certa. No CSS, a propriedade *display: inline-block*; pode ser utilizada para criar elementos alinháveis horizontalmente sem ocupar toda a largura disponível: A propriedade *display: inline-block* permite que elementos sejam alinhados horizontalmente (como em linha), ocupando apenas a largura necessária para seu conteúdo, ao contrário de *display: block*, que ocupa toda a largura disponível. Essa propriedade é útil para criar layouts responsivos, como painéis ou botões em uma interface de consulta de relatórios.

III – Certa. O DHTML é um conjunto de tecnologias que inclui HTML, CSS e *JavaScript* para criar páginas dinâmicas e interativas.: DHTML (Dynamic HTML) não é uma tecnologia distinta, mas um termo que descreve a combinação de HTML, CSS e *JavaScript* para criar páginas web dinâmicas e interativas. Embora o termo seja menos comum hoje, ele ainda é válido para descrever a integração dessas tecnologias, como na manipulação de elementos para exibir relatórios fiscais de forma interativa.

IV – Certa. O AJAX permite que páginas web façam requisições assíncronas ao servidor, evitando recarregamentos completos: AJAX (Asynchronous JavaScript and XML) é uma técnica que utiliza *JavaScript* (geralmente com XMLHttpRequest ou a API fetch) para realizar requisições assíncronas ao servidor, permitindo atualizar partes específicas da página, como a exibição de novos dados fiscais, sem a necessidade de recarregar a página inteira. Isso melhora a usabilidade e o desempenho da interface.

V – Errada. A tag <script> no HTML deve ser sempre posicionada dentro do <head> para garantir que os scripts sejam carregados antes da renderização da página: Embora a tag <script> possa ser colocada no <head>, não é uma obrigatoriedade, e posicioná-la lá nem sempre é a melhor prática. Colocar scripts no <head> sem o atributo defer ou async pode bloquear a renderização da página até que o script seja carregado e executado, prejudicando o desempenho. Uma prática comum é posicionar a tag <script> no final do <body> ou usar defer/async para scripts no <head>, garantindo que o HTML seja renderizado antes da execução do *JavaScript*. Para uma interface de relatórios fiscais, otimizar o carregamento de scripts é crucial para melhorar a responsividade.

Letra d.

021. (INSTITUTO CONSULPLAN/PREFEITURA DE NOVA IGUAÇU-RJ/TÉCNICO PROGRAMADOR DE COMPUTAÇÃO/2025) O desenvolvedor da SEMCONGER precisa criar uma página dinâmica utilizando HTML, CSS e *JavaScript* para um sistema interno de gerenciamento de contratos. Ele deseja utilizar CSS para definir o layout responsivo e *JavaScript* para interatividade. A seguir está um trecho do código HTML com CSS embutido:

```
<!DOCTYPE html> <html lang="pt-br"> <head> <meta charset="UTF-8"> <meta
name="viewport" content="width=device-width, initial-scale=1.0"> <title>Contratos</
title> <style>.container { display: _____; justify-content: center; align-items: center;
height: 100vh; } </style> </head> <body> <div class="container"> <h1>Gerenciamento de
Contratos</h1> </div> </body> </html>
```

Assinale a alternativa que preenche corretamente a lacuna no código CSS, para que o div. container centralize o conteúdo tanto na horizontal quanto na vertical.

- a) flex
- b) grid

- c) block
- d) inline-block



- a) Certa. O valor `display: flex` ativa o modelo de layout Flexbox, que permite alinhar e distribuir elementos filhos de forma flexível. As propriedades `justify-content` e `align-items` são compatíveis com Flexbox, e `display: flex` centralizará o `<h1>` tanto horizontalmente (com `justify-content: center`) quanto verticalmente (com `align-items: center`) dentro do `div.container`.
- b) Errada. O valor `display: grid` ativa o modelo de layout CSS Grid, que também suporta `justify-content` e `align-items`. Ele pode centralizar o conteúdo de forma semelhante ao Flexbox. No entanto, para um caso simples como centralizar um único elemento filho (o `<h1>`), o Flexbox é geralmente mais direto e comum, enquanto o Grid é mais adequado para layouts bidimensionais complexos. Ainda assim, tecnicamente, grid também funcionaria.
- c) Errada. O valor `display: block` faz o elemento ocupar toda a largura disponível, mas não suporta `justify-content` nem `align-items`, pois essas propriedades são exclusivas de Flexbox e Grid. Portanto, `block` não centralizaria o conteúdo verticalmente ou horizontalmente como desejado.
- d) Errada. O valor `display: inline-block` faz o elemento se comportar como um bloco que flui em linha, mas também não suporta `justify-content` nem `align-items`. Além disso, não é adequado para centralizar conteúdo em um contêiner com altura definida como `100vh`.

Letra a.

022. (AVANÇA SP/UNITAU/PROGRAMADOR PLENO/2025) Em uma aplicação que utiliza CSS-in-JS com styled-components, qual abordagem é mais adequada para implementar temas dinâmicos?

- a) Usar variáveis CSS globais
- b) Implementar ThemeProvider com contexto React
- c) Definir estilos inline
- d) Criar múltiplos arquivos CSS
- e) Aplicar classes CSS diretamente no JSX



- a) Errada. Embora variáveis CSS (como `--primary-color`) sejam úteis em CSS puro, elas não se integram tão bem com o ecossistema do styled-components. Para alterar temas dinamicamente, seria necessário manipular essas variáveis via JavaScript, o que pode ser menos intuitivo e não aproveita as vantagens do styled-components, como a encapsulação de estilos e a integração com o estado da aplicação React.
- b) Certa. O ThemeProvider do styled-components é uma solução projetada especificamente para gerenciar temas dinâmicos. Ele permite definir um objeto de tema (contendo cores, tamanhos, etc.) e disponibilizá-lo para todos os componentes estilizados dentro de sua

árvore de componentes via contexto React. Para trocar o tema, basta atualizar o objeto passado ao ThemeProvider, e todos os componentes que usam theme (via props.theme) refletem a mudança automaticamente. Essa abordagem é escalável, mantém a consistência com a filosofia do styled-components e é ideal para aplicações React.

c) Errada. Estilos inline (definidos diretamente no atributo style dos elementos JSX) não são recomendados em aplicações com styled-components, pois perdem as vantagens do CSS-in-JS, como reutilização, encapsulação e suporte a temas dinâmicos. Além disso, gerenciar temas dinâmicos com estilos inline seria trabalhoso e propenso a erros, já que exigiria manipular cada elemento individualmente.

d) Errada. Criar múltiplos arquivos CSS vai contra a filosofia do CSS-in-JS, que busca manter estilos próximos ao código JavaScript e encapsulados nos componentes. Além disso, gerenciar temas dinâmicos com arquivos CSS separados exigiria carregar/descarregar arquivos dinamicamente, o que é ineficiente e não se integra bem com o styled-components.

e) Errada. Aplicar classes CSS tradicionais (definidas em arquivos.css) no JSX é uma abordagem válida em outros contextos, mas não aproveita os benefícios do styled-components, como estilos encapsulados e integração com props/state do React. Além disso, gerenciar temas dinâmicos com classes CSS exigiria manipulações complexas (ex.: alternar classes condicionalmente), sendo menos eficiente que o ThemeProvider.

Letra b.

023. (IV-UFG/PREFEITURA DE RIO BRANCO-AC/ANALISTA DE SISTEMAS – ESPECIALIZAÇÃO EM GESTÃO DE SISTEMAS COMPUTACIONAIS/2024) CSS é chamado de linguagem Cascading Style Sheet e é usado para estilizar elementos escritos em uma linguagem de marcação como HTML. O CSS separa o conteúdo da representação visual do site. Pense na decoração da sua página. Utilizando o CSS, é possível alterar a cor do texto e do fundo, fonte e espaçamento entre parágrafos. Também pode criar tabelas, usar variações de layouts, ajustar imagens para suas respectivas telas. São características da Cascading Style Sheets (CSS):

- a) definir cores, fontes e layout de apresentação do documento.
- b) descrever o estilo de documentos estruturados utilizando Java, Python e C++.
- c) permitir o aumento do código que determina o conteúdo do que vai ser apresentado na página.
- d) possibilitar criar novas folhas de estilo sem permitir a herança de outra folha de estilo já existente.



a) Certa. O CSS (Cascading Style Sheets) é projetado especificamente para controlar a apresentação visual de documentos HTML. Ele gerencia a aparência dos elementos, como cores, tipografia (fontes), espaçamento, e o layout geral da página.

- b) Errada. O CSS é usado para estilizar linguagens de marcação, como HTML e XML. Ele não é usado para estilizar linguagens de programação como Java, Python ou C++.
- c) Errada. O CSS tem a função oposta: ele separa a apresentação do conteúdo. Isso reduz a necessidade de estilos embutidos no HTML, tornando o código mais limpo e fácil de manter.
- d) Errada. A “Cascata” do CSS (Cascading) permite a herança de estilos. As regras de estilo são aplicadas em uma hierarquia, onde as regras de folhas de estilo externas, internas e embutidas podem ser combinadas e sobrescritas. Isso permite que estilos sejam reutilizados e herdados de outras folhas de estilo.

Letra a.

024. (FGV/TCE-PA/AUDITOR DE CONTROLE EXTERNO – ÁREA DE INFORMÁTICA – WEB DESIGN/2024) No Cascading Style Sheets (CSS), uma área de um documento que emprega *flexbox* é chamada de flex container. Para criar um contêiner flexível, é necessário definir uma determinada propriedade como *flex* ou *inline-flex*. Dessa forma, os elementos filhos do contêiner tornam-se itens do tipo *flex*.

Nesse caso, assinale a referida propriedade do CSS.

- a) *flex-wrap*.
- b) *display*.
- c) *container*.
- d) *flex-flow*.
- e) *flex-basis*.



Para transformar um elemento HTML em um flex container, você deve aplicar a propriedade *display* com um dos seguintes valores:

- *display: flex;*: Cria um contêiner flexível em nível de bloco.
- *display: inline-flex;*: Cria um contêiner flexível em nível de linha.

Ao fazer isso, os elementos filhos diretos desse contêiner (os flex items) se alinham e se ajustam automaticamente de acordo com as regras do modelo flexbox.

- a) Errada. Controla se os itens flexíveis devem quebrar para a próxima linha ou permanecer em uma única linha. É uma propriedade do flex container, mas não a que o define.
- b) Certa. A propriedade correta para definir o tipo de layout do elemento, incluindo flex e inline-flex.
- c) Errada. Não é uma propriedade CSS padrão do flexbox para essa finalidade.
- d) Errada. É uma propriedade de atalho que combina flex-direction e flex-wrap. Não é usada para criar o contêiner flexível.
- e) Errada. Define o tamanho inicial de um flex item antes que o espaço restante seja distribuído. É uma propriedade dos flex items, não do flex container.

Letra b.

025. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT – ÁREA 36: INFORMAÇÃO E COMUNICAÇÃO III/2025) Em CSS 3 é possível criar ou complementar conteúdos que irão compor a página do site. Uma forma de complementar um elemento com conteúdos é por meio da utilização de pseudoelementos, como *before* e *after*, na declaração da regra CSS, e da utilização da propriedade *content* em seu conteúdo.

Considerando o pseudoelemento *before*, qual opção apresenta um exemplo de aplicação para elementos *p*, com a sintaxe correta?

- a) *p:before*
- b) *p>before*
- c) *p::before*
- d) *p before*



a) Errada. Usa um único dois-pontos (:), que é a sintaxe mais antiga para pseudoelementos. Embora ainda funcione em navegadores modernos por compatibilidade, não é a sintaxe recomendada em CSS3. Válida, mas não a mais correta para CSS3.

b) Errada. O operador > é usado em CSS para selecionar filhos diretos (ex.: *p > span* seleciona um que é filho direto de <p>). Usar *p>before* é sintaticamente incorreto, pois *before* não é um elemento HTML, mas um pseudoelemento.

c) Certa. Usa dois dois-pontos (::), que é a sintaxe moderna e recomendada em CSS3 para pseudoelementos. Essa é a forma correta de aplicar o pseudoelemento::*before* a elementos <p>.

d) Errada. Um espaço entre *p* e *before* implica que *before* seria um elemento ou classe, o que não é o caso, pois *before* é um pseudoelemento. A sintaxe com espaço é inválida.

Letra c.

026. (IF SUL RIO-GRANDENSE/IF SUL RIO-GRANDENSE/PROFESSOR EBTT – ÁREA 36: INFORMAÇÃO E COMUNICAÇÃO III/2025) A função CSS *linear-gradient* é capaz de criar imagens com gradientes em páginas HTML, devendo-se informar a direção e as cores que farão parte do efeito.

Qual opção apresenta um exemplo de sintaxe correta para sua utilização?

- a) *background-color: linear-gradient(to bottom #4377FA #0537B7)*
- b) *background-color: linear-gradient(to bottom, #4377FA, #0537B7)*
- c) *background-image: linear-gradient(to bottom #4377FA #0537B7)*
- d) *background-image: linear-gradient(to bottom, #4377FA, #0537B7)*



a) Errada. A propriedade *background-color* aceita apenas cores sólidas (ex.: #4377FA, *rgb(67, 119, 250)*), não imagens ou gradientes. A função *linear-gradient* cria uma imagem, não uma cor, então aplicá-la a *background-color* é sintaticamente incorreto. Além disso, a sintaxe está faltando uma vírgula entre as cores (#4377FA e #0537B7).

- b) Errada. Embora a sintaxe do linear-gradient esteja correta (com vírgulas separando as cores), a propriedade background-color não pode ser usada com linear-gradient, pois, como mencionado, ela aceita apenas cores sólidas, não imagens.
- c) Errada. A propriedade background-image é correta para aplicar linear-gradient, pois o gradiente é considerado uma imagem. No entanto, a sintaxe do linear-gradient está errada, pois falta uma vírgula entre as cores (#4377FA e #0537B7). A sintaxe correta para linear-gradient exige vírgulas para separar as cores.
- d) Certa. Essa opção usa a propriedade background-image, que é apropriada para linear-gradient. A sintaxe do linear-gradient está correta: especifica a direção (to bottom) e as cores (#4377FA, #0537B7) separadas por vírgulas.

Letra d.

027. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) Em CSS, a propriedade que é usada para definir a largura de um elemento é denominada

- a) Length.
- b) Size.
- c) Breadth.
- d) Width.
- e) Measure.



- a) Errada. Não é uma propriedade CSS. "Length" refere-se a um tipo de valor em CSS (como px, rem, %), mas não é usada para definir largura.
- b) Errada. Não é uma propriedade CSS para largura. A propriedade size existe em contextos específicos (como em @page para impressão), mas não para definir a largura de elementos.
- c) Errada. Não é uma propriedade CSS. "Breadth" não é um termo usado em CSS.
- d) Certa. É a propriedade correta. A propriedade width define a largura de um elemento, aceitando valores como px, %, vw, auto, etc. Exemplo: width: 200px;.
- e) Errada. Não é uma propriedade CSS. Não há propriedade chamada measure em CSS.

Letra d.

028. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) A propriedade CSS que é usada para alterar a cor de fundo de um elemento é denominada:

- a) background-color.
- b) bgcolor.

- c) color.
- d) background-style.
- e) color-background.



- a) Certa. É a propriedade correta. A propriedade background-color define a cor de fundo de um elemento, aceitando valores como nomes de cores (ex.: blue), códigos hexadecimais (ex.: #1E90FF), RGB, etc. Exemplo: background-color: #1E90FF;.
- b) Errada. Não é uma propriedade CSS. bgcolor era um atributo obsoleto em HTML4, mas não é usado em CSS.
- c) Errada. Define a cor do texto ou do conteúdo de um elemento, não a cor de fundo. Exemplo: color: white;.
- d) Errada. Não existe em CSS. Há propriedades como background (que pode incluir cor, imagem, etc.), mas background-style não é válida.
- e) Errada. Não é uma propriedade CSS. É uma combinação incorreta de termos.

Letra a.

029. (UECE-CEV/PGE-CE/TÉCNICO DE REPRESENTAÇÃO JUDICIAL – TECNOLOGIA DA INFORMAÇÃO – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS/2025) Assinale a opção que corresponde ao seletor CSS que pode ser utilizado para selecionar todos os elementos <p> que são filhos diretos de <div>.

- a) div p
- b) div > p
- c) div + p
- d) div.p
- e) #div p



- a) Errada. Este é o seletor de descendência. Ele seleciona todos os elementos <p> que estão dentro de um <div>, independentemente de serem filhos diretos ou descendentes mais profundos (ex.: dentro de outros elementos aninhados). Como a questão especifica “filhos diretos”, essa opção não é a mais adequada, pois seleciona mais do que apenas filhos diretos.
- b) Certa. Este é o seletor de filho direto (combinador >). Ele seleciona apenas os elementos <p> que são filhos diretos de um <div>, ou seja, estão imediatamente dentro do <div> sem outros elementos intermediários.
- c) Errada. Este é o seletor de irmão adjacente (combinador +). Ele seleciona o elemento <p> que é o próximo irmão imediato de um <div>, ou seja, o <p> que vem logo após um <div> no mesmo nível hierárquico.

d) Errada. Essa sintaxe indica um elemento `<div>` com a classe `p` (ex.: `<div class="p">`). Não seleciona elementos `<p>` dentro de `<div>`, mas sim um `<div>` com a classe `p`. Isso não atende à exigência da questão.

e) Errada. Essa sintaxe usa um seletor de ID (`#div`), que seleciona um elemento com `id="div"`. Ela seleciona todos os elementos `<p>` que são descendentes (não necessariamente filhos diretos) de um elemento com `id="div"`. Como a questão pede filhos diretos e não especifica um ID, essa opção não é adequada.

Letra b.

030. (CESPE-CEBRASPE/TCE-AC/ANALISTA DE TECNOLOGIA DA INFORMAÇÃO – ÁREA: SISTEMAS DE INFORMAÇÃO/2024) Em relação ao desenvolvimento de sistemas *web*, julgue o próximo item.

“O uso extensivo de pseudoelementos no CSS3 pode melhorar a estrutura semântica de um documento HTML.”



Pseudoelementos do CSS3 (como `::before`, `::after`, `::first-letter`) servem para aplicar estilo ou inserir conteúdo meramente decorativo, mas não alteram nem melhoram a estrutura semântica do HTML.

A semântica de um documento é definida pelas tags HTML adequadas (`<header>`, `<article>`, `<section>`, etc.), não por CSS.

Errado.

REFERÊNCIAS

CSS GRID GARDEN. Disponível em: <https://cssgridgarden.com/>

FLEXBOX DEFENSE. Disponível em: <http://www.flexboxdefense.com/>

FLEXBOX FROGGY. Disponível em: <https://flexboxfroggy.com/#pt-br>

GODADDY. *Você sabe o que é CSS? Entenda como funciona e para que serve.* [S. l.], 2023. Disponível em: <https://www.godaddy.com/resources/br/artigos/voce-sabe-o-que-e-css-entenda-como-funciona-e-para-que-serve>. Acesso em: 16 ago. 2025.

HOSTINGER. *O que é CSS? Guia básico de CSS.* [S. l.], 2025. Disponível em: <https://www.hostinger.com/br/tutoriais/o-que-e-css-guia-basico-de-css>. Acesso em: 17 ago. 2025.

LOCAWEB. *O que é CSS?*. [S. l.], 2024. Disponível em: <https://www.locaweb.com.br/blog/temas/codigo-aberto/css/>. Acesso em: 17 ago. 2025.

MDN WEB DOCS. CSS. [S. l.], ?. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/CSS>. Acesso em: 20 ago. 2025.

W3SCHOOLS. *CSS Tutorial.* [S. l.], 2025. Disponível em: <https://www.w3schools.com/css/>. Acesso em: 20 ago. 2025.

WEB.DEV. *Aprenda CSS.* [S. l.], ?. Disponível em: <https://web.dev/learn/css?hl=pt>. Acesso em: 18 ago. 2025.

Abra



caminhos



crie

futuros

gran.com.br

